Telecommunications Industry Association

TIA STANDARD

Project 25

Digital Radio Over-The-Air-Rekeying (OTAR) Messages and Procedures

TIA-102.AACA-C
(Revision of TIA-102.AACA)

August 2023

TIAONLINE.ORG

NOTICE

TIA Engineering Standards and Publications are designed to serve the public interest through eliminating misunderstandings between manufacturers and purchasers, facilitating interchangeability and improvement of products, and assisting the purchaser in selecting and obtaining with minimum delay the proper product for their particular need. The existence of such Standards and Publications shall not in any respect preclude any member or non-member of TIA from manufacturing or selling products not conforming to such Standards and Publications. Neither shall the existence of such Standards and Publications preclude their voluntary use by Non-TIA members, either domestically or internationally.

Standards and Publications are adopted by TIA in accordance with the American National Standards Institute (ANSI) patent policy. By such action, TIA does not assume any liability to any patent owner, nor does it assume any obligation whatever to parties adopting the Standard or Publication.

This Standard does not purport to address all safety problems associated with its use or all applicable regulatory requirements. It is the responsibility of the user of this Standard to establish appropriate safety and health practices and to determine the applicability of regulatory limitations before its use.

Any use of trademarks in this document are for information purposes and do not constitute an endorsement by TIA or this committee of the products or services of the company.

(From Project Proposal No. TIA-PN-102.AACA-C, formulated under the cognizance of the TIA TR-8 Mobile and Personal Private Radio Standards, TR-8.3 Subcommittee on Encryption).

Published by
©TELECOMMUNICATIONS INDUSTRY ASSOCIATION
Technology and Standards Department
1310 N. Courthouse Road, Suite 890
Arlington, VA 22201 U.S.A.

NOTICE OF COPYRIGHT

PATENT IDENTIFICATION

The reader's attention is called to the possibility that compliance with this document may require the use of one or more inventions covered by the patent rights.

By publication of this document, no position is taken with respect to the validity of those claims or any patent rights in connection therewith. The patent holders so far identified have, we believe, filed statements of willingness to grant licenses under those rights on reasonable and nondiscriminatory terms and conditions to applicants desiring to obtain such licenses for the purpose of implementing this TIA Publication. Details regarding the filed statements may be obtained from TIA.

The following patent holders and patents have been identified in accordance with the TIA intellectual property rights policy:

- No patents have been identified

TIA shall not be responsible for identifying patents for which licenses may be required by this document or for conducting inquiries into the legal validity or scope of those patents that are brought to its attention.

NOTICE OF DISCLAIMER AND LIMITATION OF LIABILITY

The document to which this Notice is affixed (the "Document") has been prepared by one or more Engineering Committees or Formulating Groups of the Telecommunications Industry Association ("TIA"). TIA is not the author of the Document contents, but publishes and claims copyright to the Document pursuant to licenses and permission granted by the authors of the contents.

TIA Engineering Committees and Formulating Groups are expected to conduct their affairs in accordance with the TIA Procedures for American National Standards and TIA Engineering Committee Operating Procedures, the current and predecessor versions of which are available at Standards Procedures and Guidelines. TIA's function is to administer the process, but not the content, of document preparation in accordance with the Manual and, when appropriate, the policies and procedures of the American National Standards Institute ("ANSI"). TIA does not evaluate, test, verify or investigate the information, accuracy, soundness, or credibility of the contents of the Document. In publishing the Document, TIA disclaims any undertaking to perform any duty owed to or for anyone.

If the Document is identified or marked as a project number (PN) document, or as a standards proposal (SP) document, persons or parties reading or in any way interested in the Document are cautioned that: (a) the Document is a proposal; (b) there is no assurance that the Document will be approved by any Committee of TIA or any other body in its present or any other form; (c) the Document may be amended, modified or changed in the standards development or any editing process.

The use or practice of contents of this Document may involve the use of intellectual property rights ("IPR"), including pending or issued patents, or copyrights, owned by one or more parties. TIA makes no search or investigation for IPR. When IPR consisting of patents and published pending patent applications are claimed and called to TIA's attention, a statement from the holder thereof is requested, all in accordance with the Manual. TIA takes no position with reference to, and disclaims any obligation to investigate or inquire into, the scope or validity of any claims of IPR. TIA will neither be a party to discussions of any licensing terms or conditions, which are instead left to the parties involved, nor will TIA opine or judge whether proposed licensing terms or conditions are reasonable or non-discriminatory. TIA does not warrant or represent that procedures or practices suggested or provided in the Manual have been complied with as respects the Document or its contents.

If the Document contains one or more Normative References to a document published by another organization ("other SSO") engaged in the formulation, development or publication of standards (whether designated as a standard, specification, recommendation or otherwise), whether such reference consists of mandatory, alternate or optional elements (as defined in the TIA Procedures for American National Standards) then (i) TIA disclaims any duty or obligation to search or investigate the records of any other SSO for IPR or letters of assurance relating to any such Normative Reference; (ii) TIA's policy of encouragement of voluntary disclosure (see TIA Procedures for American National Standards Annex C.1.2.3) of Essential Patent(s) and published pending patent applications shall apply; and (iii) Information as to claims of IPR in the records or publications of the other SSO shall not constitute identification to TIA of a claim of Essential Patent(s) or published pending patent applications.

TIA does not enforce or monitor compliance with the contents of the Document. TIA does not certify, inspect, test or otherwise investigate products, designs or services or any claims of compliance with the contents of the Document.

ALL WARRANTIES, EXPRESS OR IMPLIED, ARE DISCLAIMED, INCLUDING WITHOUT LIMITATION, ANY AND ALL WARRANTIES CONCERNING THE ACCURACY OF THE CONTENTS, ITS FITNESS OR APPROPRIATENESS FOR A PARTICULAR PURPOSE OR USE, ITS MERCHANTABILITY AND ITS NONINFRINGEMENT OF ANY THIRD PARTY'S INTELLECTUAL PROPERTY RIGHTS. TIA EXPRESSLY DISCLAIMS ANY AND ALL RESPONSIBILITIES FOR THE ACCURACY OF THE CONTENTS AND MAKES NO REPRESENTATIONS OR WARRANTIES REGARDING THE CONTENT'S COMPLIANCE WITH ANY APPLICABLE STATUTE, RULE OR REGULATION, OR THE SAFETY OR HEALTH EFFECTS OF THE CONTENTS OR ANY PRODUCT OR SERVICE REFERRED TO IN THE DOCUMENT OR PRODUCED OR RENDERED TO COMPLY WITH THE CONTENTS. TIA SHALL NOT BE LIABLE FOR ANY AND ALL DAMAGES, DIRECT OR INDIRECT, ARISING FROM OR RELATING TO ANY USE OF THE CONTENTS CONTAINED HEREIN, INCLUDING WITHOUT LIMITATION ANY AND ALL INDIRECT, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES (INCLUDING DAMAGES FOR LOSS OF BUSINESS, LOSS OF PROFITS, LITIGATION, OR THE LIKE), WHETHER BASED UPON BREACH OF CONTRACT, BREACH OF WARRANTY, TORT (INCLUDING NEGLIGENCE), PRODUCT LIABILITY OR OTHERWISE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. THE FOREGOING NEGATION OF DAMAGES IS A FUNDAMENTAL ELEMENT OF THE USE OF THE CONTENTS HEREOF, AND THESE CONTENTS WOULD NOT BE PUBLISHED BY TIA WITHOUT SUCH LIMITATIONS.

Contents

Contents

Contents

Contents

Contents

Figures

Figures

TABLES

Figures

FOREWORD

(This foreword is not part of this document)

This document was created in response to a request by the APCO/NASTD/FED1 Project 25 Steering Committee as provided for in a Memorandum of Understanding (MOU) dated April 1992 and amended December 1993. The MOU states APCO/NASTD/FED shall devise a common system standard for digital public safety communications (the Standard) commonly referred to as “Project 25” or “P25”, and that TIA, as agreed to by the membership of the appropriate TIA Engineering Committee, will provide technical assistance in the development of documentation for the Standard. If the abbreviation “P25” or the wording “Project 25” appears on the cover sheet of this document when published or in the title of any TIA-102 document or Telecommunications Service Bulletin (TSB) referenced herein, that indicates the APCO/NASTD/FED Project 25 Steering Committee has adopted the document as part of the Standard. The appearance of the abbreviation “P25” or the wording “Project 25” on the cover sheet of this document or in the title of any document referenced herein should not be interpreted as limiting the applicability of the information contained in any document to “P25” or “Project 25” implementations exclusively.

This document has been developed by the formulating group TR8.3 Encryption Subcommittee with inputs from the APCO Project 25 Interface Committee (APIC), the APIC Encryption Task Group, and TIA Industry members.

This document is being published because it is felt that there is an urgent need for technical information on the emerging digital techniques for Land Mobile Radio Service.

This document describes a method of sending the encryption keys and other related key management messages through the Common Air Interface (CAI) in such a way that they are protected from unauthorized disclosure and, in some cases, unauthorized modification.

This TIA document presents a design for a Key Management and OTAR Protocol which was recommended by TIA to APCO/NASTD/FED as being suitable for use as part of their standard for a digital public safety radio system.

Figures

PATENT IDENTIFICATION

The reader's attention is called to the possibility that compliance with this document may require the use of one or more inventions covered by patent rights.

By publication of this document no position is taken with respect to the validity of those claims or any patent rights in connection therewith. The patent holders so far identified have, we believe, filed with APCO/NASTD/FED statements of willingness to grant licenses under those rights on reasonable and nondiscriminatory terms and conditions to applicants desiring to obtain such licenses. Details may be obtained from APCO/NASTD/FED.

The following patent holders and patents have been identified in accordance with the TIA intellectual property rights policy:

- No patents have been identified

TIA shall not be responsible for identifying patents for which licenses may be required by this document or for conducting inquiries into the legal validity or scope of those patents that are brought to its attention.

1 SCOPE

The TIA-102 suite of documents describes the interfaces associated with a system for public safety land mobile radio communications. These systems include subscriber units, base stations and other fixed equipment. The term Subscriber Unit (SU) includes portable radios for handheld operation and mobile radios for vehicular operation. The base stations are used for geographically fixed installations. Other fixed equipment is used for wide area operation and console operator positions. Computer equipment may be used to interface between each of these equipment items. A Common Air Interface (CAI), defined in [4], allows these SUs to send and receive digital information over a radio channel.

Many of the parts of a public safety Land Mobile Radio (LMR) communications system use encryption to protect the information which is sent through the system. The encryption algorithms require keys in order to protect the confidentiality of this information. The process by which these encryption keys are generated, stored, protected, transferred, loaded, used and destroyed is known as key management. These keys shall be protected from inadvertent disclosure and require updating or replacement in order to maintain system security. Key distribution is often accomplished manually. However, the most convenient way to distribute keys is to electronically send the keys from a key management facility to the destination equipment. This involves sending keys over the CAI and this procedure is referred to as Over-The-Air-Rekeying (OTAR). OTAR is a method of protecting and sending the encryption keys and other related key management messages through the CAI in such a way that they are protected from inadvertent disclosure and, in some cases, unauthorized modification.

This document defines the Over-The-Air-Rekeying protocol, messages and procedures designed to promote interoperability between various pieces of compliant radio equipment, regardless of manufacturer.

The scope of this document is to address methods of OTAR and associated over the air key management functions in a multi-key system. The primary objective of this document is to enable subscriber units and systems which conform to this document to be interoperable to the extent that keys can be passed via the CAI between communicating units and encrypted communications can result. It is a further objective that conformance to this document shall enable the interoperability of subscriber units and systems provided by different vendors, and operated by different agencies. This enables effective and reliable intra-agency and inter-agency encrypted communications over the air. This is in conformance to the Statement of Requirements.

Reference [5] defines a set of Algorithm ID values for encryption algorithm interoperability such that encrypted messages, either voice or data, can be encrypted and decrypted consistently between endpoints. Interoperability is obtained by using the standardized encryption algorithms identified by those Algorithm IDs. Conversely, the use of non-standardized or proprietary Algorithm IDs shall not be deemed as

interoperable. Therefore, the use of the OTAR protocol to exchange keys for non-standardized or proprietary Algorithm IDs shall preclude compliance with this document for those messages carrying non-standardized or proprietary Algorithm IDs.

Figure 1 shows a typical (example) LMR general system model with system elements that may include key management functions (such as, portable or mobile radios, RF system gateways, RF system controllers and consoles) that should be compatible with the core OTAR functions. OTAR functions include the protection of keys to maintain their confidentiality and integrity during transmission. Encryption of keys while in storage and during transit helps maintain overall system security and confidentiality. Integrity of keys is required to prevent unauthorized insertion, deletion, or modification of keys.

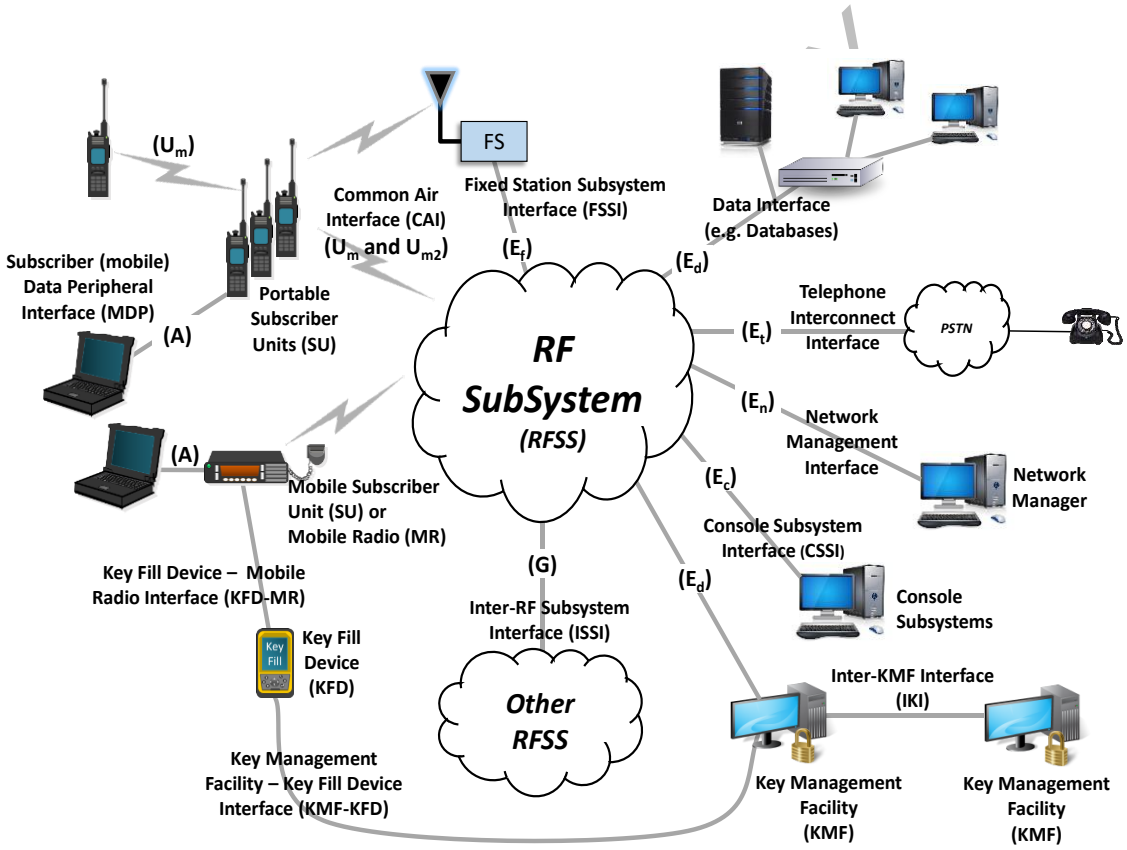


Figure 1 - Typical LMR General System Model

Keys are managed by a Key Manager function which is included in a Key Management Facility (KMF). This KMF system element maintains a link through a system's infrastructure to an U_m air interface as defined in the General System Model. The KMF performs most of its functions through the exchange of Key Management Messages (KMMs) with other system elements. Figure 1 shows a typical (example) placement of the KMF. The KMF functionality may also reside in other system elements, such as, the console or a network management controller. The protocols

for these interfaces are not defined and are beyond the scope of this document. Multiple KMFs may exist within one RF sub-system concurrently.

The Phase 2 TDMA standard defines a new air interface reference point designated as U_{m2}. This interface conveys encrypted voice messages encrypted with the same algorithms and keys as the U_m interface. The initial version of U_{m2} does not define packet data transmissions, so the OTAR functions are to be conveyed through the U_m interface to subscriber units.

1.1 Introduction

The concept of OTAR enables key management through transmittal of Key Management Messages (KMMs) between a single Key Management Facility (KMF) and one or more Subscriber Units (SU). OTAR provides the capability to dynamically populate and modify key management parameters within the SU as necessary.

This document defines the protocols and procedures for providing OTAR and related over-the-air key management services. The SU management concepts, protocols and supporting messages that are defined in this document are intended for use in SUs that conform to Land Mobile Radios (LMR) standards. The philosophy used in defining protocols and messages is to define many of the possible OTAR and related over the air key management procedures that may be implemented. OTAR messages and procedures, when implemented, shall conform to those messages and procedures defined within this document.

In a simple example of a system employing OTAR, an SU is initially manually loaded with a Unique Key Encryption Key (UKEK) per supported algorithm, and may also be loaded with one or more Traffic Encryption Keys (TEKs) as part of the same procedure. UKEKs are used to encrypt keys in transport. TEKs are used to encrypt end-to-end voice and data traffic.

When a TEK or groups of TEKs approach the end of their cryptoperiod, new TEKs are sent from the KMF to the SU. These TEKs are encrypted by the KMF using the UKEK held by that SU. The encrypted TEKs are then inserted into KMMs and transmitted to the SU via the air interface.

When the SU receives the KMM, it uses its Unique KEK to decrypt the new TEKs. Once an SU has received replacement keys, it may be commanded by the KMF through a changeover procedure to begin using these new keys.

For systems that support group OTAR[1]; a group of SUs can be assigned a Common KEK (CKEK), and a cryptonet consisting of units which hold this CKEK is formed. In this case, new TEKs, encrypted with this CKEK, can be sent to the SUs with a single message addressed to the group rather than individually. SUs that have been

[1] Note that Group Data is currently not a standardized feature for Trunked systems.

programmed with the same Group RSI are considered to be members of the same group. A KMM addressed to a Group RSI is known as group rekeying.

In order to implement a realistic OTAR system, many other key management services are necessary. These are discussed in detail in later sections of this document.

This document specifies the messages and procedures used in the OTAR of SUs. The transport used for transferring KMMs over the air is defined in Section 12. Most of the messages defined in this document require encryption, while a few may be sent unencrypted. Section 13 specifies the security requirements for KMMs.

Figure 2 is an example of OTAR key material distribution in a Trunked or Conventional system. Key material may be distributed from a centralized source or key material may be distributed from one KMF to one or more KMFs (reference [8]). The distribution of key material may be performed using the IKI. The KMF is a function which provides OTAR and related key management services to end user devices.

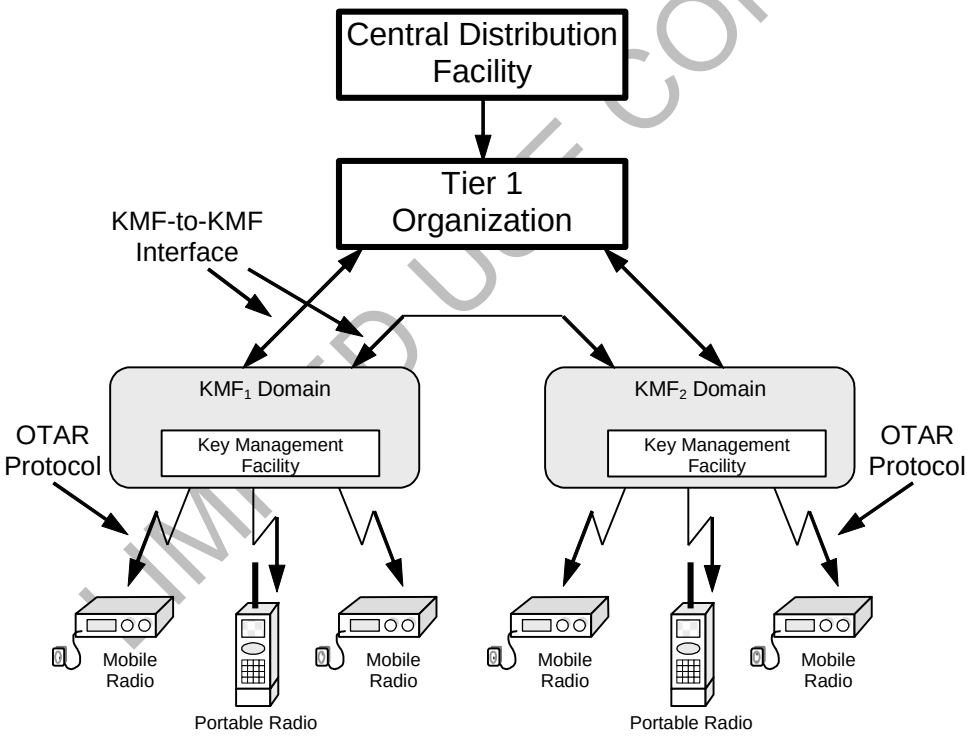


Figure 2 - Secure Subscriber Unit System Level Diagram

A key manager function at the KMF provides subscriber units with OTAR and other over the air key management services. All subscriber units which implement the procedures outlined in this document shall be interoperable with the KMF using the procedures as defined herein. The role of the KMF is defined in Section 6.6.

NOTE: The National Institute of Standards and Technology (NIST) withdrew Federal Information Processing Standard (FIPS)-46, Data Encryption Standard (DES) as a standard on May 19, 2005, citing the weakness of DES and the availability of the current federal standard specified in FIPS-197, Advanced Encryption Standard (AES). DES has since been deprecated in TIA-102 (P25) standards and is maintained in the standard for the support of (to enable) P25 AES encryption compliant equipment to also include DES for the purposes of interoperability with legacy DES only products.

2 REVISION HISTORY

SP-4828-RV1, September 2014, Merged OTAR Protocol document TIA-102.AACA with OTAR Operational Description document TIA-102.AACB, addendum TIA-102.AACA-1, and addendum TIA-102.AACA-2 to form OTAR Messages and Procedures document.

SP-4828-RV2, September 2020, Update of OTAR Messages and Protocol document ANSI/TIA-102.AACA-A to ANSI/TIA-102.AACA-B based on comments in “19-017-R15 TR8.3 Combined comments on AACA-A”.

SP-4828-RV3, April 2023, Update of OTAR Messages and Protocol document TIA-102.AACA-B to TIA-102.AACA-C to add Cipher Based Message Authentication Code (CMAC) as an additional enhanced message authentication method.

3 REFERENCES

The following normative and informative references, through reference in this text constitute provisions of this document. At the time of publication, the editions indicated were valid. All documents are subject to revision, and parties to agreements based on this document are encouraged to investigate the possibility of applying the most recent editions of the documents indicated below. ANSI and TIA maintain registers of currently valid national standards published by them.

3.1 Normative References:

[1] *Deleted*
[2] *Project 25 Block Encryption Protocol,* TIA-102.AAAD-A, August 2009
[3] *Deleted*
[4] *Project 25 FDMA Common Air Interface,* TIA-102.BAAA-A, September 2003
[5] *Project 25 Common Air Interface Reserved Values,* TIA-102.BAAC-C, April 2011
[6] *Project 25 Packet Data Specification,* TIA-102.BAEB-A, March 2005
[7] *Project 25 Data Overview and Specification,* TIA-102.BAEA-B, June 2012
[8] *Project 25 KMF to KMF Interface,* TIA-102.BAKA, April 2012
[9] *Project 25 TCP/UDP Port Number Assignments,* TIA-102.BAJD, October 2010

3.2 Informative References:

[10] *Data Encryption Standard,* NIST, FIPS Publication 46-3, October 25, 1999
[11] *DES Modes of Operation,* NIST, FIPS Publication 81, December 2, 1980
[12] *Advanced Encryption Standard,* FIPS Publication 197, November 26, 2001
[13] Special Publication (SP) 800-38A *Recommendation for Block Cipher Modes of Operation Methods and Techniques,* NIST, December 2001
[14] *Special Publication (SP) 800-38F Recommendation for Block Cipher Modes of Operation: Methods for Key Wrapping,* NIST, December 2012
[15] *Triple Data Encryption Algorithm Modes of Operation,* ANSI X9.52 – 1998, July 29, 1998
[16] *Data Encryption Algorithm,* ANSI, ANSI X3.92 - 1981
[17] *Data Encryption Algorithm - Modes of Operation,* ANSI, ANSI X3.106 - 1981
[18] IETF RFC 791, *Internet Protocol*, September 1981
[19] IETF RFC 768*, User Datagram Protocol*, August 1980
[20] *TIA-102 Documentation Suite Overview,* TSB 102-B, June 2012
[21] *Digital Land Mobile Radio – Key Fill Device (KFD) Interface Protocol* TIA-102.AACD, February 2005
[22] *Link Layer Authentication,* TIA-102.AACE-A, April 2011
[23] *Project 25 Packet Data Logical Link Control Procedures,* TIA-102.BAED, October 2013

[24] *Special Publication (SP) 800-108r1, Recommendation for Key Derivation Using Pseudorandom Functions*, August 2022
[25] *IETF RFC 4634, US Secure Hash Algorithms (SHA and HMAC-SHA), July 2006*
[26] *Special Publication (SP) 800-38B, Recommendation for Block* Cipher Modes of Operation: The CMAC Mode for Authentication, May 2005

4 DEFINITIONS

Algorithm ID	An 8-bit identifier for an encryption algorithm. Algorithm IDs for standardized crypto algorithms are listed in [5].
Channel	A Channel consists of the transmit frequency, receive frequency and associated SU parameters (scan mode, squelch, etc.). The channel may optionally include an algorithm ID and key ID to be used when the channel is in use.
CKEK	A Common Key Encryption Key used for encrypting other keys when addressed to a specific group of target devices.
Cryptographic Variable	A parameter used in conjunction with a cryptographic algorithm that is used to perform a cryptographic transformation. "Key" may be used for short.
Crypto Group (CG)	A grouping of one or more keysets consisting of the same type of key (i.e. TEKs or KEKs). Only one keyset within a Crypto Group is active at any given time.
Cryptoperiod	The time span during which an encryption key or keyset remains active. The cryptoperiod of a key is the cryptoperiod of the keyset in which it resides.
Delivery Type	Supported delivery types for OTAR are Data Link Dependent (DLD) and Data Link Independent (DLI). DLD applies to the CAI Data Bearer Service as defined in [6] and is suitable for Conventional OTAR. DLI applies to the IP Data Bearer Service as defined in [6] and is suitable for Trunked or Conventional OTAR.
Group Service	A key management service which is provided to a Key Management Group.
IKI	Inter-KMF Interface refers to the KMF-to-KMF interface defined in [8].
Inner layer encryption	Inner layer encryption is the protection of a key carried by a KMM.
Key	A parameter used in conjunction with a cryptographic algorithm that is used to perform a cryptographic transformation. Also called a cryptographic variable.
Key Encryption Key	A key used only to encrypt other keys, a KEK.

Key ID	The Key ID is a 16-bit identifier for an encryption key. The combination of a Key ID and an Algorithm ID uniquely identify a key within the KMF or subscriber units.
Key Management Facility	A Key Management Facility is responsible for providing Over-The-Air-Rekeying and related key management services to the subscriber units.
Key Management Group	A Key Management Group is two or more SUs that respond to the same Group Radio Set Identifier. The members of a Key Management Group have one or more keysets in common and are managed as a single subscriber by the KMF.
Key Management Message	The Key Management Message (KMM) is the message protocol used to support the over-the-air rekeying (OTAR) and manual rekeying architectures.
Key Management Procedure	A key management procedure is an exchange of messages that intend to change or obtain information regarding a key management state.
Key Management Service	A key management service is a set of key management procedures as conducted between a KMF and an OTAR endpoint.
Key Wrapping	Key wrapping is a process for protecting a key.
Keyset	A keyset is a group of keys that can be managed (e.g. activated, deactivated, etc.) as a single entity.
Keyset ID	A Keyset ID is an indexing parameter used to identify a specific set of keys (keyset) within a Crypto Group. There are a maximum of 16 keysets per Crypto Group.
Link Layer Authentication	LLA is an exchange of challenge and response messages performed on the air interface in order to authenticate a subscriber or FNE as described in reference [22]. LLA is also referred to as “subscriber authentication”.

Logical Message	A PDU of the sending or receiving SAP. It may be fragmented prior to being encapsulated in one or more data packets for transmission over the CAI. A “logical message” is synonymous with a “data message” as defined in [4].
Message Authentication	Integrity protection of a KMM by including a Message Authentication Code (MAC), calculated using a common secret key.
Outer layer encryption	Outer layer encryption is the protection of an entire KMM, which may or may not carry a key.
Procedure	A Procedure is a sequence of key management messages provided in succession.
Rekey	Rekey is the process of preparing, sending (if necessary) and loading of encryption keys into a unit for use in the current or future cryptoperiod. The process may be done over the air or by a direct physical connection to a key fill device.
Response Kind	The response kind field of a message determines the kind of response that the message is required to have. A message may have no response (Response Kind None (1)), a response after a delay (Response Kind Delayed (2)) or an immediate response (Response Kind Immediate (3)).
SLN	The SLN (Storage Location Number) is a reference (along with a Keyset ID) used to uniquely identify a key‘s logical position within any of the available Crypto Groups.
Subscriber Authentication	See Link Layer Authentication. Not to be confused with Message Authentication.
Traffic Encryption Key	A key used to encrypt voice traffic, data traffic, provide KMM outer layer protection, or message authentication; a TEK.
UKEK	A Unique Key Encryption Key used for encrypting other keys when addressed to an individual target device.

5 ABBREVIATIONS

ACK	Acknowledgment
AES	Advanced Encryption Standard
ALGID	Algorithm Identifier
ANSI	American National Standards Institute
CAI	Common Air Interface
CBC	Cipher Block Chaining
CBC-MAC	Cipher Block Chaining-Message Authentication Code
CFB	Cipher Feedback
CMAC	Cipher-based Message Authentication Code
$CIPH_K$	Forward cipher function (encryption) using key K
$CIPH^{-1}_K$	Inverse cipher function (decryption) using key K
CKEK	Common Key Encryption Key
CRC	Cyclic Redundancy Checksum
CSS	Channel Selector Switch
DES[2]	Data Encryption Standard
DLD	Data Link Dependent
DLI	Data Link Independent
ES	Encryption Synchronization
ECB	Electronic Codebook
EDAC	Error Detection and Correction
ESYNC	Encryption Synchronization
FIPS	Federal Information Processing Standards
FNE	Fixed Network Equipment
ID	Identifier
IV	Initialization Vector
KDF	Key Derivation Function
KEK	Key Encryption Key
Key ID	Key Identifier
KFD	Key Fill Device
KM	Key Management
KMF	Key Management Facility
KMG	Key Management Group
KMM	Key Management Message
KSID	Keyset Identifier
LLA	Link Layer Authentication
LLID	Logical Link Identifier
LS	Least Significant
LSB	Least Significant Bit
MAC	Message Authentication Code
MI	Message Indicator
MN	Message Number
MNL	Last Message Number
MNP	Message Number Period

[2] DES is only utilized for backward compatibility, see NOTE in section 1.1.

MNR	Message Number Received
MS	Most Significant
MSB	Most Significant Bit
NIST	National Institute of Standards and Technology
OFB	Output FeedBack mode of operation
OSI	Open Systems Interconnection
OTAR	Over-The-Air-Rekeying
PDU	Packet Data Unit
PRF	Pseudo Random Function
PSL	Physical Storage Location
PSTN	Private Switched Telephone Network
PTT	Push To Talk
RF	Radio Frequency
RFSS	Radio Frequency Sub-System
RSI	Radio Set Identifier
SAP	Service Access Point
SLN	Storage Location Number
SU	Subscriber Unit
TBD	To Be Determined
TDEA	Triple Data Encryption Algorithm
TEK	Traffic Encryption Key
TG	Talk Group
TGID	Talk Group Identifier
TOD	Time Of Day
UKEK	Unique Key Encryption Key
UTD	Unable To Decrypt

6 KEY MANAGEMENT OVERVIEW

This section gives a description of a LMR key management approach and provides an overview of the various procedures provided. The procedures defined in this document are intended to be utilized over the air using the communication channel defined in the LMR General System Model as the U_m interface.

The protocols used to exchange the Key Management messages defined in this document rely upon the Packet Data Service defined in [7]. The Packet Data Service includes both a CAI Data Bearer Service and an IP Data Bearer Service. The IP Data Bearer Service uses either the SCEP or the SNDCP protocol as defined in [6] across the CAI.

This document defines Data Link Dependent (DLD) and Data Link Independent (DLI) key management. DLD key management relies on the CAI Data Bearer Service. DLI key management relies on the IP Data Bearer Service and is especially useful in the Trunked FNE Data configuration where the CAI Data Bearer Service is not available. To maintain backward compatibility in other configurations, DLI key management is utilized in conjunction with the IP Data Bearer Service and the SNDCP protocol; DLD key management is utilized in all other cases.

The Packet Data Service also supports confirmed and unconfirmed data conveyance across the CAI. Key Management Messages (KMM) use the unconfirmed data mode transmission format for all group broadcast messages, while KMMs that are point to point messages between the SU and the KMF can use either the unconfirmed data mode or the confirmed data mode. The choice of using confirmed or unconfirmed data mode transmissions for non-group broadcast messages is determined on a per message basis.

Figure 3 shows a simplified OSI reference model and a simplified CAI Data Bearer Service for Data Link Dependent (DLD) key management. This figure identifies the relationships between the CAI Data Bearer Service and the key management protocols specified herein. As seen in the shaded area of the figure, key management services such as OTAR, key encryption and authentication are defined as part of an application layer process. This application layer process makes use of the CAI Data Bearer Service for data transmission, KMM encryption, Error Detection and Correction (EDAC), modulation, etc.

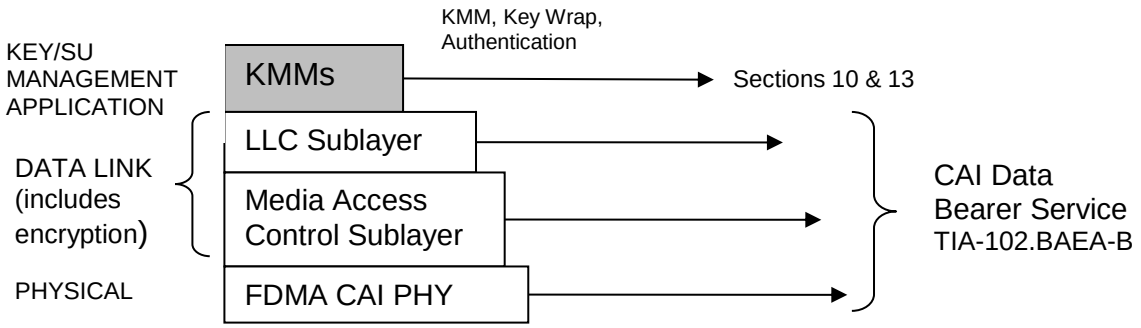


Figure 3 - Data Link Dependent OSI Reference Model for SUs

Figure 4 shows a simplified OSI reference model and a simplified IP Data Bearer Service for Data Link Independent (DLI) key management. This figure identifies the relationships between the IP Data Bearer Service and the key management protocols. As with DLD key management, the application layer process makes use of the data bearer service for data transmission, Error Detection and Correction (EDAC), modulation, etc. However, the application does not rely on the IP Data Bearer Service for KMM encryption; the Key Management application performs both key and KMM encryption.

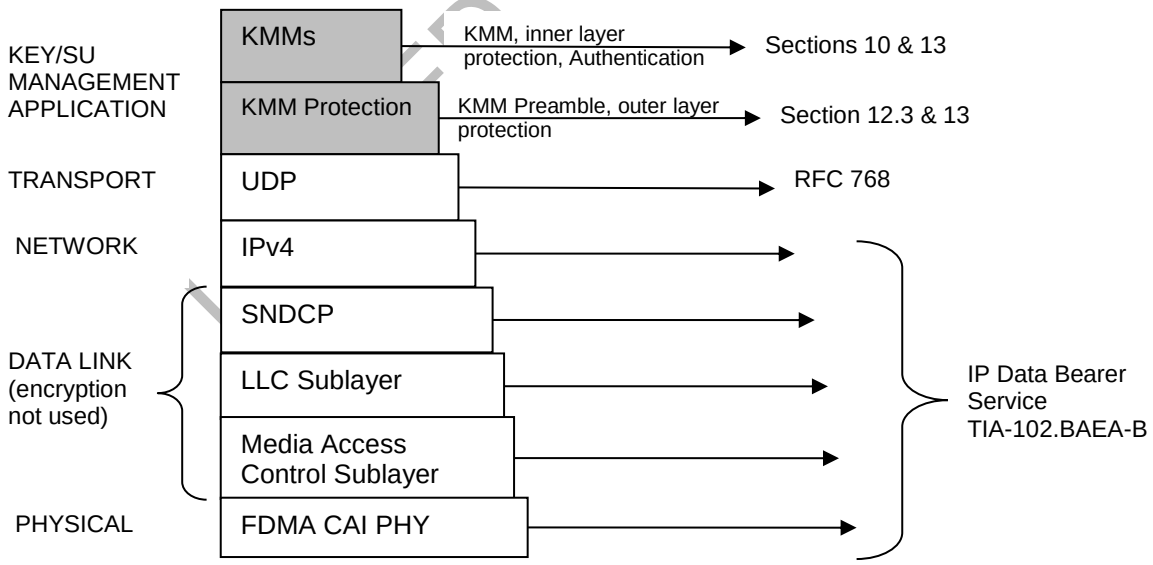


Figure 4 - Data Link Independent OSI Reference Model for SUs

The OTAR protocol and key management procedures are defined in Section 8, Section 9 and Section 10. The key encryption and other related key management security services (i.e. authentication) are defined in Section 13.

It's important to recognize that the data link and physical layers are interface specific. The Reference Models shown in Figure 3 and Figure 4 only apply to the U_m interface. For example, if DLI Key Management messages are conveyed across the E_d interface, the E_d defined Data Link and PHY layers are present instead of the CAI defined Data Link and PHY layers.

There are two types of keys defined in this document. These are Traffic Encryption Keys (TEKs) and Key Encryption Keys (KEKs). TEKs are used to encrypt voice or data traffic (and to authenticate messages). KEKs are used only to encrypt other keys such as KEKs or TEKs. Keying methods (loading of keys into subscriber units) may include the manual fill of TEKs, KEKs, and/or the OTAR of keys to individual units and groups. Group rekey requires that each SU in a particular group shares a Common KEK (CKEK) and any keys sent to this group are encrypted on this CKEK.

For group OTAR, a Subscriber Unit (SU) may or may not be required to acknowledge a key management message depending on the protocol used.

Procedures defined in this document include changing of the keys from the current cryptoperiod to the keys of the next cryptoperiod. These procedures may include a changeover command sent from the KMF, an automatic changeover based on the date and time (i.e. Time Of Day), a user initiated changeover, or direct replacement of active keys using the Rekey or Modify-Key commands. During a changeover period, both the old and new set of keys may be used for encryption, decryption and authentication (generation of a Message Authentication Code, or MAC). An SU should be capable of storing keys for both the current cryptoperiod as well as the next cryptoperiod. This is normally done through the use of the Changeover command but may be performed by manipulating individual keys within a single keyset.

6.1 Keyset Definition and Operation

To facilitate efficient key management, a number of keys may be grouped together. These groups of keys are called keysets. Keysets contain one or more keys of the same type (TEK or KEK). Typically, all of the keys within a keyset have the same cryptoperiod, parameters (when used), and activation time when automatic KM is used. However, individually updating a key using the Modify-Key-Command (for example) may effectively provide individual keys within the same keyset with a cryptoperiod that is independent of the keyset. Note that since certain information is common to all keys in the keyset, it does not necessarily need to be stored with each individual key. Keysets are assigned logical IDs and key management is performed via keysets or direct management of keys within a keyset using specific OTAR messages such as the Rekey, Modify-Key, Delete-Key, or Zeroize commands. Keysets may be activated by several methods including by operator selection, by an over-the-air command from the KMF, or by date and time for automatic KM. When a keyset is an active keyset, it means that the keys in that keyset are used to encrypt traffic.

A grouping of keysets is defined as a Crypto Group (CG) and contains the same type of keys, either TEKs or KEKs. If a Crypto Group has two or more keysets then one keyset is active while the remaining keysets are inactive. Once keys are placed in an inactive keyset, a changeover command is issued which activates the new keyset and inactivates the current or old keyset. Having two or more keysets in a Crypto Group provides for a smooth transition between keys so secure communications are not interrupted during SU rekeying. Multiple Crypto Groups facilitate key management when keysets have different activation times or crypto periods.

There are two methods of updating and managing keys within an SU; A “managed cryptoperiod” method, and an “unmanaged cryptoperiod” method. The “managed cryptoperiod” method provides keys to an inactive keyset, and then uses the Changeover command to activate these keys after all (or an acceptable number) of the SUs have been rekeyed. The managed cryptoperiod method provides a more seamless transition from one key to the next with minimal impact to encrypted communications. An “unmanaged cryptoperiod” method provides individual keys to the active keyset. Replacing keys in the active keyset may lead to interruptions in communications between SUs while the keys are being updated.

A key map contains a maximum of 16 Crypto Groups with a maximum of 16 keysets in each CG (the exception is Crypto Group 15 which only has 15). All keysets are uniquely numbered from zero to 254 and each one supports up to a maximum of 4096 keys. The largest supported mapping scheme would be 16 Crypto Groups and 16 keysets for total of 256 keysets. However Keyset ID = 0 is not defined and therefore only 255 keysets are available for use. Because of this limitation, the last Crypto Group does not contain the full 16 keysets.

Two parameters are defined and used in the management of keys: A Storage Location Number (SLN) and a Keyset Identifier (Keyset ID). An SLN is a unique identifier that points to a row of keys within a Crypto Group. A Keyset ID identifies the keyset, or vertical column within a Crypto Group. These two parameters when combined uniquely identify which key is to be used for secure transmissions and for the management of a specific key. When the SU needs to encrypt a message it selects the key by specifying the SLN and Keyset ID.

The SLN is a two octet number made from the concatenation of the Crypto Group (upper 4 bits numbered 0 through 15), followed by 12 bits for the Key Number (numbered 0 through 4095), and may take on a value within the range of zero to 65,535.

The Keyset ID is a one octet field defined by the concatenation of the Crypto Group (upper 4 bits) and the keyset (lower 4 bits). Note that a value of 0 is not a valid Keyset ID so an offset of 1 is added to the concatenated value to create a Keyset ID range of 1 through 255.

Keys are managed in storage at the subscriber in a key map and are identified by a combination of the Storage Location Number (SLN) and the Keyset ID. The SLN and the Keyset ID are used to indirectly point (without reference to the Key ID or ALGID) to the proper key to use for encrypted transmission. Indirect key storage is the principle of using a pointer mechanism to index into memory where key information resides, such as the ALGID and Key ID. This indirect pointing allows channel definitions to remain constant while Keyset IDs, Key IDs, and ALGIDs may change. The keysets allow for the KMF to provide the reference from a SLN to a key. An example key map is shown in Figure 5.

CG	SLN	Column 1	Column 2	Column 3	…	Column 15	Column 16
Crypto Group (CG) 0	00000						
	00001	Key Variable A	Key Variable B				
	…						
	00008						
	…						
	00016						
	…						
		Keyset 0	Keyset 1	Keyset 2		Keyset 14	Keyset 15
	…	Keyset ID 1	Keyset ID 2	Keyset ID 3		Keyset ID 15	Keyset ID 16
	00128						
	…						
	00256						
	…						
	00512						
	…						
	04095						
CG 1	04096						
	…	Keyset 16	Keyset 17	Keyset 18		Keyset 30	Keyset 31
		Keyset ID 17	Keyset ID 18	Keyset ID 19		Keyset ID 31	Keyset ID 32
	08191						
CG2-CG14							
	…						
CG15 (Reserved for KEK's)	61440						
	…	Keyset 240	Keyset 241	Keyset 242		Keyset 254	
	…	Keyset ID 241	Keyset ID 242	Keyset ID 243		Keyset ID 255	
	65535						

Figure 5 - Key Map

Multiple algorithms may be stored within a Crypto Group and within a keyset. The protocol to transfer keys only allows for one algorithm type to be transferred at a time within a Rekey-Command message. However, the Rekey procedure supports multiple Rekey-Commands to be sent in sequence and thereby transferring keys for multiple algorithms in multiple Rekey-Command messages.

Key management for multiple algorithms in Crypto Group 0 and Crypto Group 15 shall be supported by both the KMF and Subscriber Units. Key management for Crypto Groups 1 through 14 may be supported. If key management for Crypto Groups 1 through 14 is supported, then multiple algorithms shall also be supported for those Crypto Groups.

Crypto Group 15 is reserved for UKEKs and CKEKs (called the KEK Crypto Group). As a minimum within Crypto Group 15, Keyset ID 255 shall be supported for storing UKEKs and CKEKs. Keyset IDs 241-254 may also be supported. The recommended layout of Crypto Group 15 is shown in Figure 6. All SLNs in Crypto Group 15 are available for KEK storage.

UKEKs shall be stored in Crypto Group 15 in an even-numbered SLN only, while CKEKs shall be stored in Crypto Group 15 in an odd-numbered SLN only. This way, UKEKs and CKEKs can be identified and key managed based on their SLN value within Crypto Group 15.

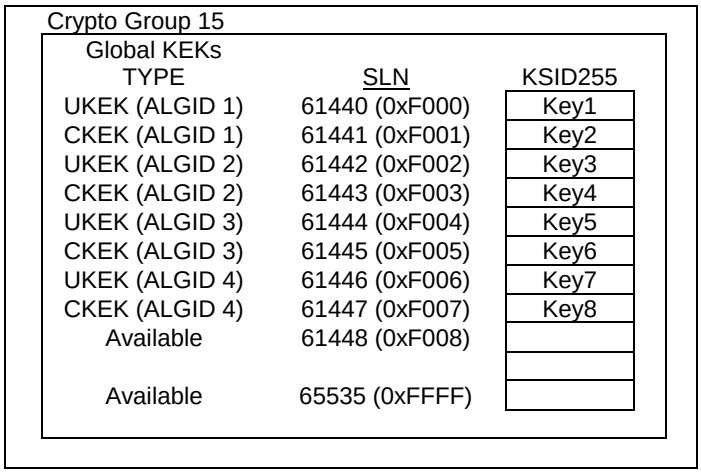


Figure 6 - KEK Storage Example

The KMF also manages keys for cryptonets. A cryptonet is a group of SU users that share a common traffic encryption key. Cryptonets may consist of voice encryption keys, data encryption keys, or a combination of both. Operators of the key management system may choose to map Channel Selector Switches (CSSs), Talk Groups (TGs), or Logical Link Identifiers (LLIDs) to cryptonets for ease of management.

The TEK is selected based on the SLN and the active keyset, where the SLN is selected by the conventional channel, talk group, mode selection, etc.

The voice TEK selected for receive is identified by the received ALGID and Key ID in the message and is validated one of three ways:

[1] The received ALGID and Key ID match the transmit key's ALGID and Key ID identified by the SLN and active keyset,
[2] The received ALGID and Key ID match the ALGID and Key ID of any key in the 16 keysets identified by the SLN selected for transmit,
[3] The received ALGID and Key ID match the ALGID and Key ID of any key from the key storage database.

Option 1 is the most restrictive implementation, and Option 3 is the least. Option 2 shall be normal mode of operation for trunking systems and cannot be disabled. The Key Encryption Key selected for receive may be any KEK in Crypto Group 15 identified by the received ALGID and Key ID.

For conventional operation, it is recommended that the first key found matching the ALGID and Key ID received in the message be used to decrypt the message (Option 3).

For preconfigured Crypto Groups, the active keyset defaults to the lowest numbered keyset or by user provisioning. For dynamically created Crypto Groups, the first

keyset created becomes the active keyset. Keyset ID 255 shall always be active for UKEKs and CKEKs.

The Key ID parameter is defined in section 10.3.10 and should be unique for each key. If more than one key matches the same Key ID and ALGID there is no guarantee which key is selected, therefore the KMF should prevent duplication of Key IDs and ALGIDs for keys with different key values.

Any identified TEK in the received OTAR KMM which is also present in the key storage database may be used for outer layer decryption. Any identified KEK in the received OTAR KMM which is also present in the key storage database may be used for inner layer decryption. By assigning key(s) to SLN(s) that do not have a voice channel assigned to it allows keys to be dedicated to Key Management functions. Keys to be transferred in an OTAR message are (inner layer) encrypted with a KEK. An encrypted key is only decrypted using a KEK from Crypto Group 15. Outer layer encryption and authentication of the OTAR message may use any traffic key in the database.

The OTAR protocol allows for an alias name to be assigned to each key variable as an optional feature. If this feature is enabled, the SU may display the key alias for the key being used.

Figure 7 through Figure 9 show an example of key organization and management within an SU. Figure 7 shows how keys could be stored in an SU, where three parameters are used to select a key; the Crypto Group selector, the Key selector, and the keyset selector.

Figure 8 shows one method of how the key selector may be obtained through the selection of a channel, talk group, or data group. The output of the key selection process is an SLN. Similarly, the keyset may be selected by commands sent over the air from the KMF or could be selected through the user interface if allowed by the manufacturer.

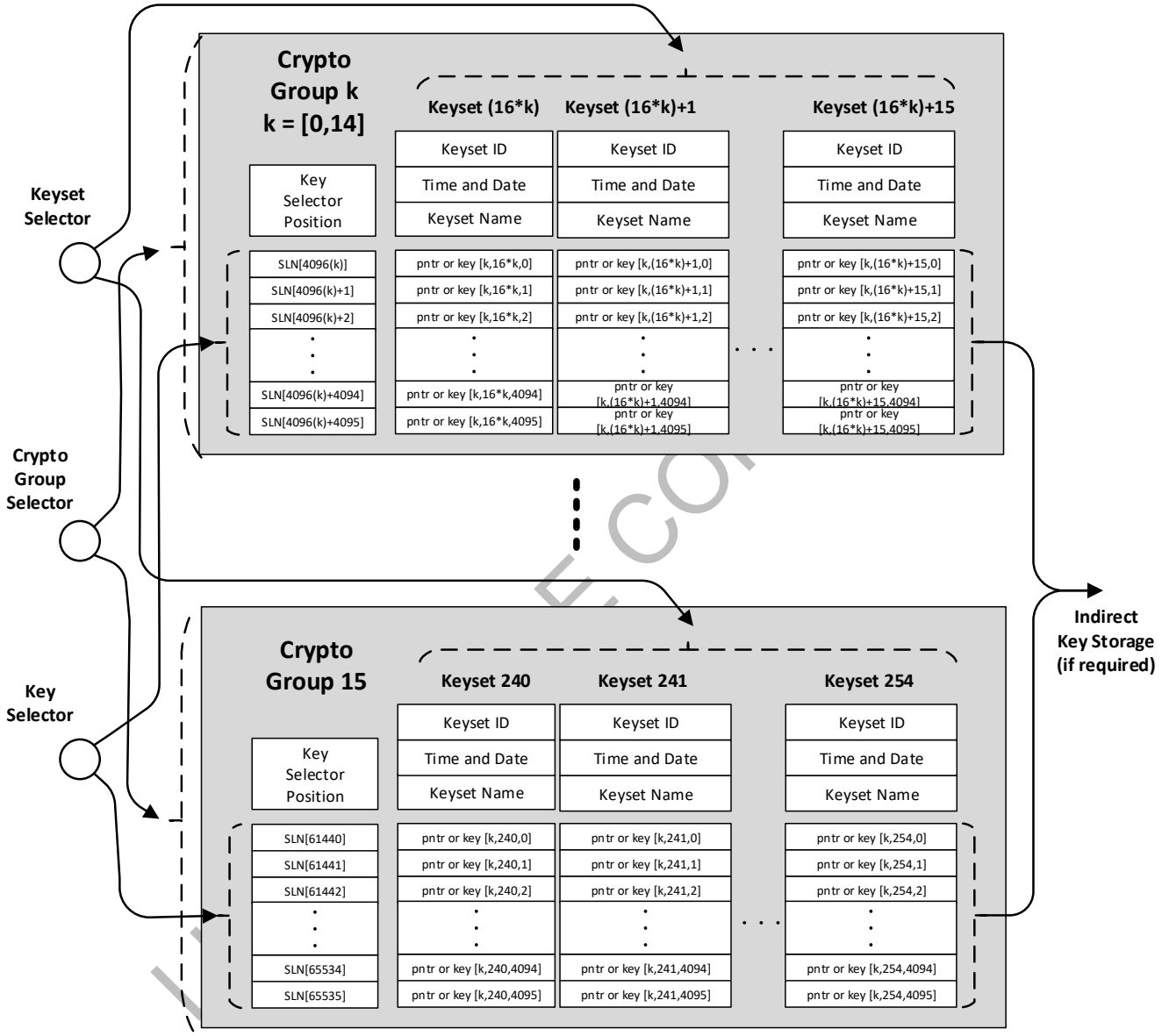


Figure 7 - Subscriber Unit Key Management Concept

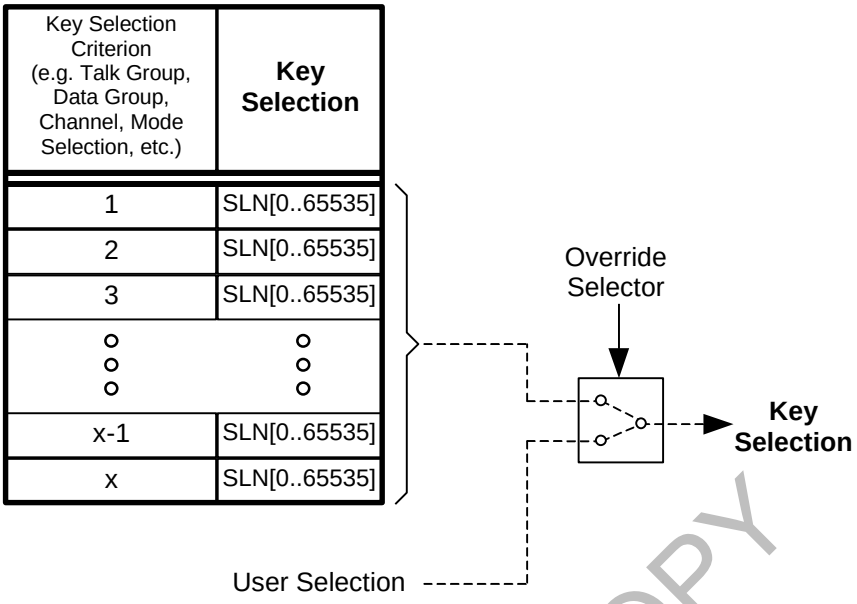


Figure 8 - Key Selection

Figure 9 - VOID

6.2 Key selection and Message Encryption

A KEK is used to encrypt other keys such as TEKs or KEKs (inner layer encryption) for transport in a KMM, while a TEK is used to generate the MAC and provide outer layer encryption of the KMM. The selection of which key is used for inner layer and outer layer encryption is left to the message originator. Any key selected for encryption of the KMM or the KMM payload is a key held in common with the individual SU or group. For a KMM sent to an individual SU, a unique KEK (UKEK) known only to the KMF and SU should be selected. Similarly, for a KMM sent to a group of SUs, a common KEK (CKEK) known only to the KMF and the group of SUs is selected[3]. For outer layer encryption or for the generation of the MAC, any TEK that the KMF and the SU, or group of SUs, has in common may be selected. Typically, the key used to encrypt the outer layer of the KMM is the same key used to generate the MAC. Any TEK (or a derived MAC key as specified in Section 13.5.2 for Enhanced Message Authentication) may be used to authenticate the message (i.e. generate a MAC). Another option is to dedicate certain TEKs as authentication only keys.

The SU typically replies to a received command in the same mode (encrypted or clear) and if the message is encrypted, using the same key(s) as the received command (referred to as “respond in kind”). In other words, if a validated KMM is received clear from the KMF, the response from the SU may also be sent clear (if allowed as specified in Section 13). If the command is encrypted, then the key used to encrypt the command is the same key used to encrypt the response, and the key used to

[3]Group data is currently not a standardized feature for Trunked systems.

authenticate (MAC) the command is the same key used to authenticate (MAC) the response. Section 13.5 provides a description of the MAC rules and exception cases.

If the command changes the key that is being used to encrypt and/or authenticate the message, then the response uses the new key(s). When such a KMM is received the ALGID and Key ID are used to determine the SLN and Keyset ID for the response key. A similar example is the Warm Start Procedure where a successful Modify-Key or Rekey Command triggers the SU to stop using the warm start key and use one of the new TEKs to send the response message (see Section 9.1).

If a command is received that deletes the key needed to encrypt the response message, the key is temporarily maintained to encrypt and/or authenticate (MAC) the response. Once the response message is encrypted and/or authenticated, the key is then permanently discarded.

6.3 Key Management Hierarchy

Note that the use of OTAR does not preclude the use of manual key management procedures once OTAR is enabled and in service, however if both OTAR and manual key management are used simultaneously, careful execution is needed to avoid Key ID and SLN collisions.

For the examples in this section, an SU employing a KMF that enforces cryptoperiods has the following components:

- One keyset for KEKs (one UKEK and one CKEK) in Keyset ID 255
- Two keysets for TEKs in Crypto Group 0
- An Individual RSI
- A Group RSI
- A KMF RSI
- A message number period
- One encryption algorithm

A set of SUs that have a common Group RSI and hold one or more Crypto Groups in common forms a Key Management Group (KMG). An SU is associated with one TEK Crypto Net and one KEK Crypto Net through the KMG. In this example, the KMF keeps track of SUs, keysets, keys (TEKs, and KEKs), Key Management Groups (KMGs) and Crypto Groups. Each type of key (TEK or KEK), is treated separately.

Figure 10 shows the hierarchy of data stored at the KMF. Each particular piece of data is addressable: the SU is addressable through the individual RSI, the KMG is addressable through the Group RSI, the Crypto Group is represented by the most significant 4 bits of the SLN, the keyset is addressable through the Keyset ID of which the most significant 4 bits are the Crypto Group, and the Keys are represented by the least significant 12-bits remaining in the SLN.

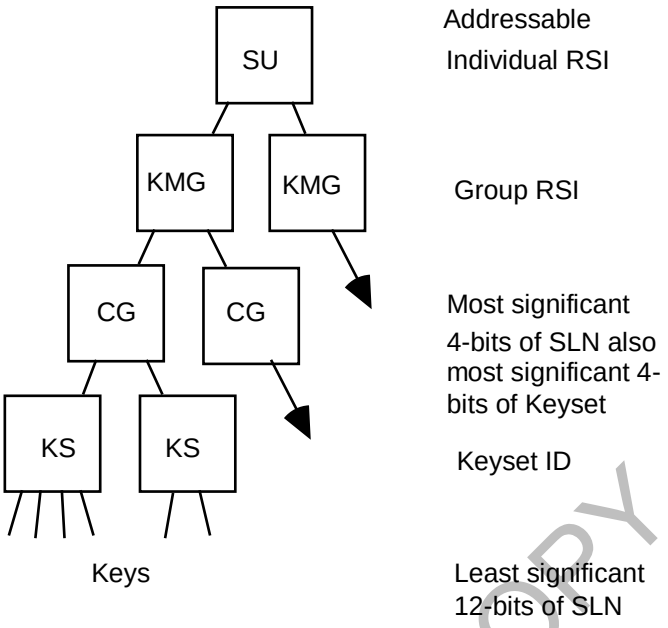


Figure 10 - KMF Data Hierarchy

EXAMPLE 1: SU1 is assigned to KMG1. KMG1 consists of Crypto Group 0 and Crypto Group 15. Crypto Group 0 consists of TEK Keyset IDs 1 and 2, and Crypto Group 15 consists of KEK Keyset ID 255. Keyset ID 1 is marked Active. Keyset ID 1 is shown consisting of TEK 1, TEK 3, TEK 5, TEK 7 and TEK 9, and Keyset ID 2 is shown consisting of TEK 2, TEK 4, TEK 6, TEK 8 and TEK 10. Keyset ID 255 consists of a UKEK and CKEK. Figure 11 shows the SU storage requirements for this example.

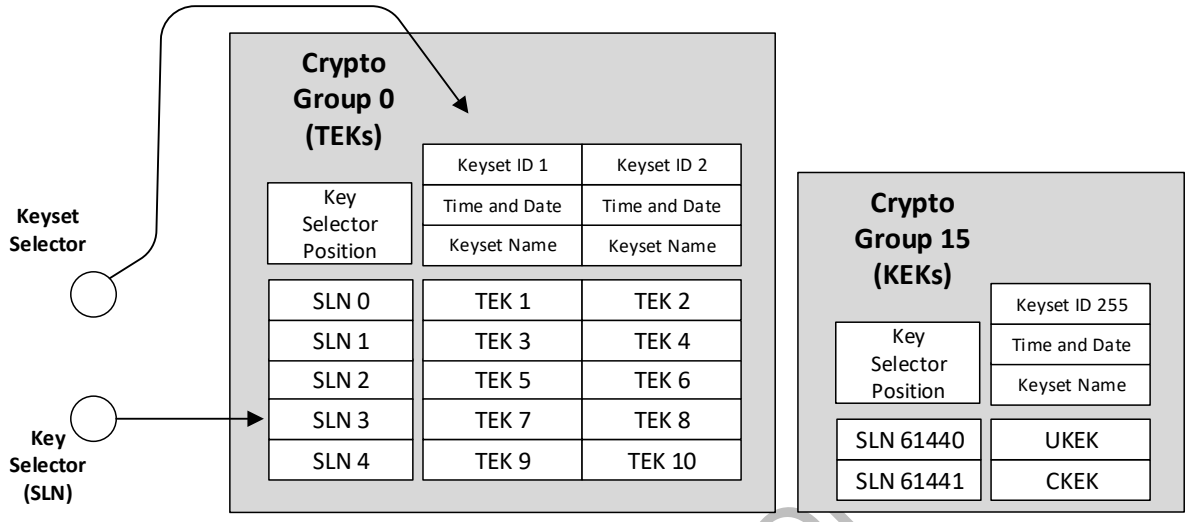


Figure 11 - SU Storage Example

6.4 Initial Key Fill of Subscriber Units

In order for an SU to operate in the protected mode, an initial key or set of keys shall be loaded. To do this, either the SU is brought into a maintenance facility or the initial key(s) are transported to the SU and loaded in the field using some form of key loading device, as defined in [21]. For SUs with no OTAR capability, these keys would be TEKs only. An SU with OTAR capability shall be loaded with at least one UKEK. One or more TEKs may also be loaded at the same time. Keysets and other related KM parameters and tables may also be defined and programmed into the SU at this time. The absolute minimum number of keys would be one UKEK (per algorithm supported). This UKEK should be unique to the SU and held in common with the KMF.

Additional CKEKs for group operations may be loaded at this time or sent to the SU via OTAR at some later time. These CKEKs are held in common by a group of SUs and the KMF so that the KMF can perform group rekeying operations. The initial TEK (if loaded) would be used to provide traffic encryption for user traffic and KMMs.

6.5 Radio Set Identifiers and Key Management Groups

Each SU is assigned one or more addresses for key management purposes. These addresses are called Radio Set Identifiers (RSIs) and may be assigned by the KMF. An RSI can be either an individual RSI or a group RSI. A group RSI is a common RSI assigned to a group of SUs. An SU is always assigned one Individual RSI (Range = 1–9,999,998) and zero or more Group RSI (Range = 10,000,000–16,777,215). A KMF may use any RSI from 1 to 9,999,999. An SU is not to be assigned an individual RSI value of 9,999,999. An Individual RSI cannot be substituted for a Group RSI and vice versa. Individual RSIs are typically programmed into the SU when it is brought into

service. KMMs are addressed to destination units through use of RSIs. An SU accepts KMMs addressed to its individual RSI, and to the group RSI to which the SU is currently assigned. Note that an SU shall support only one active Group RSI at any one time. An SU always uses its individual RSI as the Source RSI in any KMM that it originates. An SU also uses a KMF's RSI as the destination RSI in any KMM that the SU originates.

A Key Management Group (KMG) is one or more SUs which respond to the same Group RSI. The SUs in a KMG typically hold a set of keys in common. These keys include CKEKs and TEKs. For KM purposes, a KMG is managed as a single entity to the greatest extent possible. For systems supporting group OTAR, commands from the KMF are addressed to the Group RSI and members of the group that receive the commands act on them simultaneously. The use of key management groups can help to minimize the air time required for key management purposes.

Key Management Groups may be initially programmed into an SU when it is brought into service, but it is more likely to be given this information from the KMF that is managing the SU. For each group RSI, at least one group CKEK is required and a group TEK may also be loaded. It is possible to create and modify a KMG by over the air commands from the KMF.

In a data message, a Logical Link ID usually identifies a particular UNIT or GROUP in the radio system. The RSI is a similar identifier for OTAR. For convenience, the individual and group ranges for these two are the same, and in the actual message these two identifiers could be the same, but don't necessarily have to be. The RSI may be different from the LLID to provide address anonymity for encrypted KMMs. For example, in order to protect the RSI from possible disclosure, the LLID may be assigned as an ALL CALL identifier, and the RSI may be assigned as an individual or group RSI. The RSI itself is part of the KMM and is typically protected from disclosure through encryption.

6.6 **Role of the Key Management Facility**

The role of the KMF is to provide key management to SUs using the OTAR services defined in this document.

7 OVERVIEW OF PROTOCOL

The OTAR protocol is designed to operate over a channel that is not reliable. The key management application layer protocol provides the capability for a message originator to require that certain types of messages be acknowledged or responded to by the destination unit.

In addition, the protocol includes other methods to help ensure messages have been received and acted upon. For example, if the KMF sends a message requesting that a unit take a specific action but does not request or require an acknowledgment, the KMF is not certain that the unit received the message. The KMF after some time can either provide the message again or query the unit to check its current status (providing both the KMF and target support the functionality to do so).

When a sequence of messages is exchanged between a KMF and SU, a key management procedure is performed.

7.1 Definition of Response Kinds

There are three types of response kinds defined in this document; messages which do not need to be acknowledged (Response Kind 1 (None)), messages which require a specific response to be sent back after some delay (Response Kind 2 (Delayed)), and messages which require an immediate response (Response Kind 3 (Immediate)). Each response kind is described in more detail below. Every key management message has a response kind field which is used by the receiving device to determine the type of response. A KMM may have more than one valid response kind depending on how the message is used in the procedure.

7.1.1 Response Kind 1 (None)

A Response Kind 1 (None) message does not need to be acknowledged by the intended receiving unit. The originator of the message is not expecting a response from a Response Kind 1 (None) message. A Response Kind 1 (None) message is transmitted at least once but may be transmitted multiple times to improve the probability of being received by the originator of the request.

7.1.2 Response Kind 2 (Delayed)

Response Kind 2 (Delayed) messages require a delayed response by the target device. Response Kind 2 (Delayed) messages are used for group procedures in which a response is required but sent in such a way as to not flood the channel. The Delayed-Acknowledgment message is used by the SU to respond to a message requiring a Response Kind 2 (Delayed) response. A Delayed-Acknowledgement message may be sent either after a random time, called t_{DLY} (defined as the time between receiving a message requesting a Response Kind 2 (Delayed) response and the Delayed Acknowledgement response message being sent), or initiated by one or more trigger mechanisms in the SU itself (i.e. power on, power off, PTT activation,

PTT deactivation, receipt of a Response Kind 3 (Immediate) message, etc.). Individually generating t_{DLY} within each target unit prevents an entire group from responding at the same time. The parameter t_{DLY} is further defined in Section 7.3. The delay time, t_{DLY}, is not reset if another Response Kind 2 (Delayed) message is received before the response is sent. The event is only cleared once the Delayed-Acknowledgement response message has been sent.

The KMF shall discard any Response Kind 2 Command initiated from an SU.

7.1.3 Response Kind 3 (Immediate)

Response Kind 3 (Immediate) messages require an immediate response by the unit receiving the message. In general, if a response is not received in a timely manner, the sending device times out after t_{TO} seconds (Section 7.3) and a retry mechanism may be executed.

If the unit receiving a Response Kind 3 (Immediate) message acknowledges the message but the originator does not receive the acknowledgment, a retry of the original message may be repeated. The unit that received the message shall be able to receive another identical Response Kind 3 (Immediate) message and respond with a correct response provided that no other messages have been received during that time. The response to the identical message may be different than the first response. However, sufficient information should be provided in the response to indicate that the desired result has been obtained. For example, the first Delete-Key-Command message (which does not delete the TEK encrypting the message) would cause a Delete-Key-Response to be sent back indicating that the command was performed. A duplicate Delete-Key-Command would cause a Delete-Key-Response indicating that the key does not exist (which indicates that the first Delete-Key-Command has been executed or that the SU never had the key). As a second example, if a Zeroize-Command is sent; a Zeroize-Response is returned. If a duplicate Zeroize-Command is sent to the SU after the SU has successfully executed the first Zeroize Command, an Unable to Decrypt message may be returned since the message cannot be decrypted (all of the keys have been deleted).

A Negative Acknowledgment shall not be sent if it cannot be sent encrypted and authenticated. If a target receives an unencrypted NACK, it shall be discarded.

7.1.4 Response Kinds Summary

An overview of Response Kinds is shown in Table 1:

Table 1 - Summary of Response Kind Types

Kind of message	Response	Time of Response
Response Kind 1 (None)	None	N/A
Response Kind 2 (Delayed)	Response Kind 1 (None) Response	After a delay
Response Kind 3 (Immediate)	Response Kind 1 (None) Response	Immediately

Messages are divided into different response kinds, and the combinations of these response kinds for key management procedures are shown in Table 2. There are some messages that have more than one valid response kind.

Table 2 - OTAR Messages and Valid Response Kinds

OTAR Message	Response Kind		
	1 (None)	2 (Delayed)	3 (Immediate)
Capabilities-Command			X
Capabilities-Response	X		
Change-RSI Command			X
Change-RSI Response	X		
Changeover-Command	X	X	X
Changeover-Response	X		
Delayed-Acknowledgment	X		
Delete-Key-Command	X	X	X
Delete-Key-Response	X		
Delete-Keyset-Command	X	X	X
Delete-Keyset-Response	X		
Deregistration Command	X		X
Deregistration Response	X		
Hello	X		X
Inventory-Command			X
Inventory-Response	X		
Key-Assignment-Command			X
Key-Assignment-Response	X		
Modify-Key-Command	X	X	X
Modify-Keyset-Attributes-Command	X	X	X
Modify-Keyset-Attributes-Response	X		
Negative-Acknowledgment	X		
No-Service	X		
Registration Command	X		X
Registration Response	X		
Rekey-Command	X	X	X
Rekey-Acknowledgment	X		
Set-Date-Time	X		
Unable-to-Decrypt Response	X		
Warm-Start-Command	X	X	X
Zeroize-Command			X
Zeroize-Response	X		

7.2 Use of Response Kinds

A procedure is a set of one or more messages sent between two entities, such as from an SU to a KMF or KMF to an SU. Multiple procedures may be concatenated during a single communication session. For example, a Hello procedure may be used before any other OTAR procedure to initiate communications between the KMF and an SU.

7.2.1 Procedures That Use Response Kind 1 (None) Messages

If the message sent from either the KMF or SU requests a Response Kind 1 (None) response, the recipient does not send a response message. Response Kind 1 (None) responses are used for certain types of commands where a response is not appropriate or required. An example would be broadcast messages such as Set-Date-Time. Three possible scenarios for using a Respond Kind 1 (None) message are identified here.

1. The Response Kind 1 (None) message is sent to an individual SU (i.e. the address field in the message is for an individual RSI instead of a group) and a response message is not needed.

2. The Response Kind 1 (None) message is sent to a group of SUs (i.e. the address field of the message is for an entire group). This procedure is used for group messages and is especially useful for large groups that would otherwise flood the channel with responses. The KMF may be configured to re-broadcast the message to maximize delivery success.

3. The Response Kind 1 (None) message is broadcast to a group of SUs where the intent is to later individually poll each SU for success using a command such as the Inventory-Command message. This procedure is useful when the message, and any retries, is much larger than the inventory solicitation performed on an individual basis. The procedure is verified for each of the SUs in the group while minimizing use of the channel.

7.2.2 Procedures That Use Response Kind 2 (Delayed) Messages

If the message sent from the KMF requests a Response Kind 2 (Delayed) response, the SU responds with a Response Kind 1 (None) message at some time in the future. This delay time or trigger is a trigger mechanism in the SU (e.g. PTT).
Only the KMF shall request Response Kind 2 (Delayed) responses. Two scenarios for using a Respond Kind 2 (Delayed) message are identified here.

1. The Response Kind 2 (Delayed) message is broadcast to a group of SUs. The SUs send a response the next time the SU is used (e.g. the user presses Push-to-Talk, or the SU receives a response kind 3 message). Doing so allows a pseudo-randomization of the times that the responses are sent to the KMF which help prevent flooding of the channel.

2. The Response Kind 2 (Delayed) message is broadcast to a group of SUs. The SUs send an automatic response based on a random delay time, t_{DLY}, determined by some mechanism in the SU.

The KMF shall discard any Response Kind 2 Command initiated from an SU.

Note that error responses to a Response Kind 2 (Delayed) message are also delayed.

7.2.3 Procedures That Use Response Kind 3 (Immediate) Messages

The procedure for using a Response Kind 3 (Immediate) message is intended for one-on-one key management communications (i.e. not group messaging) and may be initiated from either the KMF or the SU. If the KMF initiates the communication session with the Response Kind 3 (Immediate) message, the SU responds with the appropriate Response Kind 1 (None) response message. If the SU initiates the communication session with a Response Kind 3 (Immediate) message, the KMF responds appropriately.

If the recipient receives one message then receives a repeat message identical to the first, the recipient responds with the correct initial response provided that the first message has not been processed. Note that the response to additional repeat messages may be different than the first response if the message has already been processed (e.g. Zeroize-Command and Delete-Key-Command). Sufficient information is provided in the response to indicate the result of the command. For example, this situation could occur if the SU accepts and processes a key management command from the KMF but the Response Kind 1 (None) message is not received by the KMF.

7.3 System Parameter Summary

The delay time parameter (t_{DLY}) is used to randomly determine how much delay the SU waits before sending a Response Kind 2 (Delayed) response and is recalculated by the SU for every Response Kind 2 (Delayed) response to be sent.

The timeout parameter (t_{TO}) is used by the KMF and SU to manage the wait time for Response Kind 3 (Immediate) commands. If a Response Kind 3 (Immediate) message is sent and a response is not received, the KMF or SU times-out after t_{TO} seconds and may re-transmit the message or perform some other form of error recovery. The receiving unit of a Response Kind 3 (Immediate) message should send a response message as soon as the response is ready. The t_{TO} parameter shall be configurable in the KMF and SU using a manufacturer specific method.

See Table 3 for details of the t_{DLY} and t_{TO} parameters.

Table 3 - System Wide Parameter Definitions

System Parameter	Definitions	Min/Max value	Default
t_{DLY}	The random delay time used for Delayed-ACK (response kind 2) responses.	30 seconds /24 hours	n/a
t_{TO}	The timer used to initiate a retransmission of the KMM.	5 seconds/ 90 seconds	5 seconds

7.4 KMM Validation and Fragmentation

KMMs are validated before any OTAR command in the KMM is executed. The validation includes checking the source RSI, destination RSI, Message ID, proper encryption status, message length, message number, and message authentication code. A KMM is not processed if it fails validation of either the source RSI or destination RSI. The SU may respond to the KMM by sending a negative acknowledgment (or NACK) if the KMM passes the RSI validations but not the remaining validations. Whether or not a NACK is sent depends on the Response Kind of the received KMM and the type of error encountered. Any command contained in a valid KMM is executed by the OTAR subsystem. After a command is processed, the SU may send a response to the KMF according to the response type specified in the KMM. If the KMM is not valid, it is discarded and a response may be sent back to the KMF.

The SU attempts to execute the commands in a received KMM only after the received KMM passes validation. It is recommended that received KMMs be validated in the following order:

1. *Destination RSI* - The SU verifies that the received KMM destination RSI matches the Individual RSI, the active Group RSI stored in the SU, or the All Call RSI (0xFFFFFF). If an invalid RSI is received, the KMM is discarded and the SU sends no response.
2. *Source RSI* - The SU verifies that the received KMM source RSI matches the KMF RSI configured in the SU. If the RSI does not match, regardless of the response kind, the SU discards the KMM and sends no response.
3. *Message Length* - The valid range for a message length is 7 octets up to the Maximum KMM Data Length minus 3 (reference section 12.4). The adjustment of 3 accounts for the first 3 octets of the KMM which are not included in the message length calculation.
4. *Message ID* - If the SU receives a KMM with an unsupported message ID, the SU discards the message and responds as follows:
   - If the KMM was sent as Response Kind 3 (Immediate), the SU shall send a Negative-Acknowledgment with the status field set to Invalid Message ID (0x03).
   - If the KMM was sent as Response Kind 2 (Delayed), the SU sends a Delayed-Acknowledgement message with the status field set to Invalid Message ID (0x03).

- If the KMM was sent as Response Kind 1 (None), no OTAR response is returned.

5. *Encryption Status* - The SU ignores and discards OTAR commands that are received unencrypted if the commands are specified to be encrypted as described in Section 13. Outer layer encryption is only allowed with TEKs, therefore the SU discards any KMM that is outer layer encrypted with a KEK. If the TEK used for outer layer encryption of the OTAR command is not present in the SU, the SU should respond by sending a Negative Acknowledgment or Unable-To-Decrypt message to the KMF.
6. *Message Number* - A message number is accepted if it is within the valid message number range (Section 13.6). If the MN is not valid, the KMM is discarded and the SU responds as follows:
   - If the KMM was sent as Response Kind 3 (Immediate), the SU shall respond by sending a Negative-Acknowledgment message with the status field set to Invalid Message Number (0x07).
   - If the KMM was sent as Response Kind 2 (Delayed), the SU responds by sending a Delayed-Acknowledgment message with status field set to Invalid Message Number (0x07).
   - If the KMM was sent as Response Kind 1 (None), no response message is returned to the KMF.
7. *Message Authentication Code* - The SU validates the MAC by recalculating a MAC over the received KMM. The MAC is considered valid if the recalculated MAC and the MAC in the KMM are identical. If the recalculated MAC does not match the MAC in the KMM, the SU discards the message and responds as follows:
   - If the KMM was sent as Response Kind 3 (Immediate), the SU shall respond by sending a Negative-Acknowledgment message with the status field set to Invalid MAC (0x04).
   - If the KMM was sent as Response Kind 2 (Delayed), the SU responds by sending a Delayed-Acknowledgment message with status field set to Invalid MAC (0x04).
   - If the KMM was sent as Response Kind 1 (None), no response message is returned to the KMF.

   If the key used to generate the MAC is not present in the SU, the SU responds as follows:
   - If the KMM was sent as Response Kind 3 (Immediate), the SU responds by sending a Negative-Acknowledgment message with the status field set to Item Does not Exist (0x02).
   - If the KMM was sent as Response Kind 2 (Delayed), the SU responds by sending a Delayed-Acknowledgment message with status field set to Item Does not Exist (0x02).
   - If the KMM was sent as Response Kind 1 (None), no response message is returned to the KMF.

Once the KMM is validated, the OTAR command in the KMM may still fail due to an incorrectly formatted command, or an SU limitation such as an unsupported option,

lack of memory, etc. If the command is received in error, the response may vary depending on the Response Kind, the command, and the circumstances that cause the command to be invalid. If a response message is sent, it contains a status field indicating whether or not the command was performed.

Fragmentation of OTAR KMMs is currently not supported at the application level. CAI Delivery Types for KMMs may either be confirmed or unconfirmed and are specified in each procedure. More information on confirmed and unconfirmed CAI Delivery Types can be found in [4] and [23].

8 PROCEDURE DEFINITIONS

A key management procedure is an exchange of one or more KMMs grouped together to provide a basic key management function. These key management procedures provide OTAR interoperable key management operations.

When procedures are implemented, endpoint adherence to the protocol and message requirements as designated within those procedures enables interoperable implementations.

Details of the messages used to support these procedures are specified in Section 10.

Note that some of the procedures may support optional message responses depending on the type of the response kind declared in the initial request message. The procedure may require an immediate response, a delayed response, or no response at all. In error cases, a negative acknowledgement or Unable to Decrypt message may be returned.

8.1 Capabilities Procedure

The Capabilities procedure is initiated by the KMF when it is interested in the key management capabilities of an SU. In response, the SU provides a list of the OTAR messages which it supports. This procedure can be used to determine the capabilities of SUs which have just been put into service. This procedure assists the KMF in determining how to manage the SU. The basic procedure is shown in Figure 12.

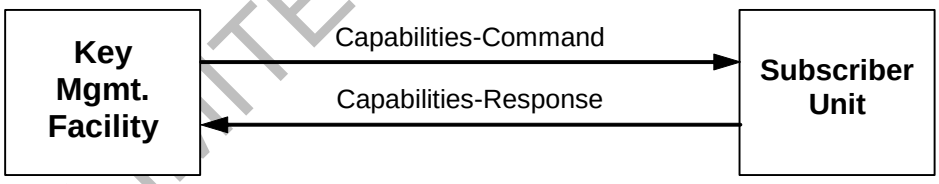


Figure 12 - Capabilities Procedure

This procedure consists of two messages; Capabilities-Command (Section 10.2.1) and Capabilities-Response (Section 10.2.2). The procedure is terminated after the SU sends the Capabilities-Response message to the KMF. The summary of the response kind for the Capabilities procedure messages is shown in Table 2.

If the Capabilities-Command is validated and executed, the SU responds with a Capabilities-Response message.

The Capabilities-Response message provides the KMF with a list of the key management services and KMM types that the SU supports. Elements of the

Capabilities-Response message include; encryption algorithm, message identifier, and optional functions that the SU supports.

8.2 Change-RSI Procedure

This procedure allows the KMF to add, modify or delete a group RSI, modify an individual RSI, and/or reset the inbound and outbound message numbers associated with that RSI. For each individual and group RSI, the SU and the KMF maintains an inbound and an outbound Message Number, which are initialized to the value sent in the Change-RSI command. Note that the KMF RSI is not manageable by the Change-RSI procedure.

The basic procedure is shown in Figure 13.

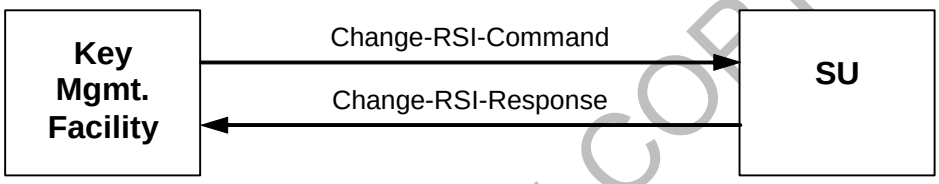


Figure 13 - Change-RSI Procedure

This procedure consists of two messages; Change-RSI-Command (Section 10.2.3) and Change-RSI-Response (Section 10.2.4). The procedure is terminated after the SU sends the Change-RSI-Response message to the KMF. The summary of the response kind for the Change-RSI procedure messages is shown in Table 2.

When a Change-RSI-Command message is sent to an SU, the information contained in the message impacts the RSI parameters in multiple ways. The Change-RSI-Command KMM contains two RSI values; the RSI currently configured in the SU (which could be no RSI) and the RSI to replace the current RSI (also could be no RSI). Similarly, the Change-RSI-Response message contains two RSI values; the first one is the RSI that existed and the second is the replacement RSI value.

If the Change-RSI-Command is successful, the current RSI is replaced with the new RSI, and both the inbound and outbound message numbers associated with the RSI are initialized to the value provided in the Message Number field. When the first and second RSI values are equal, the inbound and outbound Message Numbers associated with the RSI are still initialized although the RSI remains the same.

The SU adds, modifies or deletes the RSI as specified, initializes the inbound and outbound MNs, and then returns a Change-RSI-Response message. The Change-RSI-Response message indicates that the command was executed successfully. The Change-RSI-Response is both encrypted and authenticated.

Possible Change-RSI-Command actions are summarized in Table 4.

Table 4 - Change-RSI-Command Actions

Change-RSI-Command RSI Pair	Result
A / B	Change individual/group RSI from "A" to "B". Initialize “B” inbound and outbound Message Numbers.
Zero / A	Add group RSI "A". Initialize “A” inbound and outbound Message Numbers. Invalid for Individual RSIs
A / Zero	Delete group RSI "A" Invalid for Individual RSIs
A / A	Initialize individual/group “A” inbound and outbound Message Numbers to value sent in the command.

For each item in the Change-RSI-Command there is a corresponding pair of RSIs in the response plus a status octet. Table 5 provides possible status codes.

Table 5 - Change-RSI-Response Codes

Possible Responses	Status Octet Value	Circumstance
Change RSI Response	$00 Command was performed	
	$01 Command not performed	- Attempted to modify Individual RSI with Group RSI destination - Change from Group RSI to Individual RSI (or vice versa) - Attempt to change Individual/Group RSI to the reserved KMF RSI - Attempt to change the reserved KMF RSI to an Individual/Group RSI
	$02 Item does not exist	- Changing, deleting or initializing an RSI which is not in the SU
	$05 Out of memory	- Attempted to add more RSIs than the maximum allowed

Table 6 lists the permutations in a Change-RSI-Command and the associated responses. In Table 6, it is assumed that the SU has active Individual RSI “a” and an active Group RSI “x”.

The Group RSI value in an SU may be added, deleted or modified by the KMF. Only a single Group RSI is supported at any one time in an SU. To delete a Group RSI, the first RSI in the Change-RSI-Command is set to the target RSI and the second RSI value is set to zero. Similarly, a Group RSI may be added by setting the first RSI to zero and the second RSI to the new Group RSI value.

Table 6 - Change-RSI-Command Operation Example

RSI to be modified	RSI to become active	Perform Change-RSI	Synchronize the Message Number	Response sent to the KMF
Group RSI "x"	Any Group RSI other than "x"	Yes	Yes	Command was performed (Change Group RSI)
0	Any Group RSI	Yes	Yes	Command was performed (requires that no group existed previously) (Add Group RSI)
Group RSI "x"	0	Yes	No	Command was performed (Delete Group RSI)
Group RSI "x"	Group RSI "x"	Yes[4]	Yes	Command was successful (Group Message Numbers Synchronization)
0	0	No	No	Command was not performed (Incorrect active Individual RSI)
0	Individual RSI	No	No	Out of memory
Individual RSI "a"	0	No	No	Command was not be performed (Individual RSI's cannot be deleted)
Individual RSI "a"	Individual RSI "a"	Yes[5]	Yes	Command was successful (Individual inbound and outbound MNs Synchronized)
Individual RSI "a"	Individual RSI "b"	Yes	Yes	Command was successful (Change Individual RSI)
Any Individual RSI other than "a"	Any Individual RSI or zero	No	No	Item does not exist (Incorrect active Individual RSI)
0	Any Group RSI	No	No	Out of memory (Only one active group RSI allowed)
Any Group RSI other than "x"	Don't Care	No	No	Item does not exist (Incorrect active Group RSI)
Any Individual RSI	Any Group RSI	No	No	Command not performed (Cannot interchange Individual RSI and Group RSI)
Any Group RSI	Any Individual RSI	No	No	Command not performed (Cannot interchange Individual RSI and Group RSI)
Individual RSI "a"	Any reserved KMF RSI (i.e. 9,999,999)	No	No	Command not performed (Cannot set an individual RSI to the reserved KMF RSI)
Any reserved KMF RSI (i.e. 9,999,999)	Any Individual RSI	No	No	Command not performed (Cannot use the reserved KMF RSI as a individual address)
Group RSI "x"	Any reserved KMF RSI (i.e. 9,999,999)	No	No	Command not performed (Cannot set Group RSI to the reserved KMF RSI)
Any reserved KMF RSI (i.e. 9,999,999)	Any Group RSI	No	No	Command not performed (Cannot use the reserved KMF RSI as a Group address)

[4] The final result is effectively "no change" to the Group RSI since the Change-RSI command is attempting to replace the existing value with the same value.

[5] The final result is effectively "no change" to the Individual RSI since the Change-RSI command is attempting to replace the existing value with the same value.

If the Change-RSI-Command is validated, the SU executes the Change-RSI command and replies with a Change-RSI-Response.

8.2.1 Out of sync Message Number

If the KMF determines that a message number used by an SU is out of sync it uses a Change RSI Procedure to resynchronize the message number. The KMF does this by populating the Change-RSI-Command message with the current RSI value for both the current RSI and replacement RSI. The message also provides the new Message Number. It's important to note that the KMF needs to know the current SU MN for use in the KMM Header of the Change RSI Command KMM for the SU to process the Change RSI Command KMM.

The SU receives and authenticates the message, then updates the current RSI with the new RSI value (effectively no change) and updates both the inbound and the outbound message numbers accordingly. The SU then acknowledges the request with the Change-RSI-Response message containing the current and replacement RSI values, along with a (successful) status of execution. Resynchronization of message numbers may be performed on individual, Group or All Call RSIs.

The Change-RSI-Command may be used to resync the MN for the ALL CALL RSI address. If the ALL CALL RSI is in either (but not both) the "RSI to be changed/deleted" and "RSI to be added" fields of the Change-RSI-Command, a "command not performed" error shall be returned.

If an SU reaches the point where message numbers are out of sync and the SU has no TEKs in common with the KMF, the "New Unit" Procedure should be executed to resynchronize MNs.

See also section 13.6.3 for further information on MN resynchronization.

8.3 Changeover Procedure

The Changeover procedure is used to instruct an individual SU or group of SUs to move from using one keyset to the next keyset. The Changeover procedure shall request one of the 3 response kinds (Response Kind 1, 2 or 3) indicated in the Changeover-Command message. The basic procedure is shown in Figure 14.

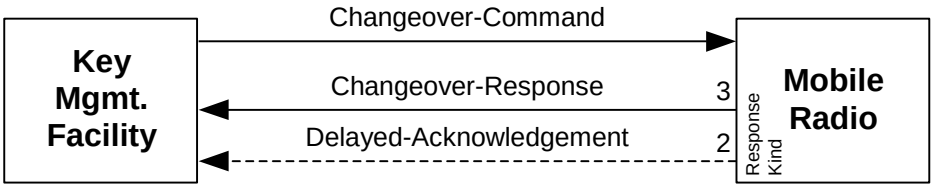


Figure 14 - Changeover Procedure

The Changeover Procedure consists of the Changeover-Command message (Section 10.2.5) and the Changeover-Response message (Section 10.2.6) or Delayed-Acknowledgement message (Section 10.2.7). The Response Kind 1 (None) Changeover-Command may be sent to either an individual SU or a group of SUs. No response message is sent in return to the Response Kind 1 (None) Changeover-Command message.

For a Response Kind 2 (Delayed) request, the SU responds with a Delayed-Acknowledgment message. The Response Kind 2 (Delayed) Changeover-Command message is used primarily for group changeovers in which an acknowledgment is desired but sent in such a way that does not flood the channel.

The Response Kind 3 (Immediate) Changeover-Command message is used primarily when targeting an individual SU. For a Response Kind 3 (Immediate) Changeover-Command message, the SU responds with an immediate Changeover-Response message.

A summary of the response kinds supported for the Changeover procedure is shown in Table 2.

The Changeover-Command message contains one or more pairs of Keyset IDs. The first Keyset ID in each pair identifies the keyset believed to be currently active and the second Keyset ID identifies the next active keyset. The SU performs the changeover upon validation of the request, and then responds with the appropriate response. If a Changeover-Response or Delayed Acknowledgment is sent by the SU, it is both encrypted and authenticated.

A Changeover-Response message provides an indication to the KMF as to how the Changeover-Command was executed. For each pair of keysets in the Changeover-Command, the Changeover-Response message contains a corresponding pair of keyset responses.

If the SU is using the currently identified active keyset and is in possession of the second keyset, the Changeover-Response contains the same two Keyset IDs as in the Changeover-Command. If the keyset specified to become active is already active, then the Changeover-Response shall contain a zero value for the first Keyset ID and contain the Keyset ID of the keyset currently in use for the second Keyset ID. If the SU is using the correct Keyset ID but does not have the keyset specified to become

active by the second Keyset ID, the Changeover-Response shall contain the Keyset ID currently in use for the first Keyset ID and shall contain a zero value for the second Keyset ID in the pair.

Table 7 lists the different permutations involved in a Changeover-Command and the possible responses. An SU only maintains one active keyset in each Cryptogroup. The Changeover procedure shall support the changeover of keysets belonging to the same Cryptogroup only. A Changeover from one keyset to a keyset in a different Cryptogroup shall not be allowed.

In this example and shown in Table 7, it is assumed that the SU has keysets A, B and C in Cryptogroup 1 and keyset D in Cryptogroup 2. Keyset E is not in the SU.

Table 7 - Changeover Operation Example

Changeover-Command Keyset ID pair	Keyset Currently Active in each Cryptogroup (CG)	Perform Changeover	Changeover Response pair	Delayed-Acknowledgement Response Status
A,A	Any but A,D	No*	Zero,Zero	Command not performed ($01) (No knowledge of current keyset)
		Yes*	Zero, A	Command performed ($00) (any but A is changed to A)
A,B	A,D	Yes	A,B	Command performed ($00) (A is changed to B)
A,B	B,D	Not required	Zero,B	Command performed ($00) (B is already active)
A,D	A,D	No	A,zero	Command not performed ($01) (A and D are not in same CG)
A,D	B,D	No	Zero,Zero	Command not performed ($01) (A and D are not in same CG)
A,B	C,D	No*	Zero,Zero	Command not performed ($01) (A not currently active)
		Yes*	Zero, B	Command performed ($00) (C is changed to B)
A,E	A,D	No	A,Zero	Command not performed ($01) (E is not in SU)
A,E	Any but A,D	No	Zero,Zero	Item does not exist ($02) (E is not in SU)
E,Any	Don't Care	No	Zero,Zero	Item does not exist ($02) (E is not in SU)
A,A	A,Don't Care	No	Zero,A	Command performed ($00) (Keyset already active)

*For interoperability between manufacturers, both responses are allowed.

8.4 **Delete-Key Procedure**

The Delete-Key procedure allows the KMF to delete keys stored in an SU. This procedure is initiated by the KMF sending a Delete-Key-Command to an individual SU or group of SUs. The Delete-Key-Command has one or more entries containing the

Algorithm ID and the Key ID of the key to be deleted. For each entry in the Delete-Key-Command, the Delete-Key-Response includes a status field indicating the status of the deletion for that particular key. The SU deletes the key prior to sending the Delete-Key-Response message to the KMF, with one exception; when the key used to encrypt the Delete-Key-Command is one of the keys being deleted. A delete-key command specifying a particular ALGID/Key ID pair shall delete any and all keys with that identifier.

The Delete-Key procedure is shown in Figure 15.

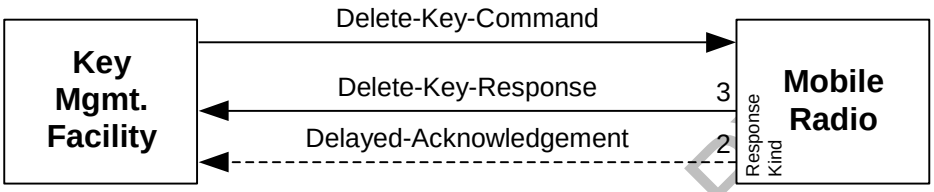


Figure 15 - Delete-Key Procedure

The Delete-Key Procedure consists of the Delete-Key-Command message (Section 10.2.8) and the Delete-Key-Response (Section 10.2.9) or Delayed-Acknowledgement message (Section 10.2.7).

The Delete-Key-Command message may request any of the 3 response kinds. For a Response Kind 1 (None), the SU does not return a response. For a Response Kind 2 (Delayed), the SU responds with a Delayed-Acknowledgment message at some later time. Note that the deleted key is unavailable for other encryption/decryption operations during the T_{DLY} period. For a Response Kind 3 (Immediate), the SU responds with an immediate Delete-Key-Response message. The summary of supported Response Kinds for the Delete-Key procedure is shown in Table 2.

When the key used to decrypt the Delete-Key-Command message is one of the keys to be deleted, the SU maintains the key for encryption and/or authentication of the response message. Once the response message is encrypted and/or authenticated, the key is permanently discarded.

Table 8 shows status values that may be provided to the KMF in the Delete-Key-Response or Delayed Acknowledgment messages.

Table 8 - Delete-Key Status Responses

Possible Responses	Status Octet Value	Circumstance
Delete-Key Response	$00 Command was performed	
	$01 Command not performed	- Unspecified error
	$02 Item does not exist	- Key ID/ALGID does not match any of the keys in the SU
Delayed Ack	$00 Command was performed	
	$01 Command not performed	- Unspecified error
	$02 Item does not exist	- Key ID/ALGID does not match any of the keys in the SU
No Response	N/A	N/A

8.4.1 Delete Key from a Keyset

Another way to delete an individual key in a keyset is to set the Delete/Rekey bar bit (b5) in the key format field (reference Section 10.3.12) and perform either a Modify-Key Procedure or a Rekey Procedure. When a keyset changes such that a particular key is eliminated (deleted), normally, the KMF would have to perform both the Modify-Key and the Delete-Key procedures in order to provide the correct keyset information. Using the Delete/Rekey bar bit reduces the number of messages required to delete a key that is no longer used.

8.5 Delete-Keyset Procedure

The Delete-Keyset Procedure is initiated by the KMF and is used to delete one or more keysets from an individual SU or group of SUs. This procedure allows the KMF to delete both TEKs and/or KEKs. When a keyset is deleted, all keys and their attributes (Key ID, key variable, Key Name) stored in the keyset as well as all of the keyset attributes (Keyset ID, Algorithm ID, Date, Time and Keyset Name) are deleted.

The Delete-Keyset-Command contains the Keyset ID(s) for one or more keysets stored in the SU. For each ID sent in the Delete-Keyset-Command, the Delete-Keyset-Response includes a status of the specified keyset. The SU deletes the keyset prior to sending the Delete-Keyset-Response message to the KMF.

The Delete-Keyset procedure is shown in Figure 16.

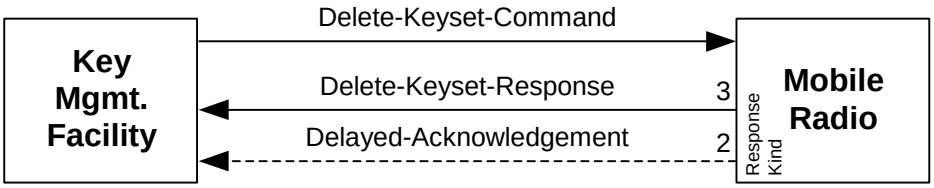


Figure 16 - Delete-Keyset Procedure

The Delete-Keyset Procedure consists of the Delete-Keyset-Command message (Section 10.2.10) and the Delete-Keyset-Response (Section 10.2.11) or the Delayed-Acknowledgement message (Section 10.2.7).

The Delete-Keyset procedure may utilize any of the 3 response kinds (Response Kind 1, 2 or 3). The KMF may instruct the SU to immediately respond with a Delete-Keyset-Response message (Response Kind 3 (Immediate)), a Delayed-Acknowledgment message (Response Kind 2 (Delayed)), or the KMF may not require a response at all (Response Kind 1 (None)). The response kind expected is indicated in the Message Format field of the Delete-Keyset-Command. The SU removes the identified keyset(s) and then responds accordingly.

The summary of the response kinds for the Delete-Keyset procedure is shown in Table 2.

An active keyset cannot be deleted when two or more keysets exist within any one Crypto Group. If an attempt is made to delete the active keyset, the receiver responds with an error code indicating the command was not performed. The active keyset is deleted only if it is the last remaining keyset in a Crypto Group. A single Delete-Keyset-Command may be used to remove both active and inactive keyset(s) provided that the inactive keyset(s) are listed (deleted) first and the active keyset is listed (deleted) last within the Delete-Keyset-Command.

Execution of the Delete-Keyset-Command on Keysets 241-255 may cause loss of key encryption keys, thus disabling the ability to execute KEK dependent operations such as the decryption of current or future TEKs.

Table 9 shows the possible status octet values for the Delete-Keyset-Response and Delayed-Acknowledgement messages when used in the Delete-Keyset procedure.

Table 9 - Delete-Keyset Responses

Possible Responses	Status Octet Value	Circumstance
Delete-Keyset Response	$00 Command was performed	
	$01 Command not performed	- The active keyset cannot be deleted
	$02 Item does not exist	- Keyset ID does not match any of the keysets in the SU
Delayed Ack	$00 Command was performed	
	$01 Command not performed	- The active keyset cannot be deleted
	$02 Item does not exist	- Keyset ID does not match any of the keysets in the SU
No Response	N/A	N/A

If a Delete-Keyset-Response or Delayed-Acknowledgment message is sent by the SU, it is both encrypted and authenticated with the TEK that was used to decrypt the Delete-Keyset-Command.

When the key used to decrypt the Delete-Keyset-Command message is a key in the keyset to be deleted, the SU deletes the keyset but maintains the key(s) for encryption and/or authentication of the response message. Once the response message is encrypted and/or authenticated, the key is permanently discarded.

8.6 Deregistration Procedure

The Deregistration Procedure is initiated by the SU and indicates to the KMF that the SU is no longer available for OTAR service. Possible circumstances for deregistration include when the SU powers down or when the SU exits an OTAR capable mode of operation.

The Deregistration procedure is shown in Figure 17.

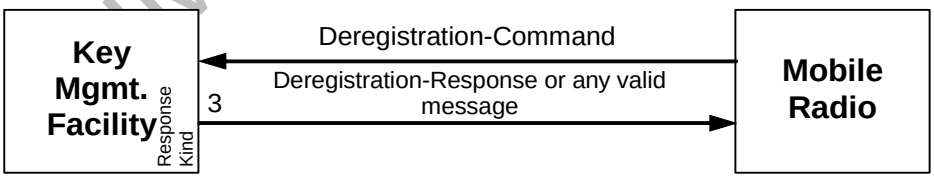


Figure 17 - Deregistration Procedure

The Deregistration Procedure consists of the Deregistration-Command message (Section 10.2.12) and the Deregistration-Response (Section 10.2.13) or another valid KMM.

The Deregistration-Command message requests either a Response Kind 1 (None) or a Response Kind 3 (Immediate) response. For Response Kind 1 (None), the KMF is not required to send a response message. This is efficient for high traffic and/or more

reliable data systems. For Response Kind 3 (Immediate), the KMF sends a Deregistration-Response message or another valid KMM. Under typical Deregistration scenarios, if no key management procedures need to be provided to the SU, then the Deregistration-Response message is sent.

The summary of response kinds for the Deregistration-Command message is shown in Table 2.

A Deregistration-Response message indicates to the SU that the KMF has received and acknowledged the Deregistration-Command. It is not used to deny OTAR services or to cease OTAR operations at the SU.

Table 10 - Deregistration Procedure Status Responses

Possible Responses	Status Octet Value	Circumstance
Deregistration-Response	$00 Command was performed	
	$01 Command not performed	
No Response	N/A	N/A

It is possible that the SU may not receive the Deregistration-Response message in a timely fashion, for instance during an SU power down situation. Therefore the SU shall not rely on reception of the Deregistration-Response message in order to complete the deregistration process.

Because the Registration procedure is required for Data Link Independent (DLI) OTAR but optional for Data Link Dependent (DLD) OTAR (see Table 79), the following Deregistration procedure recommendations are made;

- If the Registration Procedure is used for the key management registration of an SU, then the Deregistration Procedure may be used for key management deregistration of the SU.
- If the Registration Procedure is NOT used for key management registration, then the Deregistration Procedure SHALL NOT be used for key management deregistration of the SU.

8.7 **Hello**

The Hello Procedure is initiated by either the KMF or the SU and serves multiple purposes. It's used by the KMF to query the presence of a specific SU, used by an SU to identify itself to the KMF, and also used by an SU to request key management services.

When the KMF initiates the Hello Procedure (Figure 18), the KMF sends a Hello message (Section 10.2.14) as a Response Kind 3 (Immediate) message indicating that an immediate response from the target SU is requested. The Hello message may be sent clear, or encrypted and authenticated. When the SU receives and (if necessary) successfully decrypts and authenticates the Hello message, the SU replies

to the KMF with a Response Kind 1 (None) Hello message (Section 10.2.14), ending the Hello procedure. If the KMF times-out while waiting for a response to the initial Hello message, the KMF has the option to repeat the Hello procedure at a later time. The KMF may follow a successful Hello Procedure with any key management procedure.

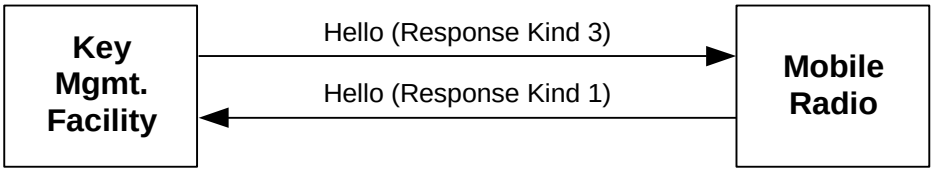


Figure 18 - KMF Initiated Hello Procedure

The summary of the response kinds supported for the Hello procedure when initiated by the KMF is shown in Table 2.

If the SU initiates the Hello procedure (Figure 19), a Hello message (Section 10.2.14) is sent to the KMF as a Response Kind 3 (Immediate) message. Flags in the Hello message allow the SU to indicate that key management services may be required. For example, an SU may send a Hello message requesting key management services when a key at the SU has been accidentally erased and needs to be restored.

The KMF responds to the Hello message with either the No-Service message (Section 10.2.23) or another valid KMM giving the KMF the opportunity to initiate key management procedures should the SU require them. If the KMF has no key management procedures to execute with the SU, the KMF may respond with a No-Service message. If the SU is requesting a rekey, the KMF may perform any key management operation, for example: inventory the SU; rekey the SU; or return a No-Service message.

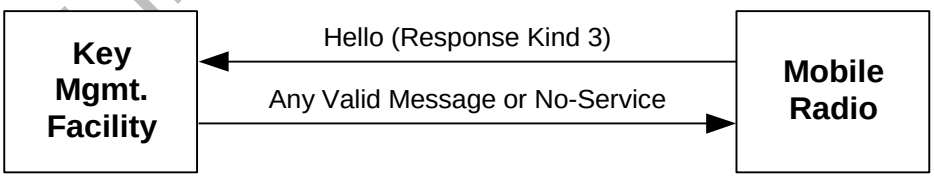


Figure 19 - SU Initiated Hello Procedure

The summary of the response kinds for the Hello procedure messages initiated by the SU is shown in Table 2.

8.7.1 Check Unit

The Check Unit operation allows the KMF to determine if the SU is on the system. For instance, this operation can be used to avoid sending a Rekey when the SU is not on the system and thus saving over-the-air bandwidth. The operation consists of a Hello message sent by the KMF and a Hello message sent in response by the SU if on the system. The SU may indicate that it would like to be rekeyed in the Hello Response.

8.7.2 Rekey Request

The operator of the SU may desire to request keys from the KMF. The SU sends a Hello Command with the appropriate bits set to indicate a Rekey Request. If the KMF declines to provide keys, the KMF may send a No-Service Response to the SU. If the KMF proceeds to provide service to the SU, the KMF may respond with any valid message. The KMF may send a Rekey Command or Modify-Key Command to give keys to the SU and perhaps follow that with a Changeover Command.

8.8 Inventory Procedure

The Inventory Procedure is an OTAR key management procedure which allows the KMF to obtain specific key management information from the SU such as; the current date/time, a list of active keysets, a list of inactive keysets, a list of all Keyset IDs, a list of the Key IDs, a list of keyset attributes, or a list of key attributes.

The Inventory Procedure is initiated by the KMF sending a Response Kind 3 (Immediate) Inventory-Command message to the SU (Section 10.2.15).

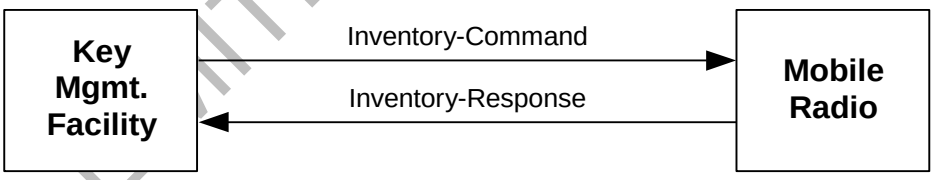


Figure 20 - Inventory Procedure

The response kinds for the Inventory-Command and Inventory-Response messages are shown in Table 2.

The Inventory-Command message contains an Inventory Type field which specifies what items are to be sent in the Inventory-Response message. The Inventory-Response message contains a list of the requested items (reference Section 10.2.16).

If the SU does not support either the Inventory Procedure or the Inventory Type requested, a Negative-Acknowledgment message with the error code $03 (Invalid Message ID) shall be returned.

8.9 Key-Assignment Procedure

The Key-Assignment Procedure enables the KMF to instruct an SU on which encryption key is to be used when communicating securely on a particular channel (conventional), Talk Group ID (trunked), or Logical Link ID (data). The Key-Assignment procedure maps Storage Location Numbers (SLN) to these Channel Selector Switches, Talk Group IDs, or Logical Link IDs.

The Key-Assignment Procedure is initiated by the KMF and consists of two messages; The Key-Assignment-Command message (Section 10.2.17) and the Key-Assignment-Response message (Section 10.2.18).

The Key-Assignment procedure is shown in Figure 21.

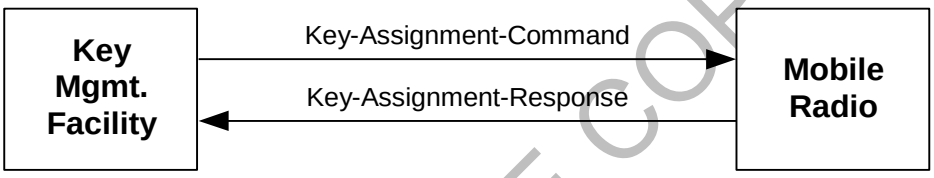


Figure 21 - Key-Assignment Procedure

In the Key-Assignment-Command message, the “Assign SLNs to CSSs” sequence contains a Channel Selector Switch and a Storage Location Number for each channel selector switch position that requires secure communications. The “Assign SLNs to TGs” sequence contains a Talk Group ID and a Storage Location Number for each talk group that requires secure communications. The “Assign SLNs to LLIDs” sequence contains a Logical Link ID and a SLN for each data group that requires secure communications

If the Key-Assignment-Command is valid the SU responds with a Key-Assignment-Response message, which reports the key assignment status for (only) those items specified in the Key-Assignment-Command.

The summary of the response kinds for the Key-Assignment procedure messages is shown in Table 2.

8.10 Modify-Key Procedure

The Modify-Key procedure is initiated by the KMF and allows the KMF to delete, replace or add a key to a keyset without sending all of the keys in the keyset. The Modify-Key Procedure may be applied to an individual SU or to a group of SUs. The keys are encrypted using a KEK held in common between the SU (or group of SUs) and the KMF. The Modify-Key Command is a shortened version of the Rekey

Command and limits key management changes (modification of keys and parameters) to a single keyset within a single message. Note that the Rekey Command allows key management changes to multiple keysets within a single message.

The Modify-Key Procedure is initiated by the KMF and consists of the Modify-Key-Command message (Section 10.2.19) and the Rekey-Acknowledgement (Section 10.2.26) or the Delayed-Acknowledgment message (Section 10.2.7).

The Modify-Key-Command message can be used to add new keys or to replace existing keys. The Modify-Key-Command message provides new keys along with SLNs, Key IDs, and Keyset ID. The SU overwrites the specified location(s) with the new keys. The Modify-key-Command may also be used to delete existing keys by setting the Delete/Rekey bar.

In order to rekey an SU with keys with different algorithms, multiple Modify-Key procedures may be performed in sequence where each one provides keys for a particular algorithm (reference Section 9.5). The Algorithm ID parameter within each sequence in the Modify-Key-Command message indicates the algorithm type.

The Message Format field in the Modify-Key-Command message indicates the response type (Response Kind 1 (None), Response Kind 2 (Delayed), or Response Kind 3 (Immediate)) the KMF expects from the SU. For a Response Kind 3 (Immediate) Modify-Key-Command message, the SU responds with an immediate Rekey Acknowledgment message. For a Response Kind 2 (Delayed) Modify-Key-Command message, the SU responds with a Delayed Acknowledgment message. For a Response Kind 1 (None) Modify-Key-Command message, no reply is sent by the SU.

The Modify-Key procedure is shown in the Figure 22.

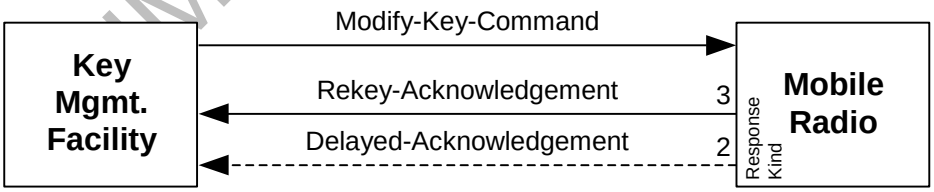


Figure 22 - Modify-Key Procedure

The summary of the response kinds for the Modify-Key procedure messages is shown in Table 2.

Table 11 shows the possible status octet values for the Rekey-Acknowledgement and Delayed-Acknowledgement messages when sent in response to a Modify-Key-Command message.

Table 11 - Modify-Key Responses

Possible Responses	Status Octet Value	Circumstance
Rekey Acknowledgement	$00 Command was performed	
	$01 Command not performed	- Attempt to add a key of an unsupported algorithm type - Invalid key length - Keyset ID and Key Format do not agree - Inner layer MI specified incorrectly - Bad parity on DES key - Message not properly authenticated - Optional field not supported - MFID is standardized and key data ALGID or inner layer ALGID is proprietary - Other unidentified errors
	$02 Item does not exist	- Attempted to modify keyset which does not exist in the SU - Deletion of non-existent key - SLN is not valid for Keyset ID given
	$04 Invalid MAC	- MAC for key transferred is invalid
	$05 Out of Memory	- Attempt to add keys to full SU
	$06 Could not decrypt key	- KEK for key transfer does not exist
Delayed Acknowledgement	$00 Command was performed	
	$01 Command not performed	- Attempt to add a key of an unsupported algorithm type - Invalid key length - Keyset ID and Key Format do not agree - Inner layer MI specified incorrectly - Bad parity on DES key - Message not properly authenticated - Optional field not supported - MFID is standardized and key data ALGID or inner layer ALGID is proprietary
	$02 Item does not exist	- Attempted to modify keyset which does not exist in the SU - Deletion of non-existent key - SLN is not valid for Keyset ID given
	$04 Invalid MAC	- MAC for key transferred is invalid
	$05 Out of Memory	- Attempt to add keys to full SU
	$06 Could not decrypt key	- KEK for key transfer does not exist
No Response	N/A	- Failure (Reference KMM validation rules in Section 7.4)
		- Success, per Response Kind 1 (None)

8.10.1 Change UKEK

Fundamentally, there are two reasons to change the UKEK of an SU. The first is because one believes that the key has been compromised, and the other is to protect the system from a brute force attack on the key.

If security is vital and there is any chance that the UKEK was compromised, the UKEK may be changed using a KFD. This method provides a greater level of security than using OTAR. However, if there is a need to perform a periodic change of the UKEK, a new one may be sent over the air.

The Modify-Key Procedure or the Rekey Procedure may be used to deliver a new UKEK to the SU. The KMF encrypts the new UKEK with the old UKEK for transit to the SU.

Although a Response Kind 1 (None) reply is allowed when modifying or rekeying a UKEK, it is recommended that a Response Kind 2 (Delayed) or Response Kind 3 (Immediate) is used instead.

8.10.2 Change KMG CKEK

Similar to a UKEK, there are two reasons to change the CKEK of a KMG; Key compromise and scheduled key refresh.

If there is any chance that the CKEK was compromised, the CKEK may be changed using individual rekeying employing the UKEK. If the intent is to perform a periodic change of the CKEK, a new CKEK can be sent over the air to the SU or KMG using the Modify-Key procedure or the Rekey Procedure

Although a Response Kind 1 (None) reply is allowed when modifying or rekeying a CKEK, it is recommended that a Response Kind 2 (Delayed) or Response Kind 3 (Immediate) is used instead.

8.10.3 Change TEKs

In order to change a keyset's TEKs, the KMF executes either a Modify-Key procedure or a Rekey Procedure. The entire Crypto Group containing the TEKs could be sent in order to guarantee that the SU has both the active and inactive keys for continuous protected communications. In some instances, multiple Modify-Key Procedures or Rekey Procedures may be required to fully change a keyset’s TEKs.

8.11 Modify-Keyset-Attributes Procedure

The Modify-Keyset-Attributes procedure is used to change the keyset name or the effective Date/Time for keys within a specified keyset. The Modify-Keyset-Attributes procedure allows a KMF to extend or shorten the crypto-period of a keyset and is useful for changing the keyset attributes without sending new key material.

The Modify-Keyset-Attributes Procedure is initiated by the KMF and consists of the Modify-Keyset-Attributes-Command message (Section 10.2.20) and the Modify-Keyset-Attributes-Response (Section 10.2.21) or the Delayed-Acknowledgment message (Section 10.2.7). The target of the Modify-Keyset-Attributes procedure is either an individual or a group of SUs.

The Modify-Keyset-Attributes procedure is shown in Figure 23.

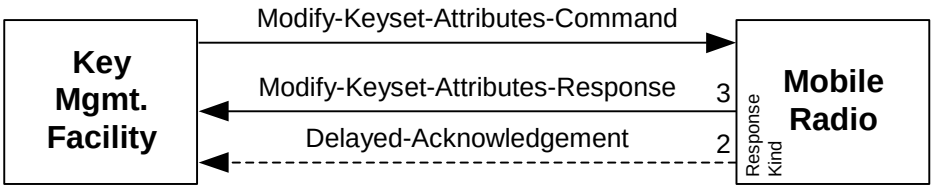


Figure 23 - Modify-Keyset-Attributes Procedure

The SU responds to the Modify-Keyset-Attributes-Command message with a Modify-Keyset-Attributes-Response, a Delayed Acknowledgment, or no response at all. The kind of response the KMF expects is indicated by the Response Kind specified in the Message Format field of the Modify-Keyset-Attributes-Command message. If the SU responds with either a Modify-Keyset-Attributes-Response or Delayed Acknowledgment message, the message is both encrypted and authenticated.

The summary of the response kinds for the Modify-Keyset-Attributes-Command message and responses is shown in Table 2.

8.12 **Registration Procedure**

The Registration Procedure is initiated by the SU to let the KMF know that an SU is on the system and available for OTAR service. Trigger events that may initiate the Registration Procedure may include the SU powering up in an OTAR capable mode, the receiver entering an OTAR capable mode of operation, or if the SU's IP address changes. The applicability of this procedure for the different OTAR protocols is shown in Table 79.

The Registration procedure consists of a Registration-Command message (Section 10.2.24), and a Registration-Response message (Section 10.2.25) or any other valid KMM.

The Registration procedure is shown in Figure 24.

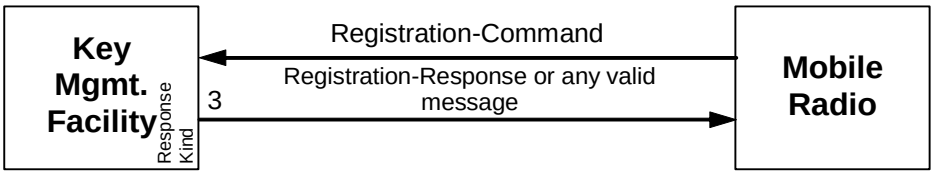


Figure 24 - Registration Procedure

The Registration Command may request either a Response Kind 1 (None) or a Response Kind 3 (Immediate) response message to allow for flexibility in performing a registration procedure. For a Response Kind 1 (None) Registration-Command message, the KMF is not required to send a response. For Response Kind 3 (Immediate) Registration-Command message, the KMF responds with the Registration-Response message or another valid KMM. If no key management procedures need to be provided to the SU, a Registration-Response message is sent to the SU to confirm or deny the registration request.

The summary of the response kinds for the Registration Procedure is shown in Table 2.

Failure to receive a Registration-Response message from the KMF does not mean that the SU may not receive OTAR service at a later time. A Registration-Response is an acknowledgement that the KMF has processed the Registration-Command message. A Registration-Response indicating that the command was not performed or lack of response does not deny OTAR service to the SU or imply that the SU should cease OTAR operations.

Table 12 shows the possible status octet values for the Registration-Response message.

Table 12 - Registration Procedure Status Responses

Possible Responses	Status Octet Value	Circumstance
Registration Response	$00 Command was performed	
	$01 Command not performed	

8.13 Rekey Procedure

The Rekey procedure is initiated by the KMF and allows the KMF to delete, replace or add a key and associated attributes to a keyset without sending all of the keys in the keyset. The Rekey procedure may be applied to an individual SU or a group of SUs. The keys are encrypted using a KEK held in common between the SU (or group of SUs) and the KMF. The Rekey procedure may modify or add (create) one or more keysets within a single message and may also be used to Change UKEKs (section 8.10.1), Change KMG CKEKs (Section 8.10.2), or Change TEKs.

The Rekey procedure is initiated by the KMF and consists of a Rekey-Command message (Section 10.2.27) followed by a Rekey-Acknowledgement message (Section 10.2.26), Delayed-Acknowledgement message (Section 10.2.7), or no response.

In order to rekey an SU with keysets with different algorithms, multiple Rekey procedures may be performed in sequence where each one provides keys for a particular algorithm (reference Section 9.5). The Algorithm ID parameter within each keyset sequence in the Rekey-Command message indicates the algorithm type.

The Rekey procedure is shown in Figure 25.

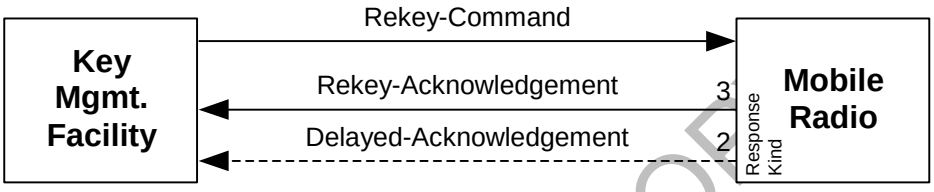


Figure 25 - Rekey-Command

The kind of response the KMF expects is indicated by the Response Kind specified in the Message Format field of the Rekey-Command message. For a Response Kind 1 (None) Rekey-Command message, the SU sends no message response. For a Response Kind 2 (Delayed) Rekey-Command message, the SU responds with a Delayed-Acknowledgment message. For a Response Kind 3 (Immediate) Rekey-Command message, the SU responds with an immediate Rekey-Acknowledgment message. The summary of the response kinds supported for the Rekey procedure is shown in Table 2.

The Rekey-Command message carries the new keys, their corresponding Key IDs and Storage Location numbers, and is encrypted using a common KEK known only to the KMF and the target individual or group SUs. The SU uses the Algorithm ID and the Key ID in the Decryption Instruction fields to identify the KEK needed to decrypt the TEKs. Once decrypted, the SU places the keys in the appropriate storage locations.

In some instances, multiple Rekey Procedures may be required to fully change the keys within a keyset.

Table 13 shows the possible status octet values for the Rekey-Acknowledgement and Delayed-Acknowledgement messages when used in the Rekey procedure.

Table 13 - Rekey Responses

Possible Responses	Status Octet Value	Circumstance
Rekey Acknowledgement	$00 Command was performed	
	$01 Command not performed	- Attempt to add a key of an unsupported algorithm type - Invalid key length - Keyset ID and Key Format do not agree - Inner layer MI specified incorrectly - Bad parity on key - Message not properly authenticated - Optional field not supported - MFID is standardized and key data ALGID or inner layer ALGID is proprietary - Other unidentified errors
	$02 Item does not exist	- Deletion of non-existent key - SLN is not valid for Keyset ID given
	$04 Invalid MAC	- MAC for key transferred is invalid
	$05 Out of Memory	- Attempt to add keys to full SU
	$06 Could not decrypt key	- KEK for key transfer does not exist
Delayed Ack	$00 Command was performed	
	$01 Command not performed	- Attempt to add a key of an unsupported algorithm type - Invalid key length - Keyset ID and Key Format do not agree - Inner layer MI specified incorrectly - Bad parity on DES key - Message not properly authenticated - Optional field not supported - MFID is standardized and key data ALGID or inner layer ALGID is proprietary
	$02 Item does not exist	- Deletion of non-existent key - SLN is not valid for Keyset ID given
	$04 Invalid MAC	- MAC for key transferred is invalid
	$05 Out of Memory	- Attempt to add keys to full SU
	$06 Could not decrypt key	- KEK for key transfer does not exist
No Response	N/A	- Failure (Reference KMM validation rules in Section 7.4)
		- Success, per Response Kind 1 (None)

8.14 Set-Date-Time Procedure

The Set-Date-Time procedure is used by the KMF to set the date and time at an individual SU or group of SUs. This procedure may be used for SUs that support automatic changeover and automatic updating of keys based on date and time-of-day (TOD).

The Set-Date-Time procedure is initiated by the KMF via the Set-Date-Time-Command message. This command is always sent as a Response Kind 1 (None) message, requiring no reply from the target SU. The Set-Date-Time-Command is an encrypted and authenticated message. If the Set-Date-Time-Command message authenticates successfully, the SU sets its internal clock to the specified date and time.

The Set-Date-Time procedure is shown in Figure 26.

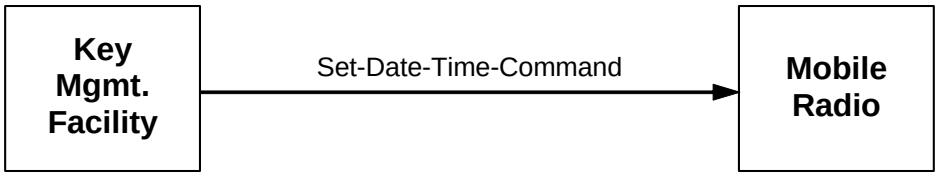


Figure 26 - Set-Date-Time Procedure

8.15 Unable-to-Decrypt Procedure

When the SU receives an encrypted KMM that it cannot decrypt, it returns an Unable-to-Decrypt–Response (UTD) message to the KMF. The SU sends the Unable-to-Decrypt-Response message to the KMF that the SU is currently configured with for OTAR communications. When the KMF receives an Unable-to-Decrypt-Response message it assumes that the message corresponds to the most recent command sent to the SU. The KMF may re-send the message, send a different message, or abort the procedure.

Support for the Unable-to-Decrypt-Response message is critical for minimum OTAR interoperability.

This message is never sent as an unsolicited message but is always in reply to a message sent by the KMF. The basic procedure is shown in Figure 27.

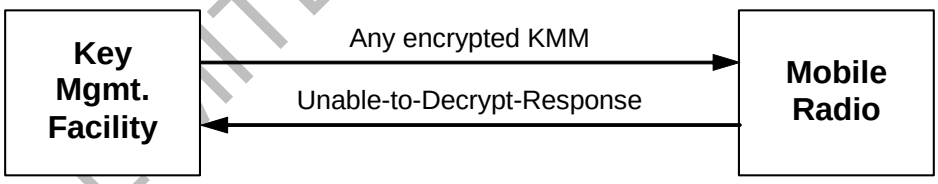


Figure 27 - Unable-to-Decrypt Procedure

The Unable-To-Decrypt-Response indicates the reason the KMM could not be decrypted and the KMF may take appropriate action to resolve the discrepancy.

The UTD response when outer layer encrypted, shall;

1. Use any available TEK matching the ALG ID of the received message, else Use any available TEK in storage.

8.16 Warm-Start Procedure

The Warm-Start procedure is initiated by the KMF when there is a need to communicate securely with an SU but the SU does not have any TEKs in common with the KMF. This procedure sends a temporary TEK to the SU which the KMF also possesses, so they can communicate securely. Once a secure communications session is established using the temporary TEK, the KMF then rekeys the SU with other traffic keys. This procedure requires the KMF and the SU to share a common KEK.

The Warm-Start procedure is initiated by the KMF sending a Warm-Start-Command message (Section 10.2.30) to an individual SU. In response to the Warm-Start-Command message, the KMF may request an immediate Rekey-Acknowledgment message (Section 10.2.26), a Delayed-Acknowledgment message (Section 10.2.7), or no response at all. The kind of response expected from the SU is identified by the Response Kind specified in the Message Format field in the Warm-Start-Command message. If the response is a Rekey-Acknowledgment or Delayed-Acknowledgment message, they are encrypted and authenticated using the warm start key received in the Warm-Start-Command.

The Warm-Start procedure is shown in Figure 28.

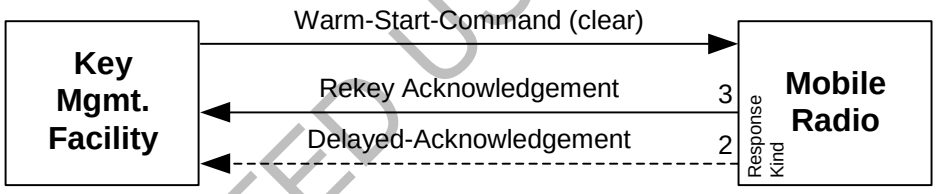


Figure 28 - Warm-Start Procedure

For a Response Kind 1 (None) Warm-Start-Command message, the SU is not required to send a response. For a Response Kind 2 (Delayed) Warm-Start-Command message, the SU responds with a Delayed-Acknowledgment. For a Response Kind 3 (Immediate) Warm-Start-Command message, the SU responds with an immediate Rekey-Acknowledgment.

The summary of the response kinds for the Warm-Start procedure messages is shown in Table 2.

Since the TEKs stored in the SU(s) are unknown, the Warm-Start-Command message is sent with no outer layer encryption. However, the warm start key itself is encrypted using the KEK. It is important to note that other fields in the Warm-Start-Command message are not protected from disclosure, since outer layer encryption is not performed.

The warm start key is temporarily used until the SU is rekeyed. During a rekey of the SU, the warm start key is used to protect the Rekey-Command message. The Rekey-Acknowledgement message sent by the SU is also protected with the warm start key. After the Rekey-Acknowledgement message, the warm-start key is no longer used and should be discarded.

At the KMF, the MAC is calculated over the Warm-Start-Command message which contains the encrypted warm start key, i.e., the MAC is calculated after the message has been fully assembled. The key used to calculate the MAC is the unencrypted version of the warm start key. The warm start key is used because there is no TEK available at the target SU to be used for authentication purposes.

At the SU, the encrypted warm start key in the Warm-Start-Command message shall first be decrypted using a KEK to obtain the plain text version. This unencrypted warm start key is then used to authenticate the original version of the Warm-Start-Command message including the encrypted warm start key. If the authentication process fails, the message and warm start key are discarded. In this case, there is no response sent to the KMF to indicate that the KMM failed to authenticate. The reason for this is because the SU has no way of calculating a MAC for a Negative-Acknowledgement message without a valid TEK.

The Warm-Start-Command message provides both the Key ID and Algorithm ID of the KEK used to encrypt the warm start key. The Warm-Start-Command message also includes the Algorithm ID and the Key ID for the warm start key encrypted with the KEK. The Storage Location Number is neither needed nor used for the warm start key. If the ALGID and Key ID of the warm start key is the same ALGID and Key ID of a TEK already held within the SU, the SU returns an error indication to the KMF (see Table 14).

The Key ID and Algorithm ID used for the warm start key needs to be unique from any key stored in the SU. If the Warm-Start-Command is successful, the warm start key is used to encrypt the KMMs to follow until the first rekey message is received and, if an immediate response is requested, acknowledged. After which the new keys transferred in the rekey message are used. Note that a Rekey procedure should immediately follow a Warm-Start-Command. For SUs that have multiple algorithms, the Warm Start procedure starts with one algorithm and finishes the Warm-Start procedure for that algorithm. At the completion of the warm start procedure for the current algorithm, the warm start procedure is performed on the next algorithm. This procedure may be extended to support more than 2 algorithms.

The Rekey-Acknowledgement and Delayed-Acknowledgement messages provide status to the KMF relating to the success or failure of the Warm-Start procedure.

Table 14 shows the possible status octet values for the Rekey-Acknowledgement and Delayed-Acknowledgement messages when used in the Warm-Start procedure.

Note that an encrypted Rekey-Acknowledgement message may or may not be returned depending on the presence of TEKs within the SU. For example, if the target SU is not able to authenticate and decrypt the warm start key and the target SU contains no other TEKs, a Rekey-Acknowledgement message is not sent.

However, it is possible that the target SU contains one or more TEKs that the KMF is unaware of (for instance, through KFD provisioning). If this is the case and the target SU is unable to authenticate and decrypt the warm start key, a Rekey-Acknowledgment message containing an error code may be sent to the KMF encrypted on one of the TEKs. If the KMF does not have the selected TEK, any actions by the KMF are left to local policy. The selection of which TEK used when encrypting the Rekey-Acknowledgment message is left up to the discretion of the manufacturer.

Table 14 - Warm-Start Responses

Possible Responses	Status Octet Value	Circumstance
Rekey-Acknowledgement	$00 Command was performed	
	$01 Command not performed	- Attempted to add a key of an unsupported algorithm type - Invalid key length - Key Format bit indicated key is a KEK - Inner layer MI specified incorrectly - Bad parity on key - Message not properly authenticated - Optional field not supported - MFID is standardized and key data ALGID or inner layer ALGID is proprietary - SU already contains a TEK with the same ALGID and Key ID of the temporary TEK. - Other unidentified errors
	$04 Invalid MAC	- MAC for key transferred is invalid
	$06 Could not decrypt key	- KEK for key transfer does not exist
Delayed-Acknowledgement	$00 Command was performed	
	$01 Command not performed	- Attempted to add a key of an unsupported algorithm type - Invalid key length - Key Format bit indicated key is a KEK - Inner layer MI specified incorrectly - Bad parity on DES key - Message not properly authenticated - Optional field not supported - MFID is standardized and key data ALGID or inner layer ALGID is proprietary - SU already contains a TEK with the same ALGID and Key ID of the temporary TEK.
	$04 Invalid MAC	- MAC for key transferred is invalid
	$06 Could not decrypt key	- KEK for key transfer does not exist
No Response	N/A	- Failure (Reference KMM validation rules in Section 7.4)

An extension of the Warm Start procedure is the Reverse Warm Start Procedure. See Section 9.6 for more details.

8.17 **Zeroize Procedure**

The Zeroize procedure is used by the KMF to instruct the SU to erase all voice and data encryption keys (including all KEKs) from the SU. The Zeroize procedure is particularly useful when the SU is suspected of being compromised.

This procedure is initiated by the KMF sending a Zeroize-Command message (Section 10.2.31) to the target SU. The SU responds by deleting all of its keys and replying with a Zeroize-Response message (Section 10.2.32).

The Zeroize-Response message may be sent either encrypted or unencrypted depending on manufacturer SU configuration. If the Zeroize-Response is sent unencrypted, the following steps are followed:

1. Prior to deleting all keys, the SU constructs and authenticates the Zeroize-Response message using the same authentication key used to generate the MAC for the Zeroize-Command message.
2. The SU deletes all keys; excluding the subscriber authentication key or keys.
3. The Zeroize-Response message is transmitted to the KMF unencrypted. The response is clear since the key used to encrypt the Zeroize-Command has been deleted in step 2.

If the Zeroize-Response is sent encrypted, the following steps are followed:

1. Prior to deleting all keys, the SU constructs and authenticates the Zeroize-Response message using the same authentication key used to generate the MAC for the Zeroize-Command message.
2. The SU deletes all keys; excluding the key used to encrypt the Zeroize-Response message and the subscriber authentication key(s).
3. The Zeroize-Response message is encrypted and transmitted to the KMF.
4. The key used to encrypt the Zeroize-Response message is discarded.

The Zeroize procedure is shown in Figure 29.

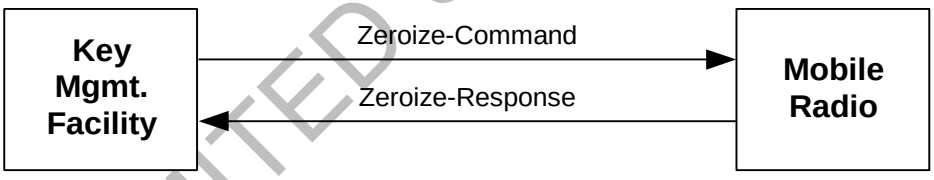


Figure 29 - Zeroize Procedure

The Zeroize-Command message is sent as a Response Kind 3 (Immediate) message. In this case, the SU responds with an immediate Zeroize-Response. If the KMF does not receive the Zeroize-Response, it may query the SU or repeat the Zeroize procedure as necessary. The Zeroize-Response message contains no message body and only provides an acknowledgement to the KMF.

The summary of the response kinds supported for the Zeroize procedure is shown in Table 2.

Rather than performing a rekey of all SUs that share keys with a suspected compromised unit, the Zeroize Procedure may be used by the KMF to delete all the keys in a particular SU. A successful execution of the Zeroize Procedure may result in the SU being unable to communicate securely with the KMF afterwards.

9 EXTENDED KEY MANAGEMENT PROCEDURES

When key management procedures are used in combination with other key management procedures to perform a more comprehensive key management operation, they are called extended Key Management procedures. This section provides a set of extended procedures that can be used for various operations such as bringing a new unit online and changing Key Management Groups. Table 15 outlines a set of extended procedures. These extended procedures assume that the SU is properly configured to communicate via OTAR.

Table 15- Extended Key Management Procedures

Procedure	Target	KMF Messages	SU Messages
New Unit	UNIT	Warm Start Rekey Changeover (possibly) Change RSI (add Group)	Rekey ACK Rekey ACK Changeover-Response Change RSI-Response
Change Key Management Group	UNIT	Modify Key/Rekey (CKEK/TEKs) Change RSI (modify)	Rekey ACK Change RSI-Response
Change Keyset	UNIT	Modify Key (TEKs) Changeover	Rekey ACK Changeover-Response
	GROUP	Modify Key (TEKs) Changeover	Delayed ACK Delayed ACK
Resynchronization of TEKs	UNIT	Warm Start Rekey Changeover (possibly)	Rekey ACK Rekey ACK Changeover-Response
Multiple Algorithm Rekey	UNIT	Rekey x n Algorithms Changeover (possibly)	Rekey ACK x n Changeover-Response
	GROUP	Rekey x n Algorithms Changeover (possibly)	Delayed ACK Delayed ACK
Reverse Warm Start	KMF	N/A	Registration-Command Deregistration-Command Unable-to-Decrypt-Response
SU Initiated Rekey Request	KMF	Rekey Procedure (possibly) Warm Start Procedure (possibly)	Hello-Command

9.1 New Unit

Before a new unit is able to receive keys from a KMF, it needs to be initially provisioned with certain OTAR parameters such as an individual RSI, KMF RSI, and a KEK. In addition, the message numbers associated with a “new unit” are reset to zero in both the SU and KMF, as defined in [21].

Once the SU has been configured with these basic parameters, the following sequence of key management procedures may be used to bring the SU to a state where it can communicate in a secure manner.

1. Warm-Start Procedure
2. Rekey Procedure

3. Changeover Procedure
4. Change-RSI Procedure

Figure 30 shows the New Unit extended Key Management procedure.

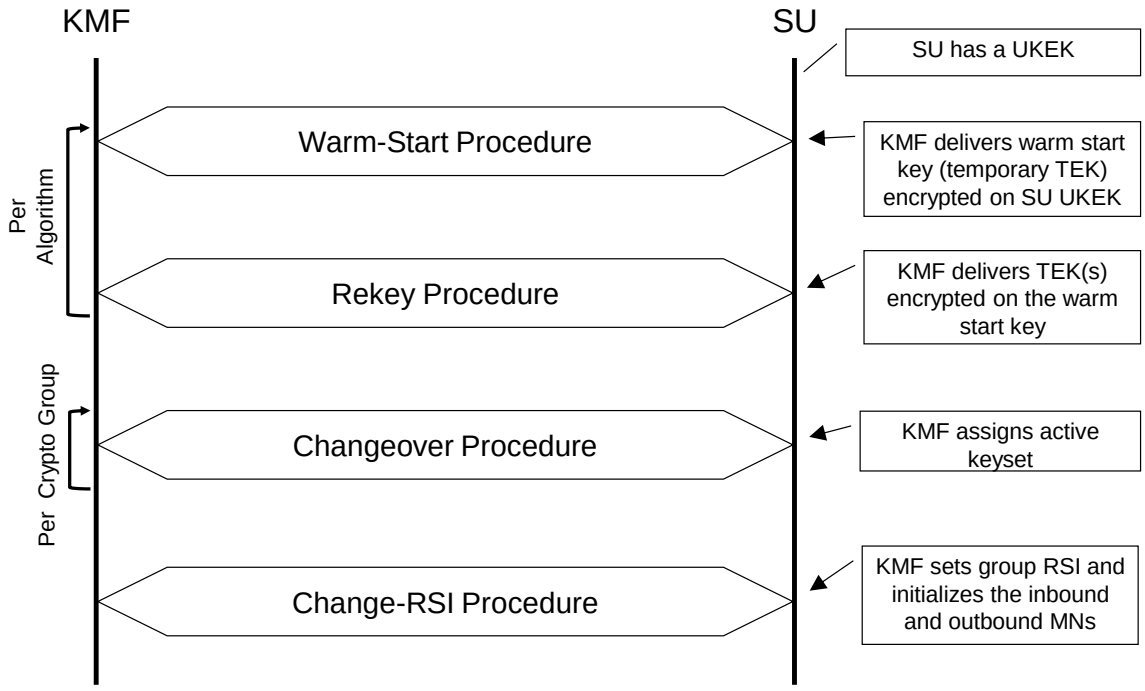


Figure 30 - Extended Key Management Procedure: New Unit

The Warm-Start Procedure transfers an initial warm start key (temporary TEK) encrypted on the SU UKEK, to the target SU via the Warm-Start-Command message. The warm start key is stored in the SU and is used until the SU is rekeyed.

The Warm-Start Procedure is followed by the Rekey Procedure in order to transfer TEK(s) to the SU. The Rekey-Command message sent by the KMF is both encrypted and authenticated using the warm start key. A successful Rekey Command triggers the SU to stop using the warm start key and use one of the new TEKs to send the Rekey response.

The Warm-Start Procedure and Rekey Procedure are repeated for each algorithm type.

If two or more keysets are rekeyed, the Changeover Procedure is performed to assign the active keyset.

If necessary, a Change-RSI Procedure is performed between the KMF and SU to assign a group RSI and synchronize the inbound and outbound group RSI Message Numbers. The Change-RSI-Command is sent as a Response Kind 3 (Immediate) message and is encrypted and authenticated using a TEK from the Rekey procedure. The Change-RSI response message is sent by the SU encrypted and authenticated also using a TEK from the Rekey procedure.

9.2 Change Key Management Group

When the KMF associates an SU with a different Key Management Group (KMG), the KMF shall change keys that are in that SU (except perhaps the UKEK) to those held by the new KMG. The following sequence of key management procedures may be used to move an SU from one KMG to another KMG.

1. Modify-Key Procedure or Rekey Procedure
2. Delete-Key Procedure (Optional)
3. Change-RSI Procedure

Figure 31 shows the procedure sequence for changing an SU to a new Key Management Group.

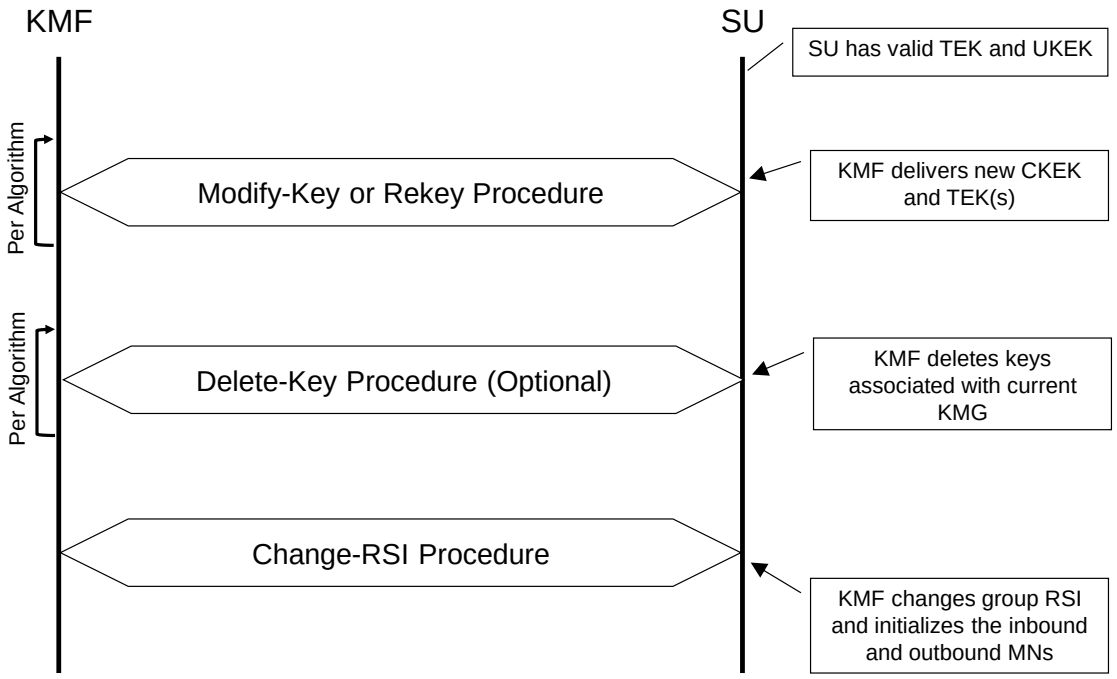


Figure 31 - Extended Key Management Procedure: Change KMG

The first step is the Modify-Key procedure or Rekey procedure in order to provide a new CKEK and new TEKs to the SU(s). The Modify-Key procedure or Rekey procedure may also be used as a method to overwrite the current keys in the SU if the Delete-Key procedure is not executed. The second step in this operation is the (optional) Delete-Key procedure. Afterward, the SU requires a Change-RSI procedure in order to change the Group RSI to the new KMG for group operations.

9.3 **Change Keyset**

This extended procedure would be used to completely modify one keyset in a particular Crypto Group and possibly make that keyset the active keyset for that CG.

1. Modify-Key Procedure
2. Changeover Procedure

Figure 32 shows the procedure sequence for changing keys within a particular keyset followed by the activation of that keyset.

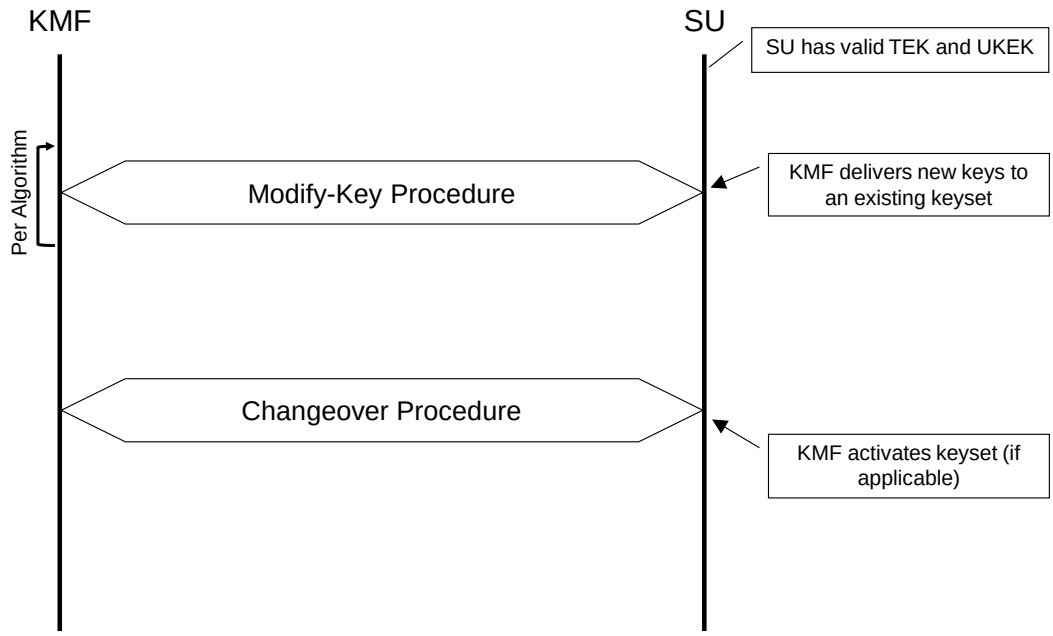


Figure 32 - Extended Key Management Procedure: Change Keyset

In order to do this, the Modify-Key Procedure would be used to deliver new keys to the particular keyset. If the new keyset has been designated as the active keyset (at the KMF), then the Changeover Procedure shall be used to activate the keyset.

9.4 Resynchronization of TEKs

Occasionally TEKs in an SU get out of sync with the KMF. For instance, an SU that has been off of the system for an extended amount of time may have missed several rekey cycles and does not have the proper TEK needed to decrypt a KMM. A Warm-Start Procedure followed by a Modify-Key Procedure or Rekey Procedure and possibly a Changeover Procedure allows the KMF to resynchronize the SU. A difference between the Modify-Key Procedure and the Rekey Procedure is the Rekey Procedure is capable of both modifying existing keysets and adding new keysets to the SU. The Modify-Key Procedure only modifies existing keysets. The Rekey-Command message sent by the KMF is both encrypted and authenticated using the warm start key. A successful Modify-Key or Rekey Command triggers the SU to stop using the warm start key and use one of the new TEKs to send the Rekey response.

1. Warm-Start Procedure
2. Modify-Key Procedure or Rekey Procedure
3. Changeover Procedure

The Warm-Start Procedure and Modify-Key Procedure or Rekey Procedure are repeated for each algorithm type.

Figure 33 shows the procedure sequence for bringing keys in an SU up to date followed by the activation of those keys.

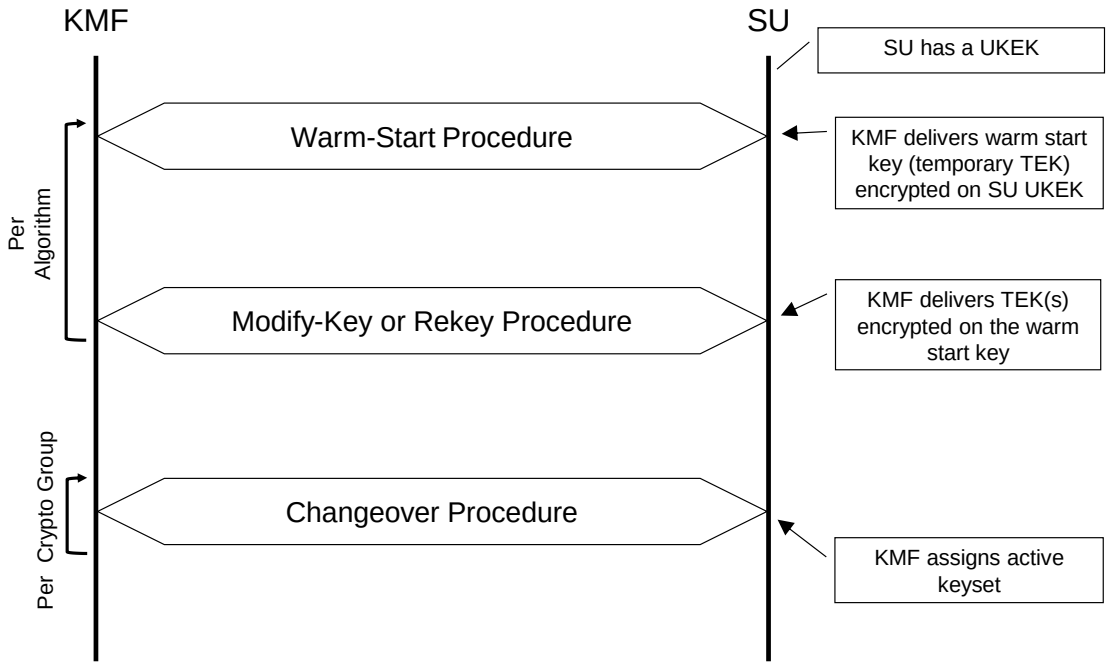


Figure 33 - Extended Key Management Procedure: Resynchronization of TEKs

9.5 Multiple Algorithm Rekey

This extended key management procedure provides key rekeying of an SU that supports multiple algorithms. In this example, back to back Modify-Key or Rekey Procedures are performed by the KMF, one for each algorithm. If the new keyset has been designated as the active keyset (at the KMF), then the Changeover Procedure shall be used to activate the keyset.

1. Modify-Key Procedure or Rekey Procedure (Repeat per Algorithm)
2. Changeover Procedure (Repeat per CG as needed)

Figure 34 shows the procedure sequence for rekeying an SU for multiple algorithms.

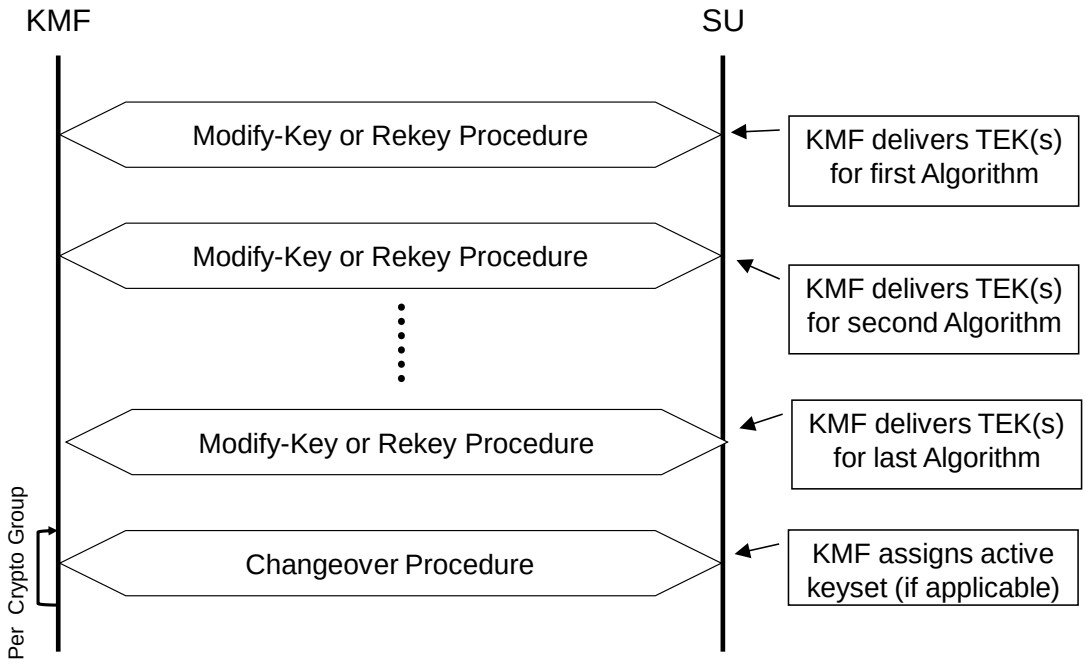


Figure 34 - Extended Key Management Procedure: Multiple Algorithm Rekey

9.6 Reverse Warm Start

There are two commands (Registration and Deregistration) and an Unable-to-Decrypt response that are initiated at the SU which may include an authentication key in the message. The authentication key is encrypted with a KEK and inserted into the optional segment of the message called the Reverse Warm Start segment. Typically, the Reverse Warm Start function is used only when the SU has determined that it does not have a valid TEK.

The Reverse Warm Start segment may be used when the SU does not contain a TEK but does contain a UKEK and it is required to authenticate (MAC) the message being sent. In this case the SU generates a temporary TEK that is used to authenticate the message. The temporary TEK shall not be used to encrypt the message since the receiving unit does not possess the key. Refer to Section 13.5 for more details on the use of message authentication.

Once the Registration, Deregistration, or UTD procedure is complete, the temporary TEK is discarded. For Registration and Deregistration, if the SU is expecting a response (i.e. a Response Kind 3 Registration or Deregistration command) the SU shall discard the temporary TEK immediately after the response is received and

processed. If a response is not expected (i.e. a Response Kind 1 Registration or Deregistration command), then the temporary TEK shall be discarded immediately upon sending the Registration or Deregistration command message. For UTD, the SU shall immediately discard the temporary TEK upon sending the UTD message.

The manufacturer security policy shall also be used to determine if the Reverse Warm Start segment is to be sent. If the security policy requires a condition that cannot be met by the Reverse Warm Start operation, then the message shall not be sent. For example, if the security policy requires that the message be encrypted for confidentiality then the Reverse Warm Start segment shall not be used since the key transmitted in the Reverse Warm Start segment cannot be used to encrypt the message. The manufacturer security policy may also disable this feature.

9.7 **SU Initiated Rekey Request**

This extended key management procedure is used by an SU to indicate to a KMF that it may need key management services such as current traffic keys. The SU makes this request through the use of an inbound Hello message and sets the Hello Flag value based on whether the SU currently has presence of a KEK or not (see Section 10.2.14). An SU that has a KEK may receive traffic keys via a Warm Start Procedure; however an SU that does not have a KEK requires the New Unit procedure.

If the Hello message is a response kind 3 then the KMF, depending on local policy, may rekey the SU, initiate a Warm Start procedure, inventory the SU or send a No Service message.

Figure 35 shows a message sequence chart for the SU Initiated Rekey Request extended key management procedure.

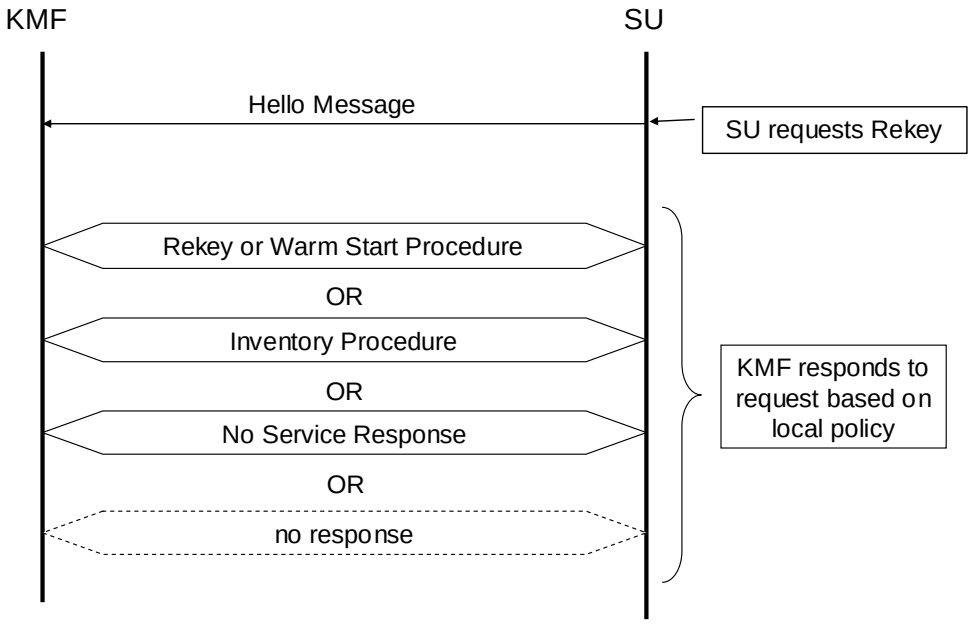


Figure 35 - Extended Key Management Procedure: SU Initiated Rekey Request

10 MESSAGE DEFINITIONS

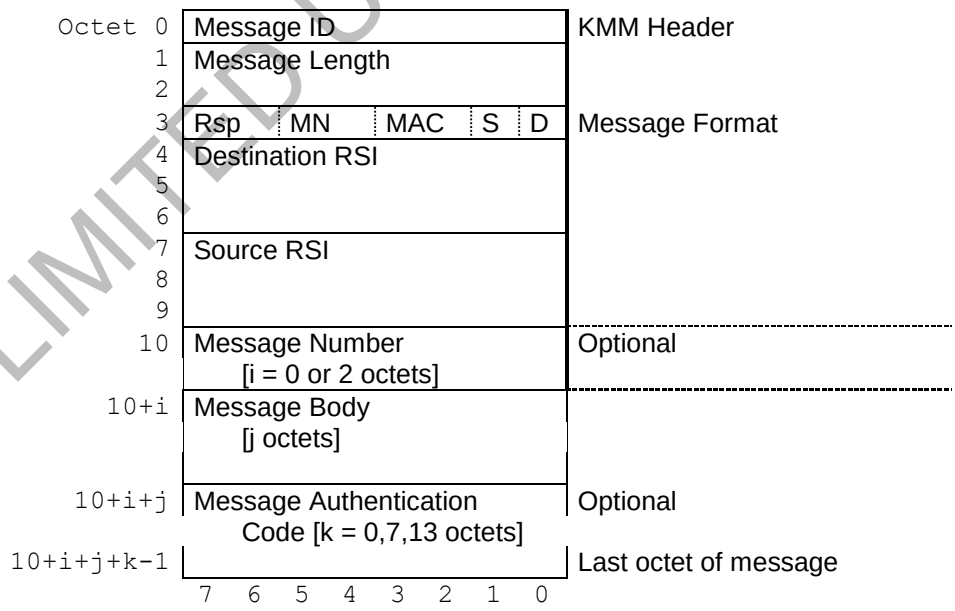
This section defines the format and structure of the OTAR Key Management Message (KMM). The message body is defined for each of the different messages (each defined message ID) in alphabetical order in Section 10.2. The Primitive Fields for each message are defined in Section 10.3.

The OTAR procedures outlined in Section 8 and Section 9 are comprised of OTAR messages defined within the sub clauses of this section. OTAR procedures that are deemed critical to interoperability contain messages that are by inclusion deemed critical to interoperability. In addition to those messages specifically identified within a procedure deemed critical to interoperability, the Negative Acknowledgement, No Service, and Unable-to-Decrypt messages shall be implemented as part of a manufacturer's interoperability goal.

10.1 Message Structure

The overall message format (as shown in Table 16) consists of the following eight fields: Message ID, Message Length, Message Format, Destination RSI, Source RSI, Message Number, Message Body, and Message Authentication Code (MAC). Some fields such as Message Number and MAC are optional.

Table 16 - Message Structure



Message ID - One of 255 possible values assigned as the message type. The message IDs are defined in Section 11 for Message ID Assignments. Zero is an invalid message type.

Message Length - A 16 bit binary number that defines the length (in octets) of the last 6 fields in the message. The minimum length is 7 octets.

Message Format - This octet identifies the selection of the response kind, the Message Number Length and the Message Authentication Type.

Rsp - Response Kind. Response Kind defines if the acknowledgment is to be returned to the sender of the KMM. A binary value of 00 (b7, b6 respectively) defines Response Kind 1 (None), a binary Value of 01 defines Response Kind 2 (Delayed), a binary value of 10 defines Response Kind 3 (Immediate) and a binary value of 11 is undefined.

MN - Message Number. The Message Number field defines the size of the Message Number field in the KMM. A binary value of 00 (b5, b4 respectively) defines that there is no message number and a binary value of 10 defines a 2 octet message number.

MAC (Message Authentication Code) - The MAC bits (b3, b2) define the authentication type used to validate the message.

00 – indicates that there is no message authentication;

01 – is Reserved;

10 – Indicates that the Enhanced algorithm based Message Authentication Code is used to validate the message;

11 – Indicates that the DES[6] algorithm based Message Authentication Code is used to validate the message.

S (Spare) – This bit (b1) is undefined and should be set to binary 0.

D (Done) – This optional bit (b0) is used to indicate that the current KMM is the last in a series of KMMs. A value of "1" indicates NOT DONE (more to follow) and a "0" indicates DONE (end of sequence). The default value for this bit shall be "0" and when not used this bit shall be "0".

Destination RSI - The Radio Set Identifier for the destination equipment or group of equipment. The RSI may be the same as the LLID. It may also be used by the KMF to provide address anonymity for encrypted KMMs.

Source RSI - The Radio Set Identifier for the equipment sending the message. The RSI may be the same as the LLID. It may also be used by the KMF to provide address anonymity for encrypted KMMs.

Message Number - The sequence number used to prevent message playback. The requirement for the Message Number field is defined in Section 13.6.

Message Body - Defined individually for each message. Definitions are provided in this section.

Message Authentication Code - A Message Authentication Code computed over the previous 7 fields if present. A field size of 0 indicates no MAC field is present, a field size of 7 indicates a DES[6] MAC, and a value of 13 indicates the Enhanced MAC. The requirements for this field are specified in Section 13.5.

10.2 **Message Body Definitions**

The different message bodies of a KMM are described in this section. The parameters and variables for each message body are further described in the Primitive Field Definitions (Section 10.3).

When transmitting, the first field in each message body is transmitted first, the most significant octet in each field is transmitted first, and the most significant bit in each octet is transmitted first.

Each KMM is encrypted, authenticated, and contains an MN unless stated otherwise. For details and exception cases, the reader is referred to Section 13 for message encryption rules, Section 13.5 for message authentication rules and Section 13.6 for Message Number rules.

[6] DES is only utilized for backward compatibility, see NOTE in section 1.1.

Section 12.4 provides additional information on maximum message length and block sizes.

For equipment that supports the optional “D” (Done) bit in the Message Field of the KMM header, multiple KMM response messages of a similar type may be sent in reply to a command where the number of parameters in the response exceeds the maximum message length. Each response message in the series would set the “D” bit to “1” indicating that another response of similar type is to follow. The “D” bit in the last message of the series would be set to “0” signifying the end of the series.

10.2.1 Capabilities-Command

The Capabilities-Command message is sent from the KMF to the SU and contains no message body.

10.2.2 Capabilities-Response

The Capabilities-Response message is sent from the SU to the KMF in response to a Capabilities-Command message. The message body format for the Capabilities-Response message is shown in Table 17.

Table 17 - Capabilities-Response Message Format

Octet 0	Number of Algorithms	
1	Algorithm ID	SEQUENCE [1..n] OF Algorithm IDs
	Subsequent Algorithm IDs (if they exist)	
n+1	Number of Optional Services	
n+2	Optional Service IDs	SEQUENCE [0..m] OF Optional Service IDs
	Subsequent Optional Service ID's (if they exist)	
m+n+2	Number of Message IDs	
m+n+3	Message ID	SEQUENCE [1..p] OF Message IDs
	Subsequent Message ID's (if they exist)	
p+m+n+2		Last octet of message
	7 6 5 4 3 2 1 0	

Number of Algorithms - The number of algorithms that the SU supports. The format for this field is defined in the Primitive Field Definition section for Number.

SEQUENCE OF Algorithm IDs - Sequence of Algorithm IDs for each algorithm that the SU supports.

Algorithm ID - The Algorithm ID identifies the algorithm used to encrypt the keys, data or voice or to identify the type of key sent in the OTAR message. The Algorithm IDs are defined in the Primitive Field Definitions for Algorithm ID.

Number of Optional Services - The number of optional services, such as, TOD, that the SU supports.

SEQUENCE OF Optional Service IDs - The list of Optional Service IDs that the SU supports.

Optional Service IDs - The Optional Service ID defines the optional services that the SU can support. The Optional Service IDs are defined in the Primitive Field Definitions section for Optional Service ID.

Number of Message IDs - The number of messages that the SU supports.

SEQUENCE OF Message IDs - The list of Message IDs that the SU supports.

Message ID - One of 255 possible values assigned as the message type. The message IDs are defined in Section 11 for Message ID Assignments. Zero is an invalid message type.

10.2.3 Change-RSI-Command

The Change-RSI-Command message is sent from the KMF to the SU to support the change of an individual or key management group RSI. The message body format is shown in Table 18.

Table 18 - Change-RSI-Command Message Format

Octet									
0	No. of Change-RSI Instruction								
1	RSI to be changed/deleted								SEQUENCE [1..n] OF Change-RSI Instructions
2									
3									
4	RSI to be added								
5									
6									
7	Message Number								
8									
	Subsequent Change-RSI Instructions (if they exist)]								
n*8									
	7	6	5	4	3	2	1	0	

No. of Change-RSI Instructions - The number of Change-RSI instructions in this message. The format for this field is defined in the Primitive Field Definition for Number.

SEQUENCE OF Change-RSI Instructions - The list of the RSI to be changed/deleted and the RSI to be added for each Change-RSI Instruction.

RSI to be changed/deleted - The first RSI identifies the individual or group identifier which is to be changed or deleted. The format for this field is defined in the Primitive Field Definition for RSI.

RSI to be added - The second RSI identifies the individual or group identifier which is to be added. The format for this field is defined in the Primitive Field Definition for RSI.

Message Number - The 16-bit initial value for both the inbound and outbound Message Numbers. The format for this field is defined in Section 13.6 Message Number Processing. When deleting a group RSI, this field is unused with a default value of all zeros.

10.2.4 Change-RSI-Response

The Change-RSI-Response message is sent from the SU to the KMF in response to a Change-RSI-Command message. The message body format for the Change-RSI-Response message is shown in Table 19.

Table 19 - Change-RSI-Response Message Format

Octet								
0	No. of Change-RSI Response							
1	RSI changed or deleted							SEQUENCE [1..n] OF Change-RSI Responses
2								
3								
4	RSI added							
5								
6								
7	Status							
	Subsequent Change-RSI Responses (if they exist)							
n*7								
	7	6	5	4	3	2	1	0

No. of Change-RSI Responses - The number of Change-RSI Responses in this message. The format for this field is defined in the Primitive Field Definition for Number.

SEQUENCE OF Change-RSI Response - The list of the RSI that was changed/deleted and the RSI that was added for each Change-RSI Instruction.

RSI changed or deleted - The first RSI identifies the individual or group identifier which was changed or deleted. The format for this field is defined in the Primitive Field Definition for RSI.

RSI added - The second RSI identifies the individual or group identifier which was added. The format for this field is defined in the Primitive Field Definition for RSI.

Status - The result of the Change-RSI request. The format for this field is defined in the Primitive Field Definition for Status.

10.2.5 Changeover-Command

The Changeover-Command message is sent from the KMF to the SU to support the change of the active keyset(s) at an individual or group of SUs. The message body format for the Changeover-Command message is shown in Table 20.

Table 20 - Changeover-Command Message Format

Octet									
0	No. of Changeover Instruction								
1	Superseded Keyset ID								SEQUENCE [1..n] OF Changeover Instructions
2	Activated Keyset ID								
n*2	Subsequent Changeover Instructions (if they exist)								
	7	6	5	4	3	2	1	0	

Number of Changeover Instructions - The number of changeover instructions in this message. The format for this field is defined in the Primitive Field Definition for Number.

SEQUENCE OF Changeover Instructions - The list of the Keyset ID to be superseded and the Keyset ID to be activated for each Changeover Instruction.

Superseded Keyset ID - The first Keyset ID identifies the keyset which needs to be superseded. The format for this field is defined in the Primitive Field Definition for Keyset ID.

Activated Keyset ID - The second Keyset ID identifies the keyset which should become activated. The format for this field is defined in the Primitive Field Definition for Keyset ID.

10.2.6 Changeover-Response

The Changeover-Response message is sent from the SU to the KMF in response to a Changeover-Command message. The message body format for the Changeover-Response message is shown in Table 21.

Table 21 - Changeover-Response Message Format

Octet									
0	No. of Changeover Responses								
1	Superseded Keyset ID								SEQUENCE [1..n] OF Changeover Responses
2	Activated Keyset ID								
n*2	Subsequent Changeover Responses (if they exist)								
	7	6	5	4	3	2	1	0	

No. of Changeover Responses - The number of Changeover Instruction Responses in this message. The format for this field is defined in the Primitive Field Definition for Number.

SEQUENCE [1..n] OF Changeover Responses - The list of the Keyset ID that was superseded and the Keyset ID that was activated for each Changeover Instruction.

Superseded Keyset ID - The first Keyset ID identifies the keyset which was superseded. A value of zero identifies that the specified keyset is not active (e.g., the changeover has already been performed or the SU is using a keyset ID that was not specified in the Changeover Instruction). The format for this field is defined in the Primitive Field Definition for Keyset ID.

Activated Keyset ID - The second Keyset ID identifies the keyset which became activate. A value of zero identifies that the specified keyset was not activated. The format for this field is defined in the Primitive Field Definition for Keyset ID.

10.2.7 Delayed-Acknowledgment

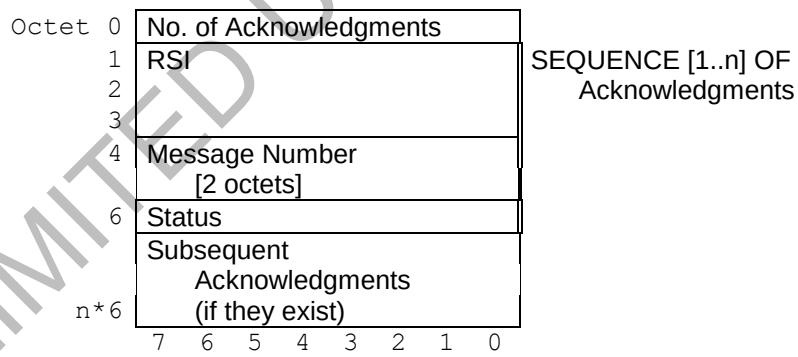
A Delayed-Acknowledgment message is the message used by the SU when replying to a Response Kind 2 (Delayed) command. A Delayed-Acknowledgment message is sent to the KMF after some delay determined by the SU (section 7.1.2).

The Delayed-Acknowledgment message contains the group or individual RSI, the message number, and an abbreviated status of the last N messages received that have requested a Response Kind 2 (Delayed) or Response Kind 3 (Immediate) response, where the value of N shall be a minimum of 4. In addition to being included in the Delayed-Acknowledgment, responses to Response Kind 3 (Immediate) messages are still transmitted immediately. The Delayed-Acknowledgement message also provides the pass/fail status for each of the N commands. In the event that a Delayed-Acknowledgement message contains one or more error status codes, the KMF is responsible for interpreting the error codes and determining some course of action.

The maximum number of status values returned in a Delayed-Acknowledgment message is limited by the maximum KMM size as defined in section 12.4.

The Delayed-Acknowledgment message body format is shown in Table 22.

Table 22 - Delayed-Acknowledgment Message Format



No. of Acknowledgments - The number of messages to be acknowledged. The format for this field is defined in the Primitive Field Definition section for Number.

SEQUENCE OF Acknowledgments - The list of the RSI, Message Number, and status for each acknowledgment.

RSI - The RSI for the individual or group of equipment receiving the message. The format for this field is defined in the Primitive Field Definitions section for RSI.

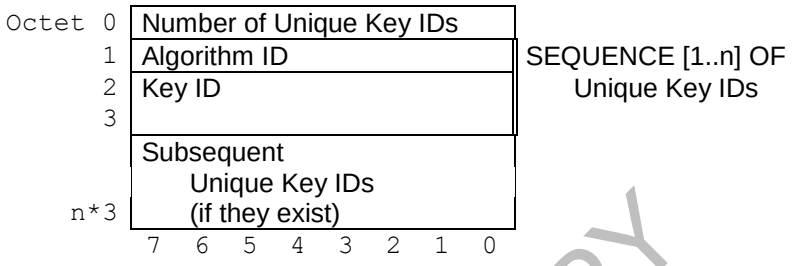
Message Number - The Message Number for the received command. The size of this field is 2 octets. The format for this field is defined in Section 13.6 - Message Number Processing.

Status - The pass/fail status of the received command. The format for this field is defined in the Primitive Field Definition for Status.

10.2.8 Delete-Key-Command

The Delete-Key-Command message is sent from the KMF and is used to delete keys stored in a target SU. The message body format of the Delete-Key-Command message is shown in Table 23.

Table 23 - Delete-Key-Command Message Format



Number of Unique Key IDs - The number of keys to be deleted. The format for this field is defined in the Primitive Field Definition for Number.

SEQUENCE OF Unique Key IDs - The list of the Algorithm ID and the Key ID for each KEK or TEK to be deleted.

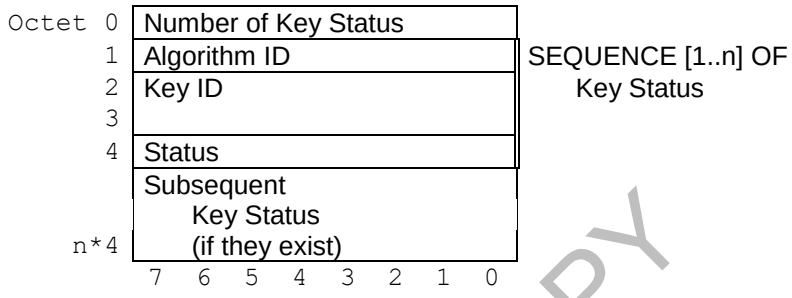
Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select a key. The format for this field is defined in the Primitive Field Definition for Algorithm ID.

Key ID - The Key ID of the key. The format for this field is defined in the Primitive Field Definition for Key ID.

10.2.9 Delete-Key-Response

The Delete-Key-Response message is sent from the SU to the KMF in response to a Delete-Key-Command message. The message body format for the Delete-Key-Response message is shown in Table 24.

Table 24 - Delete-Key-Response Message Format



Number of Key Status - The number of keys that were requested to be deleted. The format for this field is defined in the Primitive Field Definition section for Number.

SEQUENCE OF Key Status - The list of the Algorithm ID, Key ID and status for each key that was requested to be deleted.

Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select a key. The format for this field is defined in the Primitive Field Definition for Algorithm ID.

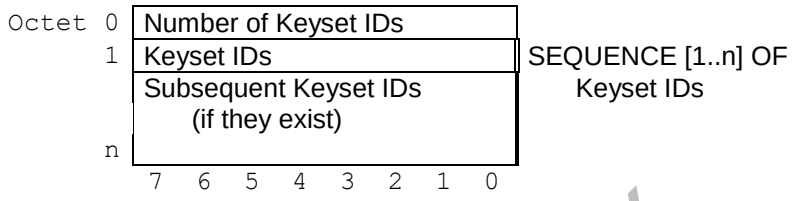
Key ID - The Key ID of the key. The format for this field is defined in the Primitive Field Definition for Key ID.

Status - The status of the key. The format for this field is defined in the Primitive Field Definition for Status.

10.2.10 Delete-Keyset-Command

The Delete-Keyset-Command message is sent from the KMF to the SU and is used to delete one or more keysets from an individual SU or group of SUs. The message body format for the Delete-Keyset-Command message is shown in Table 25.

Table 25 - Delete-Keyset-Command Message Format



Number of Keyset IDs - The number of keysets to be deleted. The format for this field is defined in the Primitive Field Definition for Number.

SEQUENCE OF Keyset IDs - The list of the Keyset ID for each keyset to be deleted.

Keyset ID - The Keyset ID of the keyset. The format of this field is defined by the Primitive Field Definition for Keyset ID.

10.2.11 Delete-Keyset-Response

The Delete-Keyset-Response message is sent from the SU to the KMF in response to the Delete-Keyset-Command message. The message body format for the Delete-Keyset-Response message is shown in Table 26.

Table 26 - Delete-Keyset-Response Message Format

Octet									
0	Number of Keyset Status								
1	Keyset ID								SEQUENCE [1..n] OF Keyset Status
2	Status								
n*2	Subsequent Keyset Status (if they exist)								
	7	6	5	4	3	2	1	0	

Number of Keyset Status - The number of keysets that were requested to be deleted. The format for this field is defined in the Primitive Field Definition for Number.

SEQUENCE OF Keyset Status - The list of the Keyset ID and the Status for each keyset that was requested to be deleted.

Keyset ID - The Keyset ID of the keyset. The format for this field is defined in the Primitive Field Definition for Keyset ID.

Status - The status of the keyset. The format for this field is defined in the Primitive Field Definition for Status.

10.2.12 Deregistration-Command

The Deregistration Command is sent from the SU to the KMF. This command is used to let the KMF know the SU is unavailable for OTAR service.

The message body format for the Deregistration-Command message is shown in Table 27.

Table 27 - Deregistration Command Message Format

Octet		
0	Body Message Format	Message Sub-header
1	KMF RSI	
2		
3		End of Message Sub-header
4	Decryption Instruction Format	Start of Reverse Warm Start & Decryption Instruction Block (optional)
5	Algorithm ID	
6	Key ID	
7		
8	Message Indicator (Optional) [m octets (0 or 9)]	End of Decryption Instruction Block
m+8	Key Length	
m+9	Algorithm ID	
m+10	Key Format	Unique Key Item Block
m+11	Storage Location Number	
m+13	Key ID	
m+15	Key [n octets (key length)]	
m+n+15	Key Name (Optional) [x octets (0..31)]	
m+n+x+14		End of Key Item Block & Reverse Warm Start Segment
	7 6 5 4 3 2 1 0	

Message Sub-header Segment

Body Message Format – The Body Message Format field is defined in the Primitive Fields Definitions.

KMF RSI –This is the KMF RSI.

Reverse Warm Start Segment (optional) – This segment is only included when the Reverse Warm Start encrypted TEK is included in the command as indicated by the T bit in the Message Format octet being set (T=1).

Decryption Instruction Block

Decryption Instruction Format - Indicates which optional fields are included in the Decryption Instructions Block field. The format for this field is defined in the Primitive Field Definition Decryption Instruction Format.

Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select the key used to encrypt the Unique Key Item Block. The format for this field is defined in the Primitive Field Definition for Algorithm ID.

Key ID - The Key ID of the key. The format for this field is defined in the Primitive Field Definition for Key ID.

Message Indicator (Optional) - Provides the Message Indicator (encryption synchronization) for encryption algorithms which do not support the Electronic Codebook mode of operation. The format for this field is defined in the Primitive Field Definitions.

Key Length - The number of octets used to transfer the key. The format for this field is defined in the Primitive Field Definitions for Number.

Algorithm ID - The algorithm to be used with the key in this message. The format for this field is defined in the Primitive Field Definitions for Algorithm ID.

Unique Key Item Block - The key and associated tagging information. This consists of the Storage Location Number, the Key ID, the Key, and the Key Name (Optional).

Key Format - Indicates which optional fields are included in the Unique Key Item Block field. The format for this field is defined in the Primitive Field Definitions for Key Format.

Storage Location Number - This field is invalid for the Deregistration command and shall default to all zeros.. The key being transferred in the Deregistration command is a temporary key and shall be deleted after completing the deregistration procedure.

Key ID - The Key ID assigned to the key in the Key field. The format for this field is defined in the Primitive Field Definition for Key ID.

Key - The Traffic Encryption Key. The format of this field is defined in the Primitive Field Definition for Key.

Key Name (Optional) - This field is invalid for the Deregistration command and shall default to all zeros.

10.2.13 Deregistration-Response

The Deregistration Response is sent from the KMF to the SU to indicate the success or failure of the deregistration request.

The message body format of the Deregistration-Response message is shown in Table 28.

Table 28 - Deregistration Response Message Format

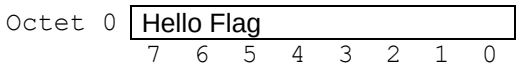
	Status
Octet 0	7 6 5 4 3 2 1 0

Status – Indicates whether the deregistration request was successful or not. The format for this field is defined in the Primitive Field Definition for Status.

10.2.14 Hello

The Hello message can be sent by either the SU or the KMF. The contents of the Hello message body is ignored by an SU receiving the message. The KMF has the opportunity to perform an inventory on the SU or rekey the unit if the Hello Flag field is non-zero. The format of the message body for the Hello message is shown in Table 29.

Table 29 - Hello Message Format



Hello Flag - The Hello Flag is used by the SU to request a rekey. The format for this field is defined in the Primitive Field Definition for Hello Flag.

10.2.15 Inventory-Command

The Inventory-Command message is sent from the KMF to the SU. The message body format for the Inventory-Command message is shown in Table 30.

Table 30 - Inventory-Command Message Format

Octet 0	Inventory Type							
	7	6	5	4	3	2	1	0

Inventory Type - The Inventory Type specifies which items need to be sent in the Inventory Response message. The format for this field is defined in the Primitive Field Definitions for Inventory Type.

10.2.16 Inventory-Response

The Inventory-Response message is sent from the SU to the KMF in response to the Inventory-Command message. The message body format for the Inventory-Response message is shown in Table 31.

Table 31 - Inventory-Response Message Format

Octet									
0	Inventory Type								
1	Number of Items								
2									
3	CHOICE OF one inventory type (defined in sections 10.2.16.1 - 10.2.16.11)								(optional)
i+2	[i octets]								
	7	6	5	4	3	2	1	0	

Inventory Type - The Inventory Type field indicates which item was requested to be sent in the Inventory Response message. The format for this field is defined in the Primitive Field Definitions section for Inventory Type.

Number of items - The number of items in the response. The format for this field is defined in the Primitive Field Definitions section for Long Number. Note that this field may represent a number of items equal to zero (i.e. no items to report in the requested inventory type).

CHOICE OF one Inventory Type:

1. Send Current Date-Time.
2. List Active Keyset IDs.
3. List Inactive Keyset IDs.
4. List Active Key IDs.
5. List Inactive Key IDs.
6. List Keyset Tagging Information.
7. List Unique Key Information.
8. List Key Assignment Items for CSSs
9. List Key Assignment Items for TGs.
10. List Long Key Assignment Items for LLIDs
11. List RSI Items.

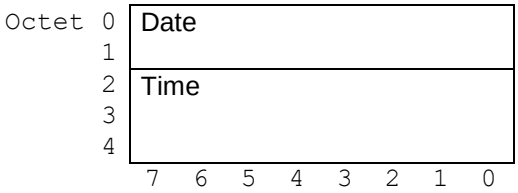
The Choice of One Inventory Type fields are defined in Sections 10.2.16.1 through 10.2.16.11.

If the size of the Inventory-Response KMM exceeds the maximum KMM size then the “D” bit may be used to send multiple response KMMs (see Section 10.2).

10.2.16.1 Send Current Date-Time

The Send Current Date-Time field is used to send the current date and time. The format for this field is shown in Table 32.

Table 32 - Send Current Date-Time Format



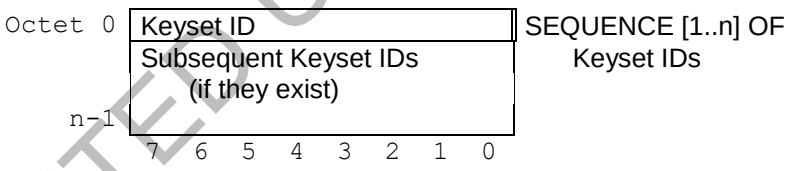
Date - The Date field is used to send the current date. The format for this field is defined in the Primitive Field Definition for Date.

Time - The Time field is used to send the current time. The format for this field is defined in the Primitive Field Definition for Time.

10.2.16.2 List Active Keyset IDs

The List Active Keyset IDs field is used to list the keyset IDs which are currently in use by the SU. The format for this field is shown in Table 33.

Table 33 - List Active Keyset IDs Format



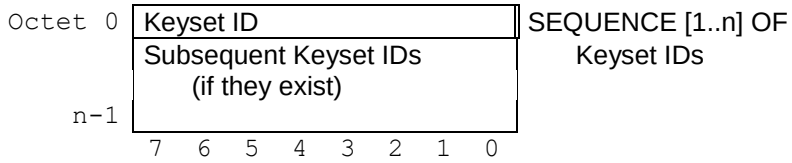
SEQUENCE OF Keyset IDs - The list of keyset IDs which are currently in use by the SU.

Keyset ID - The Keyset ID of the keyset. The format of this field is defined by the Primitive Field Definition for Keyset ID.

10.2.16.3 List Inactive Keyset IDs

This field is used to list the inactive Keyset IDs stored by the SU. The format for this field is shown in Table 34.

Table 34 - List Inactive Keyset IDs Format



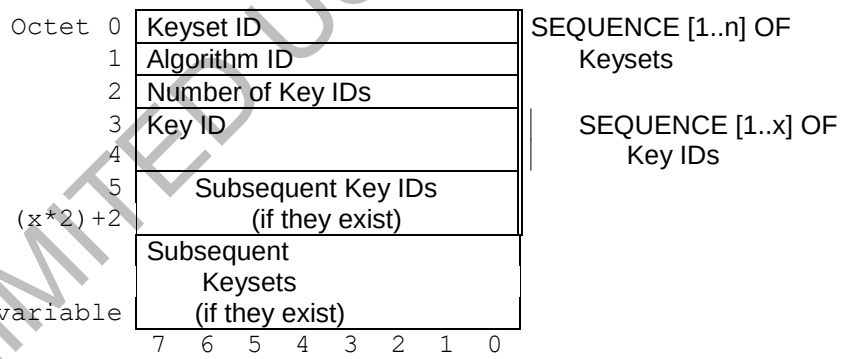
SEQUENCE OF Keyset IDs - The list of keyset IDs which are in storage but not currently in use by the SU.

Keyset ID - The Keyset ID of the keyset. The format of this field is defined by the Primitive Field Definition for Keyset ID.

10.2.16.4 List Active Key IDs

The List Active Key IDs is used to list the Key IDs for the active keysets which are currently in use by the SU. The format for this field is shown in Table 35.

Table 35 - List Active Key IDs Format



SEQUENCE OF Keysets - The list of keysets which are active in the SU.

Keyset ID - The Keyset ID of the keyset. The format of this field is defined by the Primitive Field Definition for Keyset ID.

Algorithm ID - The Algorithm ID of the keys used in the keyset. The format for this field is defined in the Primitive Field Definitions for Algorithm ID.

Number of Key IDs - The number of Key IDs in the keyset. The format for this field is defined in the Primitive Field Definition for Number.

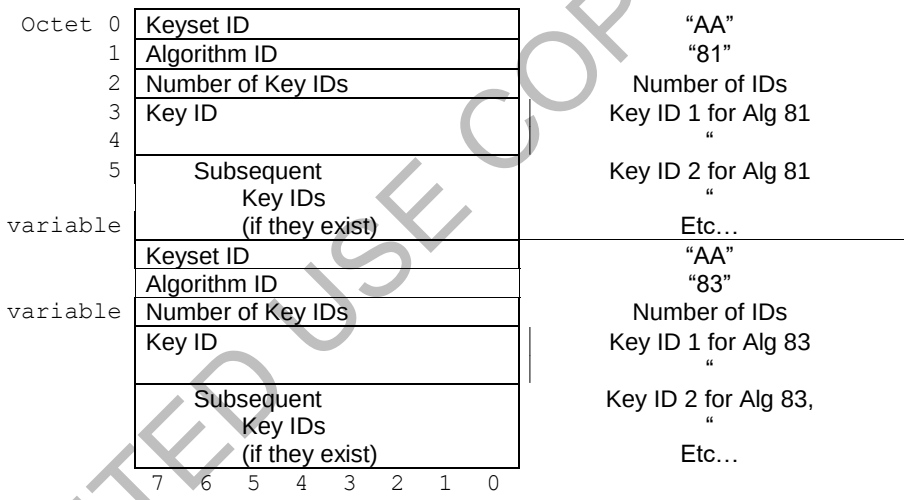
SEQUENCE OF Key IDs - The list of Key IDs in the keyset

Key ID - The Key ID of the keys in the keyset. The format of this field is defined by the Primitive Field Definition for Key ID.

Keysets may contain active keys from more than one algorithm. With this in mind, the List Active Key IDs response recursively specifies the active Key IDs for each algorithm type within the identified keyset. This is done by first specifying the active keys for the first algorithm within the keyset, followed by the active keys for the second algorithm (if any), and so on.

Assuming keyset “AA” has a set of keys for Algorithm ID “81” and another set of keys for Algorithm ID “83”, the “List Active Key IDs” response would look something like the example shown in Table 36.

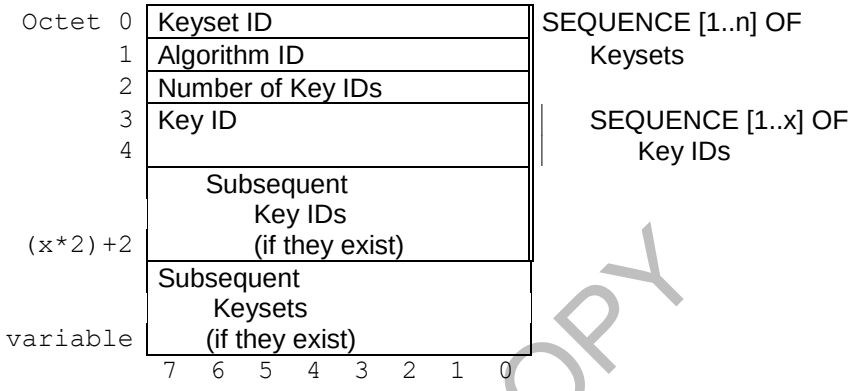
Table 36 - List Active Key ID Multiple Algorithm example



10.2.16.5 List Inactive Key IDs

The List Inactive Key IDs is used to list the Key IDs in the inactive keysets stored by the SU. The format for this field is shown in Table 37.

Table 37 - List Inactive Key IDs Format



SEQUENCE OF Keysets - The list of keysets which are inactive in the SU.

Keyset ID - The Keyset ID of the keyset. The format of this field is defined by the Primitive Field Definition for Keyset ID.

Algorithm ID - The Algorithm ID of the keys used in the keyset. The format for this field is defined in the Primitive Field Definitions for Algorithm ID.

Number of Key IDs - The number of Key IDs in the keyset. The format for this field is defined in the Primitive Field Definition for Number.

SEQUENCE OF Key IDs - The list of Key IDs in the keyset

Key ID - The Key ID of the keys in the keyset. The format of this field is defined by the Primitive Field Definition for Key ID.

Keysets may contain inactive keys from more than one algorithm. With this in mind, the List Key IDs in Storage response can recursively specify the inactive Key IDs for each algorithm type within the identified keyset. This is done by first specifying the inactive keys for the first algorithm within the keyset, followed by the inactive keys for the second algorithm (if any), and so on.

For example, keyset “AA” has a set of keys for Algorithm ID “81” and another set of keys for Algorithm ID “83”, the “List Key IDs in Storage” response would appear similar to that shown in Figure 36.

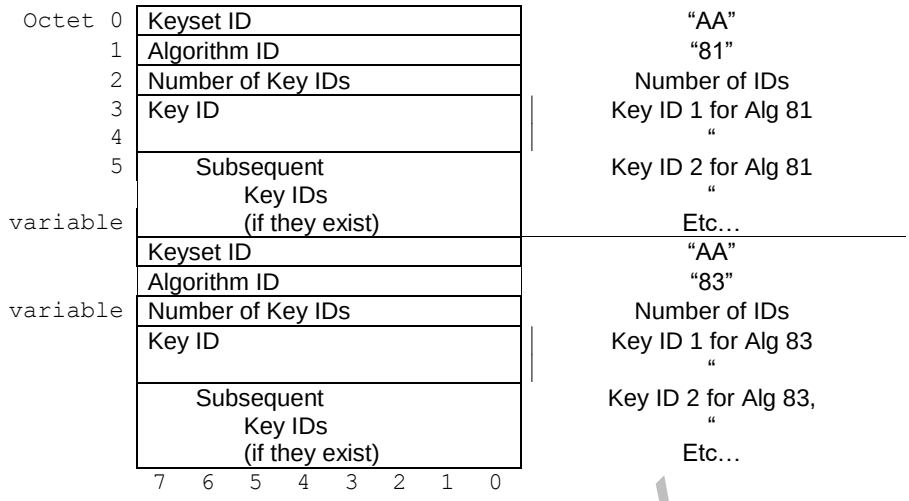


Figure 36 - List Inactive Key IDs Multiple Algorithm Example

10.2.16.6 List Keyset Tagging Information

The List Keyset Tagging Information field is used to provide the keyset tagging information for each of the keysets in the SU (both active and inactive keysets). The format for this field is shown in Table 38.

Table 38 - List Keyset Tagging Information Format

Octet 0	Keyset Format	SEQUENCE [1..n] of
1	Keyset ID	Keyset Tagging Information
2	Reserved	Reserved
3	Reserved	Reserved Block
4		(optional)
5		End of Block [0 or 3 octets]
3 or 6	Date	Date and Time Block
4 or 7		(optional)
5 or 8	Time	
6 or 9		
7 or 10		End of Block [0 or 5 octets]
3 or 6 or 11	Keyset Name (Optional) [i= 0..31]	
3+i or 6+i or 11+i	Subsequent Keyset Tagging Information	
variable	(if they exist)	

7 6 5 4 3 2 1 0

SEQUENCE OF Keyset Tagging Information - The list of keyset tagging information for each of the keysets in the SU (both the active and inactive keysets). The keyset tagging information used to identify a key (or group of keys), to control the rekey of a key (or group of keys) or to automatically select a new key (or group of keys).

Keyset Format - Indicates which optional fields are included in the Keyset Tagging Information block and the length of the Keyset Name. The format for this field is defined in the Primitive Field Definitions for Keyset Format.

Keyset ID - The Keyset ID. The format for this field is defined in the Primitive Field Definitions for Keyset ID.

Reserved – This field is reserved and is defaulted to all zeros. Note that some legacy equipment may populate this field with a non-zero value, and this field should therefore be ignored by the receiving endpoint. This field previously specified the Algorithm ID.

Reserved Block (optional) - The Reserved Block is an optional 3 octet field. This field is present if bit 6 of the Keyset Format octet is set to a 1.

Date and Time Block (optional) - The date (month, day and year) and time (hours, minutes and seconds) the keyset is to be activated. This field is present if bit 5 of the Keyset Format octet is set to a 1.

Date - The Date field is used to send the date. The format for this field is defined in the Primitive Field Definition for Date.

Time - The Time field is used to send the time. The format for this field is defined in the Primitive Field Definition for Time.

Keyset Name (optional) - An identifier (name) which can be displayed to the user to indicate which keyset is currently selected. Bit 4 through bit 0 Least Significant Bits (LSB) of the Keyset Format octet define the length of this field. If the length is zero then this field is not present. The format for this field is defined in the Primitive Field Definitions for Keyset Name.

10.2.16.7 List Unique Key Information

The List Unique Key Information is used to provide the unique identifying information for each key in the SU (both active and inactive keys). The format for this field is shown in Table 39.

Table 39 - List Unique Key Information Format

Octet									
0	Storage Location Number								SEQUENCE [1..n] OF Unique Key Information
1									
2	Algorithm ID								
3	Key ID								
4									
	Subsequent Unique Key Information (if they exist)								
(n*5)-1									
	7	6	5	4	3	2	1	0	

SEQUENCE of Unique Key Information - The unique identifying information for each key in the SU.

Storage Location Number - Identifies one of the 65536 possible storage location indexes where the key is stored. The format for this field is defined in the Primitive Field Definitions section for Storage Location Number.

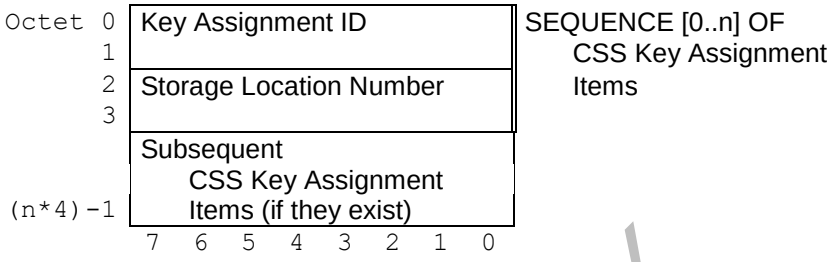
Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select a key. The format for this field is defined in the Primitive Field Definition section for Algorithm ID.

Key ID - The Key ID of the key. The format for this field is defined in the Primitive Field Definition section for Key ID.

10.2.16.8 List Key Assignment Items for CSSs

The List Key Assignment Items for CSSs is used to provide Channel Selector Switch position to SLN associations. The format for this field is shown in Table 40.

Table 40 - List Key Assignment Items for CSSs Format



SEQUENCE OF CSS Key Assignment Items - A list of Channel Selector Switch positions with their associated SLN.

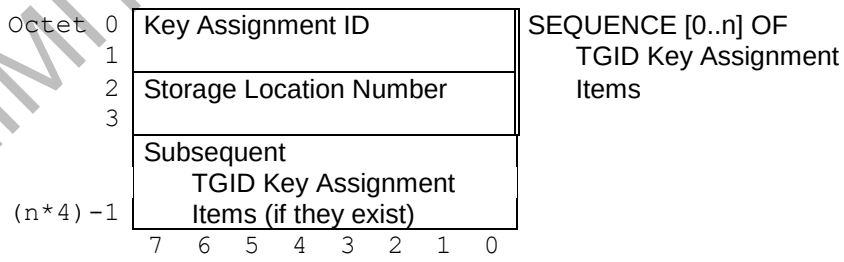
Key Assignment ID - The 16-bit identifier for the Channel Selection Switch position to which the SLN is assigned. The format for this field is defined in the Primitive Field Definition section for Key Assignment Identifier.

Storage Location Number - Identifies one of the 65536 possible storage location indexes where the key(s) is(are) stored. The format for this field is defined in the Primitive Field Definitions section for Storage Location Number.

10.2.16.9 List Key Assignment Items for TGs

The List Key Assignment Items for TGs field is used to provide Talk Group ID to SLN associations. The format for this field is shown in Table 41.

Table 41 - List Key Assignment Items for TGs Format



SEQUENCE OF TGID Key Assignment Items - A list of Talk Group IDs with their associated SLN.

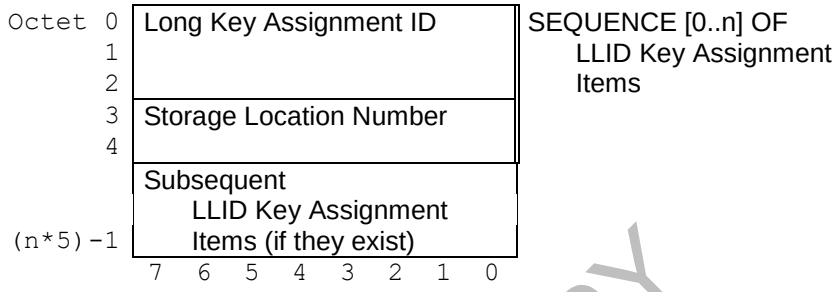
Key Assignment ID - The 16-bit identifier for the Talk Group to which the SLN is assigned. The format for this field is defined in the Primitive Field Definition for Key Assignment Identifier.

Storage Location Number - Identifies one of the 65536 possible storage location indexes where the key(s) is(are) stored. The format for this field is defined in the Primitive Field Definitions for Storage Location Number.

10.2.16.10 List Long Key Assignment Items for LLIDs

The List Long Key Assignment Items for LLIDs is used to provide Logical Link ID to SLN associations. The format for this field is shown in Table 42.

Table 42 - List Long Key Assignment Items for LLIDs Format



SEQUENCE OF LLID Key Assignment Items - A list of Logical Link IDs with their associated SLN.

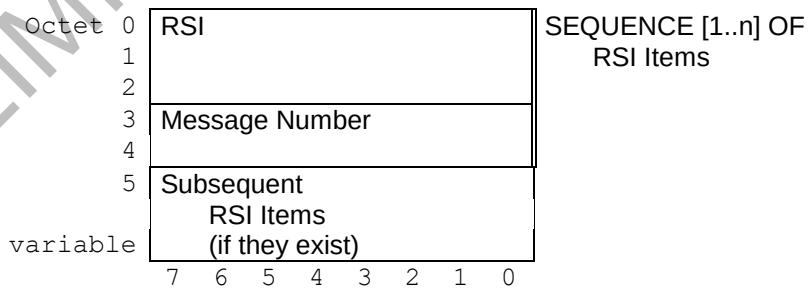
Long Key Assignment ID - The 24-bit identifier for the Logical Link ID to which the SLN is assigned. The format for this field is defined in the Primitive Field Definition for Long Key Assignment Identifier.

Storage location Number - Identifies one of the 65536 possible storage location indexes where the key(s) is(are) stored. The format for this field is defined in the Primitive Field Definitions for Storage Location Number.

10.2.16.11 List RSI Items

The List RSI Items is used to list the RSI and the associated Message Number for each individual and group RSI contained in the SU. The format for this field is shown in Table 43.

Table 43 - List RSI Items Format



SEQUENCE of RSI Items - The list of the RSI and the associated Message Number for each individual and key management group RSI contained in the SU.

RSI - The RSI for the individual or group of equipment receiving the message. The format for this field is defined in the Primitive Field Definitions for RSI.

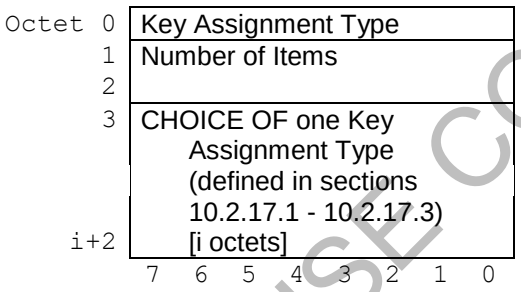
Message Number - The Message Number for the last valid received command addressed to the current RSI item. The format for this field is defined in Section 13.6 - Message Number Processing.

10.2.17 Key-Assignment-Command

The Key-Assignment-Command message is sent from the KMF to the SU and provides the associations between encryption key(s) and a particular channel (conventional), Talk Group ID (trunked), or Logical Link ID (data). CSS assignments are used for conventional channels when Talk Groups are not utilized. When TG assignments are used for Conventional channels, CSS assignments are invalid. CSS assignments are never valid for Trunked systems. LLID assignments are used for Layer 2 CAI data.

The message body format for the Key-Assignment-Command message is shown in Table 44.

Table 44 - Key-Assignment-Command Message Format



Key Assignment Type - The Key Assignment Type field indicates which group (Channel Selector Switch, Talk Groups, or data groups) SLNs is to be assigned to. The format for this field is defined in the Primitive Field Definitions for Key Assignment Type.

Number of items - The number of items in which keys are to be assigned. The format for this field is defined in the Primitive Field Definitions for Long Number.

CHOICE OF one Key Assignment Type:

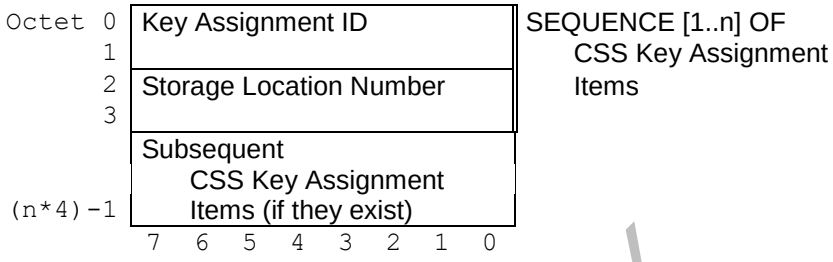
1. List/Assign SLNs to CSSs.
2. List/Assign SLNs to TGs.
3. List/Assign SLNs to LLIDs.

The Key Assignment Type Values are defined in the Primitive Field Definitions for Key Assignment Type.

10.2.17.1 Assignment of SLNs to CSS Positions

When assigning an SLN to a Channel Selector Switch, the Assignment of SLNs to CSS Positions field is used. The format for this field is shown in Table 45.

Table 45 - Assign SLNs to CSSs Format



SEQUENCE OF CSS Key Assignment Items - A list of Channel Selector Switch positions with their associated SLN.

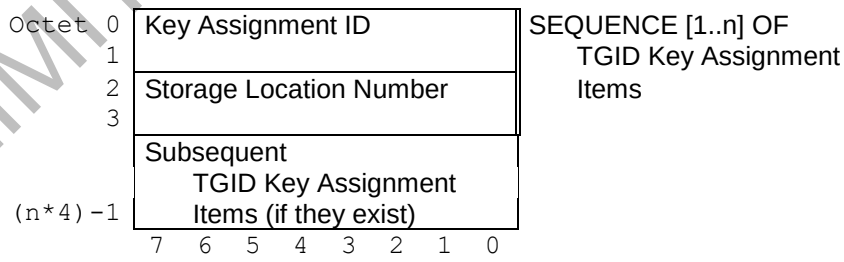
Key Assignment ID - The 16-bit identifier for the Channel Selection Switch position to which a SLN is to be assigned. The format for this field is defined in the Primitive Field Definition for Key Assignment Identifier.

Storage Location Number - Identifies one of the 65536 possible storage locations where the key(s) is(are) is stored. The format for this field is defined in the Primitive Field Definitions for Storage Location Number.

10.2.17.2 Assignment of SLNs to Talk Groups

When assigning an SLN to a Talk Group, the Assignment of SLNs to Talk Groups field is used. The format for this field is shown in Table 46.

Table 46 - Assign SLNs to TGs Format



SEQUENCE OF TGID Key Assignment Items - A list of Talk Group IDs with their associated SLN.

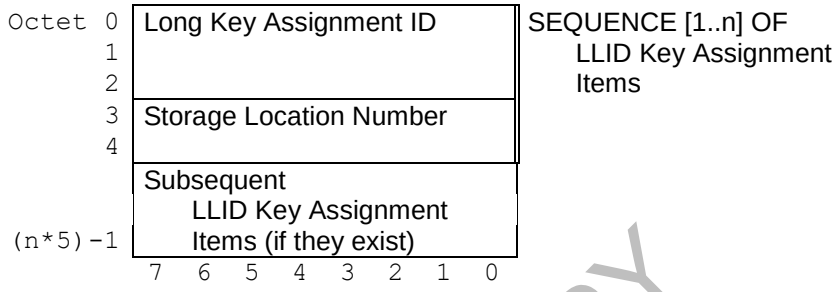
Key Assignment ID - The 16-bit identifier for the Talk Group to which a SLN is to be assigned. The format for this field is defined in the Primitive Field Definition for Key Assignment Identifier.

Storage Location Number - Identifies one of the 65536 possible storage locations where the key(s) is(are) stored. The format for this field is defined in the Primitive Field Definitions for Storage Location Number.

10.2.17.3 Assignment of SLNs to Logical Link IDs

When assigning an SLN to a data Logical Link ID, the Assignment of SLNs to Logical Link IDs field is used. The format for this field is shown in Table 47.

Table 47 - Assign SLNs to LLIDs Format



SEQUENCE OF LLID Key Assignment Items - A list of Logical Link IDs with their associated SLN.

Long Key Assignment ID - The 24-bit identifier for the Logical Link ID to which a SLN is to be assigned. The format for this field is defined in the Primitive Field Definition for Long Key Assignment Identifier.

Storage location Number - Identifies one of the 65536 possible storage locations where the key(s) is(are) stored. The format for this field is defined in the Primitive Field Definitions for Storage Location Number.

10.2.18 Key-Assignment-Response

The Key-Assignment-Response message is sent from the SU to the KMF in response to the Key-Assignment message. The message body format of the Key-Assignment-Response message is shown in Table 48.

Table 48 - Key-Assignment-Response Message Format

Octet	Key Assignment Type
1	Number of Items
2	
3	CHOICE OF one Key Assignment Type (defined in sections 10.2.18.1 - 10.2.18.3) [i octets]
i+2	

7 6 5 4 3 2 1 0

Key Assignment Type - The Key Assignment Type field indicates which group (Channel Selector Switch, Talk Groups, or data groups) SLNs are assigned. The format for this field is defined in the Primitive Field Definitions for Key Assignment Type.

Number of items - The number of items in which keys are assigned. The format for this field is defined in the Primitive Field Definitions for Long Number.

CHOICE OF one Key Assignment Type:

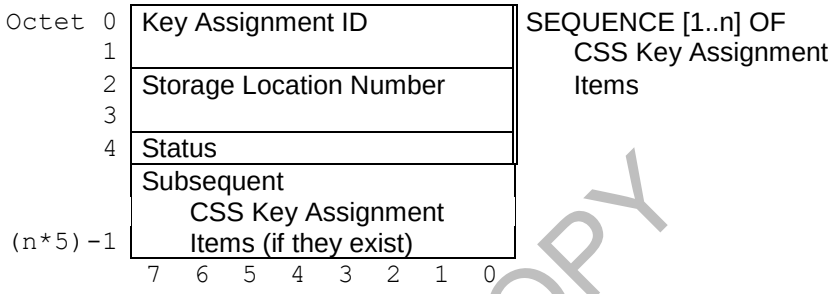
1. List/Assign SLNs to CSSs.
2. List/Assign SLNs to TGs.
3. List/Assign SLNs to LLIDs.

The Key Assignment Type Values are defined in the Primitive Field Definitions for Key Assignment Type.

10.2.18.1 List of SLNs to CSS Positions

When an SU receives a Key-Assignment-Command message containing SLN to CSS Position assignments, the SU responds by sending the List of SLNs to CSS Positions field in a Key-Assignment-Response message. The format for the List of SLNs to CSS Positions field is shown in Table 49.

Table 49 - List SLNs to CSSs Format



*SEQUENCE OF CSS Key Assignment Items - A li*st of Channel Selector Switch positions with their associated SLN.

Key Assignment ID - The 16-bit identifier for the Channel Selection Switch position to which the SLN is assigned. The format for this field is defined in the Primitive Field Definition for Key Assignment Identifier.

Storage Location Number - Identifies one of the 65536 possible storage locations where the key(s) is(are) stored. The format for this field is defined in the Primitive Field Definitions for Storage Location Number.

Status - The status of the key assignment. The format for this field is defined in the Primitive Field Definition for Status.

10.2.18.2 List of SLNs to Talk Groups

When an SU receives a Key-Assignment-Command message containing SLN to Talk Group assignments, the SU responds by sending the List of SLNs to Talk Groups field

in a Key-Assignment-Response message. The format for the List of SLNs to Talk Groups field is shown in Table 50.

Table 50 - List SLNs to TGs Format

Octet									
0	Key Assignment ID								SEQUENCE [1..n] OF TGID Key Assignment Items
1									
2	Storage Location Number								
3									
4	Status								
	Subsequent TGID Key Assignment Items (if they exist)								
(n*5)-1									
	7	6	5	4	3	2	1	0	

SEQUENCE OF TGID Key Assignment Items - A list of Talk Group IDs with their associated SLN.

Key Assignment ID - The 16-bit identifier for the Talk Group to which the SLN is assigned. The format for this field is defined in the Primitive Field Definition for Key Assignment Identifier.

Storage Location Number - Identifies one of the 65536 possible storage locations where the key(s) is(are) stored. The format for this field is defined in the Primitive Field Definitions for Storage Location Number.

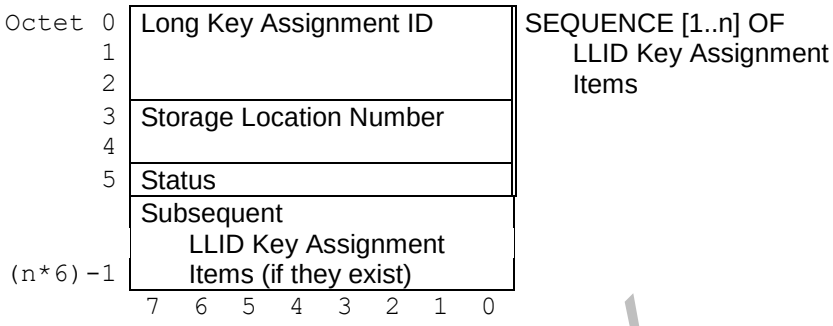
Status - The status of the key assignment. The format for this field is defined in the Primitive Field Definition for Status.

10.2.18.3 List of SLNs to Logical Link IDs

When an SU receives a Key-Assignment-Command message containing SLN to Logical Link ID assignments, the SU responds by sending the List of SLNs to Logical

Link IDs field in a Key-Assignment-Response message. The format for the List of SLNs to Logical Link IDs field is shown in Table 51.

Table 51 - Key Assignment Items for LLIDs Format



SEQUENCE OF LLID Key Assignment Items - A list of Logical Link IDs with their associated SLN.

Long Key Assignment ID - The 24-bit identifier for the Logical Link ID to which the SLN is assigned. The format for this field is defined in the Primitive Field Definition for Long Key Assignment Identifier.

Storage location Number - Identifies one of the 65536 possible storage locations where the key(s) is(are) stored. The format for this field is defined in the Primitive Field Definitions for Storage Location Number.

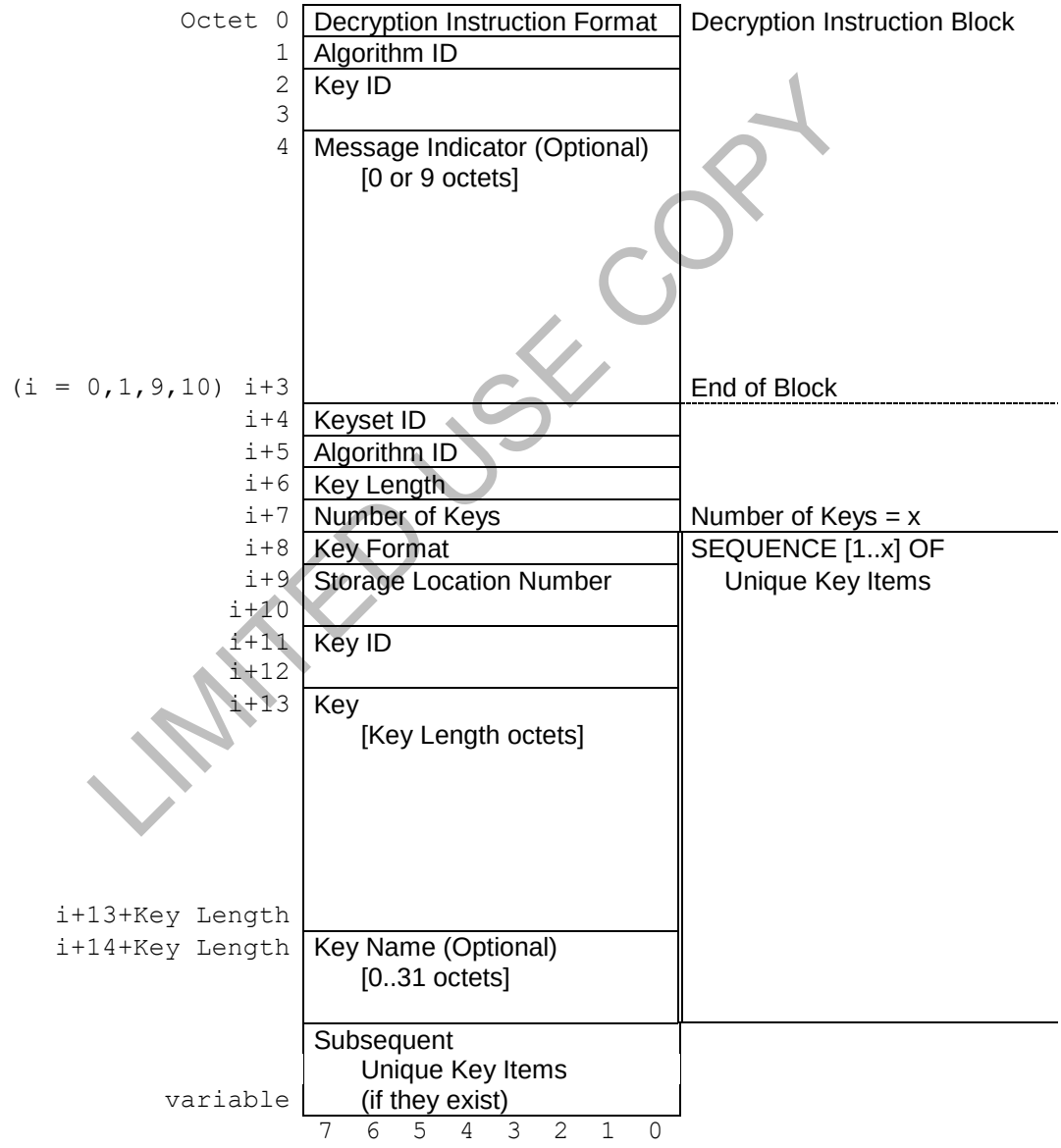
Status - The status of the key assignment. The format for this field is defined in the Primitive Field Definition for Status.

10.2.19 Modify-Key-Command

The Modify-Key-Command message is sent from the KMF to the SU and is used to modify one or more keys in a specified keyset. The message body format for the Modify-Key-Command message is shown in Table 52.

All keys transported in any single Modify-Key-Command KMM shall be of the same algorithm type.

Table 52 - Modify-Key-Command Message Format



Decryption Instruction Block

Decryption Instruction Format - Indicates which optional fields are included in the Decryption Instructions Block field. The format for this field is defined in the Primitive Field Definition for Decryption Instruction Format.

Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select a KEK. The format for this field is defined in the Primitive Field Definition for Algorithm ID.

Key ID - The Key ID of the KEK. The format for this field is defined in the Primitive Field Definition for Key ID.

Message Indicator (Optional) - Provides the Message Indicator (encryption synchronization) for encryption algorithms which do not support the Electronic Codebook mode of operation. The format for this field is defined in the Primitive Field Definition for Message Indicator.

Keyset ID - The Keyset ID of the keyset. The format for this field is defined in the Primitive Field Definition for Keyset ID.

Algorithm ID - The algorithm to be used with the keys in this keyset. The format for this field is defined in the Primitive Field Definitions for Algorithm ID.

Key Length - The length of the key in octets. The format for this field is defined in the Primitive Field Definitions for Number.

The number of octets used to transfer the key. The format for this field is defined in the Primitive Field Definitions for Number.

Number of Keys - The number of keys in this sequence. The format for this field is defined in the Primitive Field Definitions for Number.

SEQUENCE OF Unique Key Items - The key and associated tagging information. This consists of the Storage Location Number, the Key ID, the Key, and the Key Name (Optional).

Key Format - Indicates which optional fields are included in the Unique Key Item field. The format for this field is defined in the Primitive Field Definitions for Key Format.

Storage Location Number - Identifies one of the 65536 possible storage location indexes where the key is stored. The format for this field is defined in the Primitive Field Definitions for Storage Location Number

Key ID - The Key ID assigned to the key in the Key field. The format for this field is defined in the Primitive Field Definition for Key ID.

Key - The key, either a KEK or a TEK. The format of this field is defined in the Primitive Field Definition for Key.

Key Name (optional) - An identifier (name) which can be displayed to the user to indicate which key is currently selected. Bit 4 through bit 0 (LSB) of the Key Format octet define the length of this field. If the length is zero then this field is not present. The format for this field is defined in the Primitive Field Definitions for Key Name.

10.2.20 Modify-Keyset-Attributes-Command

The Modify-Keyset-Attributes-Command message is sent from the KMF to the SU and is used for changing the keyset name or the effective Date/Time for keys within a specified keyset. The message body format for the Modify-Keyset-Attributes-Command message is shown in Table 53.

Table 53 - Modify-Keyset-Attributes-Command Message Format

Octet 0	Number of Keysets	
1	Keyset Format	SEQUENCE [1..n] of
2	Keyset ID	Keyset Tagging Information
3	Reserved	Reserved
4	Reserved (Optional)	Reserved Block
5		(optional)
6		End of Block [0 or 3 octets]
4 or 7	Date	Date and Time Block
5 or 8		(optional)
6 or 9	Time	
7 or 10		
8 or 11		End of Block [0 or 5 octets]
4 or 9 or 12	Keyset Name (Optional) [i = 0..31 octets]	
3+i or 8+i or 11+i		
variable	Subsequent Keyset Tagging Information (if they exist)	
	7 6 5 4 3 2 1 0	

Number of Keysets - The number of keysets for which the attributes need to be changed. The format for this field is defined in the Primitive Field Definition for Number.

SEQUENCE OF Keyset Tagging Information - The keyset tagging information used to identify a key (or group of keys), to control the rekey of a key (or group of keys) or to automatically select a new key (or group of keys).

Keyset Format - Indicates which optional fields are included in the Keyset Tagging Information Block field and the length of the Keyset Name. The format for this field is defined in the Primitive Field Definitions for Keyset Format.

Keyset ID - The Keyset ID. The format for this field is defined in the Primitive Field Definitions for Keyset ID.

Reserved – This field is reserved and is defaulted to all zeros. Note that some legacy equipment may populate this field with a non-zero value and should therefore be ignored by non-legacy endpoints. This field previously specified the Algorithm ID.

Reserved Block (optional) - The Reserved Block is a field of 3 octets. This field is present if bit 6 of the Keyset Format octet is set to a 1.

Date and Time Block (optional) - The date (month, day and year) and time (hours, minutes and seconds) the keyset is to be activated. This field is present if bit 5 of the Keyset Format octet is set to a 1.

Date - The Date field is used to send the date. The format for this field is defined in the Primitive Field Definition for Date.

Time - The Time field is used to send the time. The format for this field is defined in the Primitive Field Definition for Time.

Keyset Name (optional) - An identifier (name) which can be displayed to the user to indicate which keyset is currently selected. Bit 4 through bit 0 (LSB) of the Keyset Format octet define the length of this field. If the length is zero then this field is not present. The format for this field is defined in the Primitive Field Definitions for Keyset Name.

10.2.21 Modify-Keyset-Attributes-Response

The Modify-Keyset-Attributes-Response message is sent from the SU to the KMF in response to a Modify-Keyset-Attributes-Command message. The message body format for the Modify-Keyset-Attributes-Response message is shown in Table 54.

Table 54 - Modify-Keyset-Attributes-Response Message Format

Octet	0	Number of Keysets	
	1	Keyset ID	SEQUENCE [1..n] OF Keyset Status
	2	Status	
	n*2	Subsequent Keyset Status (if they exist)	
		7 6 5 4 3 2 1 0	

Number of Keysets - The number of keysets for which the attributes were changed. The format for this field is defined in the Primitive Field Definition for Number.

SEQUENCE OF Keyset Status - The status of the modification of the keyset attributes is provided for each keyset identified in the associated Modify-Keyset-Attributes-Command message.

Keyset ID - The Keyset ID of the keyset. The format for this field is defined in the Primitive Field Definition for Keyset ID.

Status - The status of the keyset. The format for this field is defined in the Primitive Field Definition for Status.

10.2.22 Negative-Acknowledgment

The Negative-Acknowledgment message is sent from either the SU or the KMF.

A Negative-Acknowledgment message is used by the SU or KMF to respond to a Response Kind 3 (Immediate) message which the receiving end does not understand, has an invalid or unsupported message ID, an invalid message number, or an invalid MAC. The receiver ignores the contents of any message which fails with these errors.

When a Negative-Acknowledgment message is received, it is assumed that it corresponds to the most recent message sent. In response to receiving a negative acknowledgement, the transmitter has several options including re-sending the same message, sending a different message, or aborting the procedure all together.

A Negative-Acknowledgment message is a Response Kind 1 (None) message and may be sent in either the unconfirmed or confirmed data mode.

The Negative-Acknowledgment message contains the Message ID (if known), the Message Number of the last valid message received, and the error status of the message being negatively acknowledged.

In the case where the receiver is not able to decrypt the KMM, an Unable To Decrypt (UTD) message is returned instead of a Negative Acknowledgement message.

The Negative Acknowledgement message body format is shown in Table 55.

Table 55 - Negative Acknowledgment Message Format

Octet	7 6 5 4 3 2 1 0
0	Message ID
1	Message Number
2	
3	Status

Message ID - The Message ID that is being negatively acknowledged (if known). This field contains a zero if the SU or KMF is not able to determine the message type. The values for this field are defined in Section 11 - Message ID Assignments.

Message Number - The Message Number of the last valid received command. The format for this field is defined in Section 13.6 - Message Number Processing.

Status - The status of the negative acknowledgment. The format for this field is defined in the Primitive Field Definition for Status.

Table 56 provides a set of generic status codes and the reasons why they might be sent.

Table 56 - Generic Negative Acknowledgement Error Responses

Possible Responses	Status Octet Value	Circumstance
Negative Acknowledgement	$01 Command was not performed	- MFID is inconsistent with KMM Header fields or MAC (MSG ID or MAC ALGID) - Hello Command sent with invalid flag
	$02 Item does not exist	- MAC key not found
	$03 Invalid Message ID	- Message ID is invalid/unsupported
	$04 Invalid MAC	- MAC calculation failed - MAC key is a KEK
	$07 Invalid Message Number	- Message number out of range - Message Number in message but no MAC found

10.2.23 No-Service

The No-Service message is sent by the KMF to the SU when the KMF has no services to provide to the SU at the current time. The No Service message may be sent to the SU when the SU requests a rekey but the KMF deems that none are needed, when the SU requests a rekey but the KMF declines, or when a SU OTAR registers but the KMF declines the registration. The No-Service KMM may be used in non-error situations when, based on local policy, the KMF does not find it necessary to provide a Response such as if the SU is disabled from utilizing OTAR.

The No-Service message is a Response Kind 1 (None) message, is sent in the unconfirmed or confirmed data mode, and does not contain any information in the body of the message. This message shall be supported by the SU and optionally supported by the KMF.

10.2.24 Registration-Command

The Registration Command is sent from the SU to the KMF to indicate that the SU is on the system and available for OTAR service. The message body format for the Registration-Command message is shown in Table 57.

Table 57 - Registration Command Message Format

Octet		Body Message Format	Message Sub-header
	1	KMF RSI	
	2		
	3		End of Message Sub-header
	4	Decryption Instruction Format	Start of Reverse Warm Start & Decryption Instruction Block (optional)
	5	Algorithm ID	
	6	Key ID	
	7		
	8	Message Indicator (Optional) [0 or 9 octets]	
			End of Decryption Instruction Block
		Key Length	
		Algorithm ID	
		Key Format	Unique Key Item Block
		Storage Location Number	
		Key ID	
		Key [Key Length octets]	
		Key Name (Optional) [0..31 octets]	
variable			End of Key Item Block & Reverse Warm Start Segment
		7 6 5 4 3 2 1 0	

Message Sub-header Segment

Body Message Format – The Body Message Format field is defined in the Primitive Fields Definitions.

KMF RSI –This is the KMF RSI.

Reverse Warm Start Segment (optional) – This segment is only included when the Reverse Warm Start encrypted TEK is included in the command as indicated by the T bit in the Message Format octet being set (T=1).

Decryption Instruction Block

Decryption Instruction Format - Indicates which optional fields are included in the Decryption Instructions Block field. The format for this field is defined in the Primitive Field Definition for Decryption Instruction Format.

Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select a key. The format for this field is defined in the Primitive Field Definition for Algorithm ID.

Key ID - The Key ID of the key. The format for this field is defined in the Primitive Field Definition for Key ID.

Reserved (Optional) - A reserved field for an optional octet.

Message Indicator (Optional) - Provides the Message Indicator (encryption synchronization) for encryption algorithms which do not support the Electronic Codebook mode of operation. The format for this field is defined in the Primitive Field Definition for Message Indicator.

Key Length - The number of octets used to transfer the key. The format for this field is defined in the Primitive Field Definitions for Number.

Algorithm ID - The algorithm to be used with the key in this message. The format for this field is defined in the Primitive Field Definitions for Algorithm ID.

Unique Key Item Block - The key and associated tagging information. This consists of the Storage Location Number, the Key ID, the Key, and the Key Name (Optional).

Key Format - Indicates which optional fields are included in the Unique Key Item Block field. The format for this field is defined in the Primitive Field Definitions for Key Format.

Storage Location Number - This field is invalid for the Registration command and shall default to all zeros. The key being transferred in the Registration command is a temporary key and shall be deleted after completing the registration procedure.

Key ID - The Key ID assigned to the key in the Key field. The format for this field is defined in the Primitive Field Definition for Key ID.

Key - The Traffic Encryption Key. The format of this field is defined in the Primitive Field Definition for Key.

Key Name (Optional) - This field is invalid for the Registration command and shall default to all zeros.

10.2.25 Registration-Response

The Registration-Response message is sent from the KMF to the SU in response to the Registration-Command message and is used to indicate the success or failure of the registration request.

The message body format of the Registration-Response message is shown in Table 58.

Table 58 - Registration-Response Message Format

Octet 0	Status
	7 6 5 4 3 2 1 0

Status – Indicates whether the registration request was successful or not. The format for this field is defined in the Primitive Field Definition for Status.

10.2.26 Rekey-Acknowledgment

The Rekey-Acknowledgment message is sent from the SU to the KMF in response to a Modify-Key-Command, Rekey-Command, or Warm-Start-Command.

The Rekey-Acknowledgment message is a Response Kind 1 (None) message. When the KMF requests a Response Kind 3 (Immediate) response to a rekey instruction, the SU responds with an immediate Rekey-Acknowledgment. A Response Type 3 rekey message is typically used when a KMF targets an individual SU rather than a group of SUs.

The Rekey-Acknowledgment message contains a status for each key transferred in the original rekey instruction. The SU verifies the MAC of the rekey instruction before responding with the Rekey-Acknowledgment message.

Note that if an ALGID/Key ID pair appears more than once in a given Modify Key or Rekey message, the order of the key status entries contained in the Rekey-Acknowledgement Key Status sequence shall be the same as the order of Unique Key Items presented in the original Modify Key or Rekey KMM.

The message body format of the Rekey-Acknowledgement message is shown in Table 59.

Table 59 - Rekey-Acknowledgment Message Format

Octet		
0	Message ID	
1	Number of Key Status	
2	Algorithm ID	SEQUENCE [1..n] of Key Status
3	Key ID	
4		
5	Status	
n*5	Subsequent Key Status (if they exist)	
	7 6 5 4 3 2 1 0	

Message ID - The Message ID that is being acknowledged. The values for this field are defined in Section 11 - Message ID Assignments.

Number of Key Status - The number of keys that were requested to be added or modified. The format for this field is defined in the Primitive Field Definition for Number.

SEQUENCE OF Key Status - The list of the Unique Key ID and Status for each key that was added, modified, or deleted.

Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select a key. The format for this field is defined in the Primitive Field Definition for Algorithm ID.

Key ID - The Key ID of the key. The format for this field is defined in the Primitive Field Definition for Key ID.

Status - The status of the key. The format for this field is defined in the Primitive Field Definition for Status.

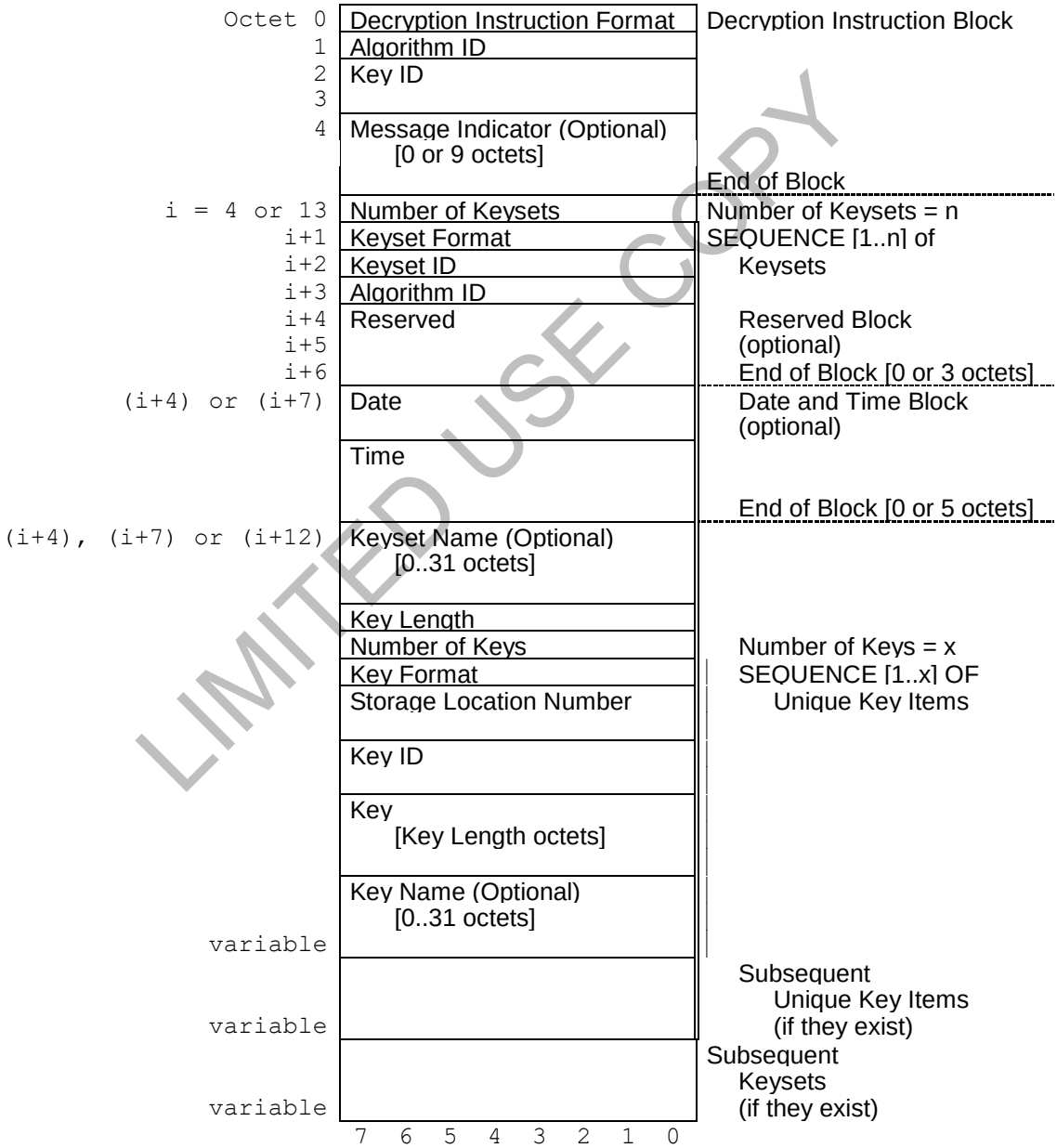
Note that a Rekey-Acknowledgement is used only in response to a Response Kind 3 (Immediate) rekey instruction (Modify-Key, Rekey, or Warm-Start Command). A Delayed Acknowledgement response is used if the rekey instruction is a Response Kind 2 (Delayed) (reference Section 10.2.7).

10.2.27 Rekey-Command

The Rekey-Command message is sent from the KMF to the SU and is used to add or modify key(s) in a keyset, to modify a keyset, or to optionally add a keyset. The message body format for the Rekey-Command message is shown in Table 60.

All keys transported in any single Rekey-Command KMM shall be of the same algorithm type.

Table 60 - Rekey-Command Message Format



Decryption Instruction Block

Decryption Instruction Format - Indicates which optional fields are included in the Decryption Instructions Block field. The format for this field is defined in the Primitive Field Definition for Decryption Instruction Format.

Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select a KEK. The format for this field is defined in the Primitive Field Definition for Algorithm ID.

Key ID - The Key ID of the KEK. The format for this field is defined in the Primitive Field Definition for Key ID.

Message Indicator (Optional) - Provides the Message Indicator (encryption synchronization) for encryption algorithms which do not support the Electronic Codebook mode of operation. The format for this field is defined in the Primitive Field Definition for Message Indicator.

Number of Keysets - The number of keysets in the rekey message. The format for this field is defined in the Primitive Field Definition for Number.

SEQUENCE OF Keysets - This block contains information pertaining to each keyset.

Keyset Tagging Information Block - The keyset tagging information used to identify a key (or group of keys) or to automatically select a new key (or group of keys).

Keyset Format - Indicates which optional fields are included in the Keyset Tagging Information Block field and the length of the Keyset Name. The format for this field is defined in the Primitive Field Definitions for Keyset Format.

Keyset ID - The Keyset ID. The format for this field is defined in the Primitive Field Definitions for Keyset ID.

Algorithm ID - The algorithm to be used is in reference to all the keys delivered within the keyset block. The format for this field is defined in the Primitive Field Definitions for Algorithm ID.

Reserved Block (optional) - A reserved field for 3 octets.

Date and Time (optional) - The date (month, day and year) and time (hours, minutes and seconds) the keyset is to be activated. This field is present if bit 5 of the Keyset Format octet is set to a 1.

Date - The Date field is used to send the date. The format for this field is defined in the Primitive Field Definition for Date.

Time - The Time field is used to send the time. The format for this field is defined in the Primitive Field Definition for Time.

Keyset Name (optional) - An identifier (name) which can be displayed to the user to indicate which keyset is currently selected. Bit 4 through bit 0 (LSB) of the Keyset Format octet define the length of this field. If the length is zero then this field is not present. The format for this field is defined in the Primitive Field Definitions for Keyset Name.

Key Length - The number of octets used to transfer the key. The format for this field is defined in the Primitive Field Definitions for Number.

Number of Keys - The number of keys in this sequence. The format for this field is defined in the Primitive Field Definitions for Number.

SEQUENCE OF Unique Key Items - The key and associated tagging information. This consists of the Storage Location Number, the Key ID, the Key, and the Key Name (Optional).

Key Format - Indicates the type of key, key length, etc. included in the Unique Key Item Block field. The format for this field is defined in the Primitive Field Definitions for Key Format.

Storage Location Number - Identifies one of the 65536 possible storage location indexes where the key is stored. The format for this field is defined in the Primitive Field Definitions for Storage Location Number

Key ID - The Key ID assigned to the key in the Key field. The format for this field is defined in the Primitive Field Definition for Key ID.

Key - The key, either a KEK or a TEK. The format of this field is defined in the Primitive Field Definition for Key.

Key Name (optional) - An identifier (name) which can be displayed to the user to indicate which key is currently selected. Bit 4 through bit 0 (LSB) of the Key Format octet define the length of this field. If the length is zero then this field is not present. The format for this field is defined in the Primitive Field Definitions for Key Name.

Note that the modification or deletion of an existing key is identified by reusing the Storage Location Number in a keyset. Modification to an existing keyset is identified by reusing the keyset ID.

10.2.28 Set-Date-Time-Command

The Set-Date-Time message is sent from the KMF to the SU and is used to update the date and time of the SU. The message body format for the Set-Date-Time-Command message is shown in Table 61.

Table 61 - Set-Date-Time Message Format

Octet	7 6 5 4 3 2 1 0
0	Date
1	
2	Time
3	
4	

Date - The current date is used by the SU to synchronize the SU date with the KMF date. The format for this field is defined in the Primitive Field Definition for Date.

Time - The current time is used by the SU to synchronize the SU time with the KMF time. The format for this field is defined in the Primitive Field Definition for Time.

10.2.29 Unable-to-Decrypt-Response

The message body format of the Unable-to-Decrypt-Response message is shown in Table 62.

Table 62 - Unable-to-Decrypt-Response Message Format

Octet									
0	Body Message Format								Message Sub-header
1	MFID								From ESYNC of original msg
2	Algorithm ID								From ESYNC of original msg
3	Key ID								From ESYNC of original msg
4									
5	Status								End of Message Sub-header
6	Decryption Instruction Format								Start of Reverse Warm Start & Decryption Instruction Block (optional)
7	Algorithm ID								
8	Key ID								
9									
10	Message Indicator (Optional) [0 or 9 octets]								End of Decryption Instruction Block
	Key Length								
	Algorithm ID								
	Key Format								Unique Key Item Block
	Storage Location Number								
	Key ID								
	Key [Key Length octets]								
	Key Name (Optional) [0..31 octets]								
variable									End of Key Item Block & Reverse Warm Start Segment
	7	6	5	4	3	2	1	0	

Message Sub-header Segment

Body Message Format – The Body Message Format field is defined in the Primitive Fields Definitions.

MFID - The MFID used in the original message which could not be decrypted.

Algorithm ID - The Algorithm ID used in the original message which could not be decrypted.

Key ID - The Key ID used in the original message which could not be decrypted.

Status – Indicates reason why the message was unable to be decrypted. The format for this field is defined in the Primitive Field Definition for Status.

Reverse Warm Start Segment (optional) – This segment is only included when the Reverse Warm Start encrypted TEK is included in the command as indicated by the T bit in the Body Message Format octet being set (T=1).

Decryption Instruction Block

Decryption Instruction Format - Indicates which optional fields are included in the Decryption Instructions Block field. The format for this field is defined in the Primitive Field Definition for Decryption Instruction Format.

Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select a key. The format for this field is defined in the Primitive Field Definition for Algorithm ID.

Key ID - The Key ID of the key. The format for this field is defined in the Primitive Field Definition for Key ID.

Message Indicator (Optional) - Provides the Message Indicator (encryption synchronization) for encryption algorithms which do not support the Electronic Codebook mode of operation. The format for this field is defined in the Primitive Field Definitions for Message Indicator.

Key Length - The number of octets used to transfer the key. The format for this field is defined in the Primitive Field Definitions for Number.

Algorithm ID - The algorithm to be used with the key in this message. The format for this field is defined in the Primitive Field Definitions for Algorithm ID.

Unique Key Item Block - The key and associated tagging information. This consists of the Storage Location Number, the Key ID, the Key, and the Key Name (Optional).

Key Format - Indicates which optional fields are included in the Unique Key Item Block field. The format for this field is defined in the Primitive Field Definitions for Key Format.

Storage Location Number - This field is not used for the Unable to Decrypt Response and shall default to all zeros. The key being transferred in the Unable to Decrypt Response is a temporary key and shall be deleted after sending the message.

Key ID - The Key ID assigned to the key in the Key field. The format for this field is defined in the Primitive Field Definition for Key ID.

Key - The Traffic Encryption Key. The format of this field is defined in the Primitive Field Definition for Key.

Key Name (Optional) - This field is invalid for the Unable to Decrypt Response and shall default to all zeros.

10.2.30 Warm-Start-Command

The Warm-Start-Command message is sent from the KMF to the SU and is used to provide TEKs to the SU when the SU contains a valid KEK but no valid TEKs. This message is sent unencrypted (i.e., SAP identifier = $28) with the exception of the Key field, which is encrypted with the KEK. The message body format for the Warm-Start-Command is shown in Table 63.

Table 63 - Warm-Start-Command Message Format

Octet		
0	Decryption Instruction Format	Decryption Instruction Block
1	Algorithm ID	
2 3	Key ID	
4	Message Indicator (Optional) [0 or 9 octets]	End of Block
4 or 13	Key Length	
	Algorithm ID	
	Key Format	Unique Key Item Block
	Storage Location Number	
	Key ID	
	Key [Key Length octets]	
	Key Name (Optional) [0..31 octets]	
variable		End of Block
	7 6 5 4 3 2 1 0	

Decryption Instruction Block

Decryption Instruction Format - Indicates which optional fields are included in the Decryption Instructions Block field. The format for this field is defined in the Primitive Field Definition for Decryption Instruction Format.

Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select the key used to encrypt the Warm-Start key. The format for this field is defined in the Primitive Field Definition for Algorithm ID.

Key ID - The Key ID of the key. The format for this field is defined in the Primitive Field Definition for Key ID.

Message Indicator (Optional) - Provides the Message Indicator (encryption synchronization) for encryption algorithms which do not support the Electronic Codebook mode of operation. The format for this field is defined in the Primitive Field Definition for Message Indicator.

Key Length - The number of octets used to transfer the key. The format for this field is defined in the Primitive Field Definitions for Number.

Algorithm ID - The algorithm type of the Warm-Start key. The format for this field is defined in the Primitive Field Definitions for Algorithm ID.

Unique Key Item Block - The key and associated tagging information. This block consists of the following fields:

Key Format - Indicates which optional fields are included in the Unique Key Item Block field. The format for this field is defined in the Primitive Field Definitions for Key Format.

Storage Location Number – The information contained in this field is not used for the Warm-Start-Command and shall default to all zeros.

Key ID - The Key ID assigned to the key in the Key field. The format for this field is defined in the Primitive Field Definition for Key ID.

Key - The (Warm-Start) Traffic Encryption Key. The format of this field is defined in the Primitive Field Definition for Key.

Key Name (optional) - This field is invalid for the Warm-Start-Command and shall default to all zeros.

10.2.31 Zeroize-Command

The Zeroize-Command message is sent from the KMF to the SU and is used to remotely zeroize all of the critical security parameters (keys and keysets) in the SU. There is no information contained in the message body of the Zeroize-Command message.

The Zeroize-Command deletes all keys including KEKs and any temporary traffic keys, excluding subscriber authentication (Link Layer Authentication) keys. All parameters within a Keyset are removed including Algorithm ID, Key ID, the keys, and optional key name fields. The existence of the keyset is also removed from the Inventory-Response message if the SU supports the dynamic creation of keysets. If the keysets are statically defined, then the existent of the keysets are reported by the Inventory-Response message to list the keysets in storage. The manufacturer's security policy determines which other security parameters (such as MNs and RSIs) are deleted.

10.2.32 Zeroize-Response

The Zeroize-Response message is sent from the SU to the KMF in response to a Zeroize-Command. There is no information contained in the message body of the Zeroize-Response message.

10.3 Primitive Field Definitions

This section contains the OTAR message primitive field definitions.

10.3.1 Algorithm ID

The Algorithm ID identifies the algorithm used to encrypt the key(s), data, voice, or identifies the type of key sent in the OTAR message. Standardized encryption algorithms are specified in [5]. Algorithm ID values that are not listed in [5] indicate non-standardized or proprietary encryption algorithms.

10.3.2 Body Message Format

The Body Message Format octet defines the message structure following this field. The Body Message Format octet defines which optional fields exist and the status of a KEK in the SU. The Body Message Format octet is defined in Table 64.

Table 64 - Body Message Format Bit Definitions

	7	6	5	4	3	2	1	0
Octet 0	T	K	0	0	0	0	0	0

T (TEK Included) - Indicates whether this message contains a Reverse Warm Start segment (an encrypted TEK) or not. A value of 0 indicates that the message only contains the Message Sub-header. A value of 1 indicates the Reverse Warm Start segment exists. The message shall be sent clear if the T bit is set to a 1.

K (KEK Exists) – Indicates to the KMF whether the SU does or does not have a KEK. A value of 0 indicates a KEK exists. A value of 1 indicates a KEK does not exist. The KMF can determine if it should attempt a rekey of the SU based on the *K* indicator. The “New Unit” Procedure (Section 9.1) is required to re-establish rekey services when the SU is not in possession of a UKEK common to both the SU and KMF.

Spare – Bit 5 (b5) through bit 0 (b0) are not used and are set to zero.

10.3.3 Date

The Date field consists of 2 octets. Octet 0 bit 7 is the Most Significant Bit and Octet 1 bit 0 is the Least Significant (LS) Bit. The date is represented by two octets as shown in Table 65.

Table 65 - Date Bit Assignments

	7	6	5	4	3	2	1	0	
Octet 0	Month				Day				
1	*	Year							* LS bit of the Day field

Month - This field represents the month of the year in the date field. Octet 0 bit 7 (b7) is the MSB of the month and Octet 0 bit 4 (b4) is the LSB of the month field. Valid values are 1 (0001 in binary) to 12 (1100 in binary). January is binary 0001, February is binary 0010, etc, and December is binary 1100.

Day - This field represents the day in the month in the date field. This field is the 4 LS bits (b3 - b0) of octet 0 and the MSB (b7) of octet 1. The MSB of the day field is Octet 0 bit 3 and the LSB is Octet 1 bit 7. Valid values are 1 (00001 in binary) to 31 (11111 in binary).

Year - This field represents the last two digits of the year. Octet 1 bit 6 (b6) is the MSB of the year and Octet 0 bit 0 (b0) is the LSB of the year field. Valid values are 0 (00 0000 in binary) to 99 (11 0011 in binary).

For example, December 30, 1992 would be represented by 11001111 01011100 in binary (i.e. Month = 1100, Day = 11110, and Year = 1011100).

10.3.4 Decryption Instruction

The Decryption Instruction octet defines the message structure that follows this field in the Decryption Instruction Block. The Decryption Instruction octet defines which optional fields exist and the size of these fields. The Decryption Instruction octet is defined in Table 66.

Table 66 - Decryption Instruction Bit Definitions

	7	6	5	4	3	2	1	0
Octet 0	R	M	0	0	0	0	0	0

R (Reserved) - The Reserved bit (b7) is not used and shall default to a value of zero.

M (Message Indicator) - The Message Indicator bit defines if the 9 octets of the Message Indicator (encryption sync) are included in the Decryption Instruction block. The Message Indicator is optional since it is not required for some modes of encryption. A value of 0 indicates that the Message Indicator field is not present and a value of 1 indicates that the Message Indicator field is present.

Spare - Bit 5 (b5) through bit 0 (b0) are not used and are set to zero.

10.3.5 Hello Flag

The Hello Flag is a component of the Hello message and is used by the KMF and SU to provide simple identification, or is used by the SU to request rekey services. The Hello Flag field is one octet in size and the format for this field is defined in Table 67.

Table 67 - Hello Flag Values

Hello Flag Definitions	Value (Hex)	Value (Binary)
KMF or SU Identification Only	$00	0000 0000
Rekey Request (UKEK exists)	$01	0000 0001
Rekey Request (UKEK does not exist)	$02	0000 0010
Reserved for Future Use	$03 - $FF	0000 0011 - 1111 1111

Note - The “New Unit” Procedure (Section 9.1) is required to re-establish rekey services when the SU is not in possession of a UKEK common to both the SU and KMF.

10.3.6 Inventory Type

The Inventory Type field identifies the type of information being requested by the KMF and is present in both the Inventory-Command and Inventory-Response messages. This field is one octet in length with bit 0 as the LSB, and is defined in Table 68.

Table 68 - Inventory Type Values

Inventory Requested	Value (Hex)	Value (Binary b7-b0)
Null	$00	0000 0000
Send Current Date/Time	$01	0000 0001
List Active Keyset IDs	$02	0000 0010
List Inactive Keyset IDs	$03	0000 0011
List Active Key IDs	$04	0000 0100
List Inactive Key IDs	$05	0000 0101
List all Keyset Tagging Information	$06	0000 0110
List all Unique Key Information	$07	0000 0111
List Key Assignment Items for CSSs	$08	0000 1000
List Key Assignment Items for TG	$09	0000 1001
List Long Key Assignment Items for LLIDs	$0A	0000 1010
List RSI Items	$0B	0000 1011
Reserved for Future Use	$0C - $FF	0000 1100 to 1111 1111

10.3.7 Key

The Key field contains the encrypted or wrapped key (either a TEK or a KEK) as defined in Section 13.4.

10.3.8 Key Assignment ID

The Key Assignment ID is a 2 octet field which contains a value in the range between 0 and 65535 and is used to specify the Channel Selector Switch position or Talk Group ID.

10.3.9 Key Assignment Type

The Key Assignment Type is a one octet parameter used to associate or list SLN to CSS, TG or LLID associations. The Key Assignment Type field is included in the Key-Assignment-Command and Key-Assignment-Response messages.

The values of the Key Assignment Type field are defined in Table 69.

Table 69 - Key Assignment Type Values

Key Assignment Type	Value (Hex)	Value (Binary b7-b0)
Null	$00	0000 0000
List/Assign SLN to CSSs	$01	0000 0001
List/Assign SLN to TGs	$02	0000 0010
List/Assign SLN to LLIDs	$03	0000 0011
Reserved for Future Use	$04 - $FF	0000 0100 to 1111 1111

10.3.10 Key ID

The Key ID is a 2 octet field which contains a value from 0 to 65535 and is used to specify either a KEK or a TEK for both single-key or multi-key equipment. See normative Annex B for historical standard text associated with the single-key and multi-key usage of the Key ID field prior to the publication of ANSI/TIA-102.AACA-A-1.

An unencrypted transmission is identified by an ALGID value of $80. Encrypted transmissions use an ALGID value other than $80. Encrypted transmissions use the Key ID associated with the key selected by the transmitting device. The Key ID range of 0 to 65535 is valid for transmissions from either single-key or multi-key equipment. This Key ID value is used by recipients of the encrypted transmission to identify the key variable to be used in decryption. Device configuration associates a key variable with a Key ID. In order for the encryption/decryption process between devices to be successful, all participating devices must associate the same key variable with the same Key ID.

When working with mixed single-key and multi-key equipment, the Key ID shall indicate the specific key to be used on the channel and must match the Key ID stored in the single-key and multi-key equipment for protected communications to take place. KEY IDs for KEKs in support of multiple algorithms require special consideration. KEKs are stored in Crypto Group 15 and Keyset IDs 241-255. The KEY ID values $F5A0 and $F5A1 are reserved and may be required by legacy equipment for the UKEK Key ID and CKEK Key ID respectively. Any valid Key ID may be used for a KEK.

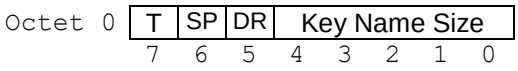
10.3.11 Keyset ID

The Keyset ID is an octet field which contains a value in the range between 1 and 255 used to identify a particular keyset. For example, Keyset ID 1 is represented by $01 and Keyset ID 255 is represented by $FF. A Keyset ID value of $00 is reserved.

10.3.12 Key Format

The Key Format field defines the key type and which message structure follows this field in the sequence or message field. The Key Format defines which optional fields exist and the size of these fields. The Key Format field is shown in Table 70.

Table 70 - Key Format Bit Definitions



T (Key Type) - The T (Key Type) bit (b7) defines if the key is a Traffic Encryption Key (TEK) or a Key Encryption Key (KEK). A value of 0 indicates a TEK and a value of 1 indicates a KEK.

SP (Spare) - This bit (b6) is not used and is always set to zero (0).

D/R (Delete/Rekey bar) - The Delete/Rekey bar (b5) defines if the key identified by the Key Format octet is to be deleted or is to be rekeyed (added or changed). A value of 0 indicates that the key is to be added or changed. A value of 1 indicates that the key is to be deleted (if it exists). When the delete key bit is set, the SLN and keyset fields are used to select the key to be deleted. The other fields (ALGID, Key ID and Key) are ignored.

Key Name Size - The Key Name Size defines the length (from 0 to 31 octets) of the key name associated with the key included in the Unique Key Item block. The Key Name provides an identifier (name) which can be displayed to the user to indicate which key is currently selected. Bit 4 (b4) is the MSB and bit 0 (b0) is the LSB. If the length is zero then the Key Name field is not present.

10.3.13 Key Name

The Key Name field provides an identifier (name) for a key which can be displayed to the user to indicate which key is currently selected. The Key Name contains up to 31 characters and is not null-terminated. Bit 4 (MSB) through bit 0 (LSB) of the Key Format field (Section 10.3.12) defines the length of this field. If the length is zero then this field is not present. Each octet defines one ascii character. The first octet contains the first ascii character to be displayed, the second octet contains the second ascii character to be displayed, etc.

10.3.14 Keyset Format

The Keyset Format field defines which message structure follows this field in the Keyset Tagging Information block. The Keyset Format octet identifies the optional fields and the size of these fields. The Keyset Format field is shown in Table 71.

Table 71 - Keyset Format Bit Definitions

	7	6	5	4	3	2	1	0
Octet 0	T	R	DT	Keyset Name Size				

T (Keyset Type) - The T (Keyset Type) bit (b7) indicates whether the keyset contains Traffic Encryption Keys (TEKs) or Key Encryption Keys (KEKs). A value of 0 indicates a keyset contains TEKs and a value of 1 indicates the keyset contains KEKs.

R (Reserved) - The R (Reserved) bit (b6) indicates whether or not the 3 octet reserved field is included in the keyset block. A value of 0 indicates that the 3 octet Reserved block is not present and a value of 1 indicates that the 3 octet Reserved block is present.

DT (Date and Time) - The DT (Date and Time) bit (b5) defines whether or not the 5 octets Date and Time block is included in the keyset block. A value of 0 indicates that the 5 octet Date and Time block is not present and a value of 1 indicates that the 5 octet Date and Time block is present. The Date and Time indicates when the keyset is to be activated.

Keyset Name Size - The Keyset Name Size defines the length (from 0 to 31 octets) of the keyset name associated with the keyset. The Keyset Name provides an identifier (name) which can be displayed to the user to indicate which keyset is currently selected. Bit 4 (b4) is the MS bit and bit 0 (b0) is the LS bit. If the length is zero then the Keyset Name field is not present.

10.3.15 Keyset Name

The Keyset Name field provides an identifier (name) for a keyset which can be displayed to the user to indicate which keyset is currently selected. The Keyset Name contains up to 31 ascii characters and is not null-terminated. Bit 4 (MSB) through bit 0 (LSB) of the Keyset Format field define the length of the Keyset Name field. If the length is zero then this field is not present. Each octet defines one ascii character. The first octet contains the first ascii character to be displayed, the second octet contains the second ascii character to be displayed, etc.

10.3.16 Long Key Assignment ID

The Long Key Assignment ID is a 3 octet field which contains a hex value in the range between 0 and 16,777,215 and is used to specify the Logical Link ID.

10.3.17 Long Number

The Long Number field is a two octet field which contains a hex value in the range between 0 and 65,535. A Long Number of zero is represented by $0000 and a Long Number of 65,535 is represented by $FFFF.

10.3.18 Message Format

The Message Format field defines the message structure which follows this field in the KMM. The Message Format field is one octet in length and is divided into three components (Response Kind, Message Number, and MAC). It indicates whether or not the optional fields exist and the size and use of the fields.

The Response Kind parameter defines if the acknowledgment is to be returned to the sender of the KMM. The Message Number parameter defines whether the MN field is present in the KMM. The MAC parameter defines the type of MAC processing performed over the entire KMM. The Message Format field is shown in Table 72.

Table 72 - Message Format Bit Definitions

Octet 0	RK	MN	MAC	0	D
	7 6	5 4	3 2	1	0

RK (Response Kind) - The RK (Response Kind) bits (b7, b6) defines the type of acknowledgment that is to be returned to the sender of the KMM. A value of 00 requests a response of Response Kind 1 (None), a value of 01 requests a response of Response Kind 2 (Delayed), and a value of 10 requests a response of Response Kind 3 (Immediate). A value of 11 is reserved for future use.

MN (Message Number) - The MN (Message Number) bits (b5, b4) defines the presence of the message number. A value of 00 indicates that there is no message number present in the message and a value of 10 indicates a two octet message number.

MAC (Message Authentication Code) - The MAC (Message Authentication Code) bits (b3, b2) defines the authentication type used to validate the message.
00 – indicates that there is no authentication;
01 – Reserved;
10 – Indicates that the Enhanced algorithm based Message Authentication Code is used to validate the message;
11 – Indicates that the DES[7] algorithm based Message Authentication Code is used to validate the message.

Spare - Bit 1 (b1) is not used and is set to zero.

D (Done) – This optional bit (b0) is used to indicate that the current KMM is the last in a series of KMMs. A value of "1" indicates DONE (end of sequence) and a "0" indicates NOT DONE. The default value for this bit shall be "0" and when not used this bit shall be "0".

10.3.19 Message Indicator

The Message Indicator is optional and is used to provide encryption synchronization for some modes of encryption. This field consists of 9 octets. Section 12.3.2 provides further information on the Message Indicator for algorithms which require this field.

[7] DES is only utilized for backward compatibility, see NOTE in section 1.1.

10.3.20 Message Length

The Message Length is a 2 octet field which contains a value in the range between 7 and 65535 and indicates the length in octets of all the fields following the Message ID and Message Length, including the MAC (if present). A Message Length of seven is represented by $0007 and a Message Length of 65,535 is represented by $FFFF.

When sending the KMM over the air, the actual message will be 3 octets longer than indicated by the Message Length field since the Message Length excludes the Message ID field (1 octet) and Message Length field itself (2 octets).

The Message Length is also used for calculation of the derived keys described in Section 13.5.2.2.

10.3.21 Number

The Number field is an octet value in the range between 0 and 255. A Number of zero is represented by $00 and a Number of 255 is represented by $FF.

10.3.22 Optional Service ID

The Optional Service ID field defines the optional services that the SU can support. The Optional Service ID values are shown in Table 73.

Table 73 - Optional Service ID Values

Optional Service ID	Value (Hex)
Reserved	$00
Time of Day* is supported	$01
Reserved	$02
Reserved	$03
Reserved	$04
Reserved	$05
Reserved	$06
Reserved	$07
Reserved	$08
Reserved	$09
Enhanced Algorithm MAC supported (CBC-MAC)	$0A
DES[8] Algorithm MAC is supported	$0B
Key Name is supported	$0C
Keyset Name is supported	$0D
Radio supports Inventory and Key Assignment extended Service IDs	$0E
Inventory “Null”	$0F
Inventory “Send Current Date/Time”	$10
Inventory “List Active Keyset IDs”	$11
Inventory “List Inactive Keyset IDs”	$12
Inventory “List Active Key IDs”	$13
Inventory “List Inactive Key IDs”	$14
Inventory “List all Keyset Tagging Information”	$15
Inventory “List all Unique Key Information”	$16
Inventory “List Key Assignment Items for CSSs”	$17
Inventory “List Key Assignment Items for TG”	$18
Inventory “List Key Assignment Items for LLIDs”	$19
Inventory “List RSI Items”	$1A
Key Assignment “Null”	$1B
Key Assignment “List/Assign SLN to CSSs”	$1C
Key Assignment “List/Assign SLN to TGs”	$1D
Key Assignment “List/Assign SLN to LLIDs”	$1E
Enhanced Algorithm MAC supported (CMAC)	$1F
Reserved for Future Use	$20 - $7F
Reserved for Manufacturer	$80 - $FF

Whenever one or more Optional Service IDs in the range of $0F-$1E are returned in a Capabilities-Response message, the “Radio supports Inventory and Key Assignment extended Service IDs” Optional Service ID ($0E) shall also be returned.

[8] DES is only utilized for backward compatibility, see NOTE in section 1.1.

* The Time of Day service means that the SU supports changeovers based on the Time of Day field in the Keyset Tagging Information block. If the unit is capable of supporting Time of Day, then automatic updating of the key occurs based on the time of day.

10.3.23 RSI

The RSI field is three octets in length and has the same definition as the Source and Destination ID specified in [5]. For a source RSI, this field indicates the identity of the sender. For a destination RSI, this field identifies a specific SU or group of SUs for which the message is destined. There is one standardized value that is used to signify 'no one', i.e., no SU in a system is assigned to this reserved value. There are also standardized values to signify group addresses. These group addresses may be used as destination addresses for KMMs, but never as source addresses. The standardized RSI value ranges are provided in Table 74.

Table 74 - RSI Values

RSI	Value (Decimal)	Value (Hex)
Reserved "No one"	0	$000000
Individual Address	1 - 9,999,998	$000001 - $98967E
Default KMF RSI	9,999,999	$98967F
Group Addresses	10,000,000 - 16,777,214	$989680 - $FFFFFE
Designates Everyone	16,777,215	$FFFFFF

10.3.24 Status

The Status field is a one octet parameter that provides the status of a requested operation or procedure. The values of the Status field are provided in Table 75.

Table 75 - Status Field Values

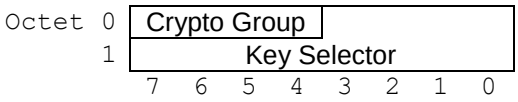
Value (Hex)	Status	Reason
$00	Command was performed	Command was executed successfully
$01	Command not performed	Command could not be performed due to an unspecified reason
$02	Item does not exist	Key / Keyset needed to perform the operation does not exist
$03	Invalid Message ID	Message ID is invalid/unsupported
$04	Invalid MAC	MAC is invalid
$05	Out of Memory	Memory unavailable to process the command / message
$06	Could not decrypt the message	KEK does not exist
$07	Invalid Message Number	Message Number is invalid
$08	Invalid Key ID	Key ID is invalid or not present
$09	Invalid Algorithm ID	ALGID is invalid or not present
$0A	Invalid MFID	MFID is invalid
$0B	Module Failure	Encryption Module failure
$0C	MI all zeros	Received MI was all zeros
$0D	Keyfail	Key identified by ALGID/Key ID is erased
$0E - $FE	Reserved for Future Use	Reserved
$FF	Unknown	Unknown

10.3.25 Storage Location Number

The Storage Location Number (SLN) field is a parameter used as a pointer to a key location within a target device. The SLN is a 2 octet field in the range between 0 and 65,535, split into two fields. The 4 most significant bits of the SLN field are used to define the Crypto Group and the 12 least significant bits are use to select the key in the keyset.

The SLN field is shown in Table 76.

Table 76 - SLN Bit Definitions



Crypto Group - The Crypto Group defines one of the 16 possible Crypto Groups. A value of $0 selects Crypto Group 0 and a value of $F selects Crypto Group 15.

Key Selector - The key selector field selects one of the 4096 possible keys in each Crypto Group. A value of zero is represented by $000 and a value of 4095 is represented by $FFF.

10.3.26 Time

The Time field consists of 3 octets representing the current time in hours, minutes, and seconds. The Time field is specified in Table 77.

Table 77 - Time Bit Assignments

Octet	7	6	5	4	3	2	1	0
0	Hours					Minutes		
1	Min (cont.)			Seconds				
2	*	0	0	0	0	0	0	0

* LSB of the Seconds field

Hours - This field specifies the hour of the time using 24-hour notation. Octet 0 bit 7 (b7) is the MSB of the hour and Octet 0 bit 3 (b3) is the LSB of the hour field. Valid values are 0 (00000 in binary) to 23 (10111 in binary).

Minutes - This field specifies the minutes of the time. This field is the 3 LS bits (b2 - b0) of octet 0 and the 3 MSBs (b7 - b5) of octet 1. The MSB of the Minutes field is Octet 0 bit 2 and the LSB is Octet 1 bit 5. Valid values are 0 (000000 in binary) to 59 (111011 in binary).

Seconds - This field specifies the seconds of the time. This field is the 5 LSBs (b4 - b0) of octet 0 and the MSB (b7) of octet 2. The MSB if the Seconds field is Octet 1 bit 4 and the LSB is Octet 2 bit 7. Valid values are 0 (000000 in binary) to 59 (111011 in binary).

Spare - Bit 6 (b6) through bit 0 (b0) of Octet 2 are not used and are set to zero.

For example, 16:03:58 would be represented by 10000 000011 111010 0000000 in binary (i.e. Hours = 10000, Minutes = 000011, and Seconds = 111010).

Note: The time base used within the system is system dependent and is controlled by the system manager. Whenever a system uses TOD to perform automatic key management, the choice of the time base shall take into account the effect of time zone differences and other factors (i.e. daylight savings time) in order to avoid loss of secure communication due to out of date keys. The use of Coordinated Universal Time is one example for a time base that may avoid the previously described problem.

11 MESSAGE ID ASSIGNMENTS

Table 78 - Message ID Assignment

Message Name	Type Value (Hex)	Message Name	Type Value (Hex)
Null	$00	Modify-Keyset-Attributes-Command	$14
Capabilities-Command	$01	Modify-Keyset-Attributes-Response	$15
Capabilities-Response	$02	Negative-Acknowledgment	$16
Change-RSI-Command	$03	No-Service	$17
Change-RSI-Response	$04	Reserved	$18
Changeover-Command	$05	Reserved	$19
Changeover-Response	$06	Reserved	$1A
Delayed-Acknowledgment	$07	Reserved	$1B
Delete-Key-Command	$08	Reserved	$1C
Delete-Key-Response	$09	Rekey-Acknowledgment	$1D
Delete-Keyset-Command	$0A	Rekey-Command	$1E
Delete-Keyset-Response	$0B	Set-Date-Time	$1F
Hello	$0C	Warm-Start-Command	$20
Inventory-Command	$0D	Zeroize-Command	$21
Inventory-Response	$0E	Zeroize-Response	$22
Key-Assignment-Command	$0F	Deregistration-Command	$23
Key-Assignment-Response	$10	Deregistration-Response	$24
Reserved	$11	Registration-Command	$25
Reserved	$12	Registration-Response	$26
Modify-Key-Command	$13	Unable-to-Decrypt-Response	$27

12 DATA TRANSPORT MECHANISMS

Two data transport mechanisms defined in this section are used to transfer OTAR Key Management messages. The first method uses the Common Air Interface (CAI) to encrypt the KMM, and is referred to as Data Link Dependent (DLD) OTAR. The second method uses the application to encrypt the KMM and is referred to as Data Link Independent (DLI) OTAR. With it, the key management message is completely encrypted above the CAI so additional encryption by the CAI is not needed.

Some messages used for key management depend on whether DLD or DLI OTAR is being used. Messages important to the registration and establishment of an OTAR communications session between a KMF and a target device are shown in Table 79.

Table 79 - Data Link Independent Messages

Message	Response Kind	Required/ Optional	Comment
Registration Command	1	Required	- Required for DLI OTAR. - Optional for DLD OTAR.
	3	Optional	
Registration Response	1	Optional	
Deregistration Command	1	Optional	
	3	Optional	
Deregistration Response	1	Optional	
Unable to Decrypt Response	1	Required	- Required for DLI OTAR. - Not applicable for DLD OTAR

12.1 Common Air Interface

The Common Air Interface (CAI) defines formats for the transmission of digital voice and data messages over the air interface and is defined in [20]. The CAI (U_m) for Packet Data is further defined in [7]. This document does not affect the operation or format of voice messages in the CAI, but it rests on the support of the CAI for the transmission of data messages.

12.1.1 CAI Confirmed Delivery

The CAI defines confirmed and unconfirmed delivery of data packets. In relation to OTAR, confirmation of delivery simply states that the recipient of the data packet was able to recover the message and pass the CRC checks defined in the CAI. Confirmed delivery in the CAI does not necessarily imply that an OTAR message was able to be decrypted by the recipient, nor does it imply that the message passes any of the check mechanisms defined in this document. Confirmed delivery is intended to support the transmission of large data packets, which require selective re-transmissions of separate blocks of the packets, in order to overcome transmission errors.

12.1.2 CAI Data Service Access Points

CAI data packets are sent with Service Access Point (SAP) identifiers. With DLD OTAR, the Unencrypted Key Management message SAP as defined in [5] is used. The Encrypted User Data SAP as defined in [2] and [5], and the Enhanced Address SAP as defined in [4] and [5] are also used. With DLI OTAR, only the Unencrypted User Data SAP as defined in [6] and [5] is used.

12.2 Data Link Dependent OTAR

With Data Link Dependent OTAR, the Key Management Application interfaces directly with the CAI Data Bearer Service in order to convey KMMs. The KMMs described in this document are logical messages at the CAI layer. The CAI Manufacturer's ID shall be set to $00. The Data Header Offset field shall be set to a value of 0. The types of CAI data packets supported for DLD OTAR are described in this section.

12.2.1 Encrypted Asymmetrically Addressed DLD OTAR

Confirmed and Unconfirmed Encrypted Asymmetrically Addressed CAI data is defined in [2]. The SAP Identifier in the Data Packet Header Block is set to the Encrypted User Data SAP per [2]. The 2ndary SAP in the ES Auxiliary Header shall be set to the Unencrypted Key Management message SAP.

12.2.2 Unencrypted Asymmetrically Addressed DLD OTAR

Confirmed and Unconfirmed Unencrypted Asymmetrically Addressed CAI data is defined in [4]. The SAP Identifier in the Data Packet Header Block shall be set to the Unencrypted Key Management message SAP.

12.2.3 Encrypted Symmetrically Addressed DLD OTAR

Confirmed and Unconfirmed Encrypted Symmetrically Addressed CAI data is defined in [4] and [2]. An Enhanced Address Second Header as defined in [4] is followed by an ES Auxiliary Header as defined in [2]. The SAP Identifier in the Data Packet Header Block is set to the Enhanced Address SAP per [4]. The SAP Identifier in the Enhanced Address Second Header is set to the Encrypted User Data SAP per [2]. The 2ndary SAP in the ES Auxiliary Header shall be set to the Unencrypted Key Management message SAP.

12.2.4 Unencrypted Symmetrically Addressed DLD OTAR

Confirmed and Unconfirmed Unencrypted Symmetrically Addressed CAI data is defined in [4]. The SAP Identifier in the Data Packet Header Block is set to the Enhanced Address SAP per [4]. The SAP Identifier in the Enhanced Address Second Header shall be set to the Unencrypted Key Management message SAP.

12.3 Data Link Independent OTAR

This section defines Data Link Independent OTAR messaging over the CAI. Data Link Independent OTAR messaging over other system interfaces (such as E_d) is beyond the scope of this document.

DLI OTAR is agnostic to physical layer and data link layer network interfaces and can be used on any interface that supports UDP/IP. When DLI OTAR is used on the P25 CAI, the KMMs are tunneled through SNDCP using the P25 IP Data Bearer Service.

DLI OTAR shall use the IP Data Bearer Service over the CAI. KMMs are encapsulated by the protocols shown in Figure 4. The format of the resulting CAI logical message for the Data Link Independent KMM is shown in Figure 37.

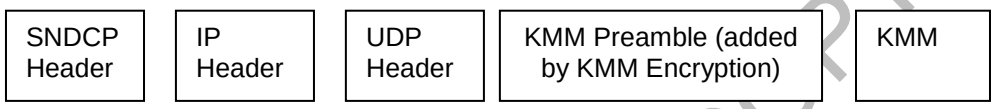


Figure 37 - Data Link Independent KMM Datagram

In order to better support a Data Link layer independent based structure, encryption of the KMM occurs at the application layer per [2]. The physical layer or data link layer may additionally encrypt the OTAR payload but it's not necessary for OTAR operation.

12.3.1 KMM

KMMs are defined in Section 10.

12.3.2 KMM Preamble

The KMM Preamble is used to specify encryption of the KMM and shall be included in all DLI OTAR messages. The KMM Preamble consists of a KMM Preamble Format octet and a KMM Preamble Message Body (Figure 38). The format of the KMM Preamble Message Body is defined by the KMM Preamble Format version number. At this time, only version 0 is supported.

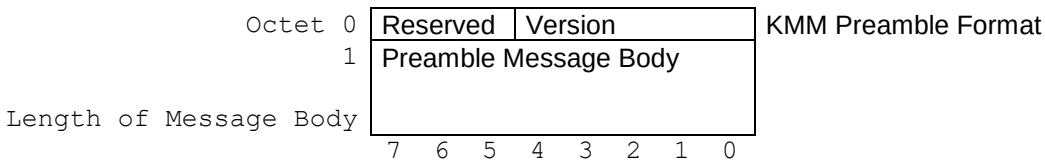


Figure 38 - KMM Preamble Structure

KMM Preamble Format – This octet contains 3 reserved bits and a version number that selects which definition of the Preamble Message Body is to be used.
Reserved - These 3 bits are reserved for future use and shall be set to 0.
Version - These 5 bits define which version of the KMM Preamble Message Body is used. Currently this field supports version 0 only (%00000).

Preamble Message Body – This block shall be uniquely defined for each version number. The Preamble Message Body shall be defined in the following sections for valid version numbers.

12.3.2.1 Preamble Message Body - Version 0

The Preamble Message Body contains the Manufacturer Identifier, Algorithm ID, Key ID and the Message Indicator as shown in Figure 39 when the version number is set to 0 in the KMM Preamble Format octet.

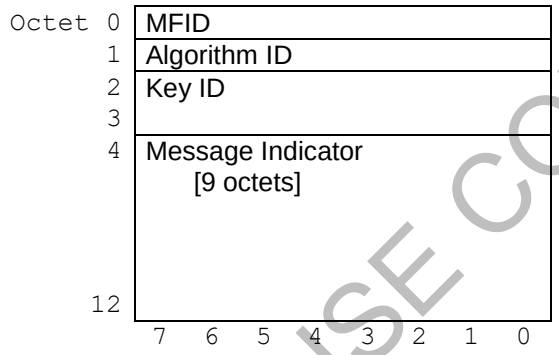


Figure 39 - Preamble Message Body Format - Version 0

MFID - The MFID is used to indicate whether the Algorithm ID is a standardized algorithm value or if it is a Manufacturer unique value. The MFID also indicates whether, any portion of the KMM Preamble and/or KMM is proprietary to the Manufacturer identified by the MFID. The format for this field is defined in [5].

Algorithm ID - The Algorithm ID is used to indicate the Algorithm used to encrypt the KMM. The format for this field is defined in the Primitive Field Definitions for Algorithm ID.

Key ID - The Key ID indicates the TEK used to encrypt the KMM. The format for this field is defined in the Primitive Field Definition for Key ID.

Message Indicator - Provides the Message Indicator (encryption synchronization) for the encryption algorithm when the message is sent encrypted. This field contains all zeros if the Algorithm ID is set to $80 and the Key ID is set to $0000 (indicating a clear message). The format for this field is defined in the Primitive Field Definition for Message Indicator.

12.3.3 Transport Layer

The User Datagram Protocol (UDP) as defined in [19] shall be used as the transport layer. The port number in the UDP header is used to identify the application message as an OTAR DLI message. The P25 OTAR port value shall be used as the default as specified in [9].

12.3.4 Network Layer

As specified in [6], the IP Data Bearer Service uses the Internet Protocol (IP) as the network layer. For messages originated by an SU, the source IP address contains the SU's address and the destination IP address contains the KMF's IP address. When messages are originated by a KMF, these fields are reversed. IP fragmentation of OTAR messages across the CAI is not supported.

12.3.5 Data Link Layer

P25 uses the Internet Protocol (IP) data bearer service as defined in [6] as the delivery method for DLI OTAR messages over the CAI. Other interfaces that support UDP/IP are possible (e.g. Wi-Fi, broadband, etc.) but are outside the standard. For backward compatibility, Data Link Independent OTAR messages are only conveyed across the CAI using the SNDCP protocol. Other configurations such as the IP Data Bearer Service in the Conventional FNE Data Configuration with the SCEP protocol shall use Data Link Dependent OTAR. In this case, the CAI Data Bearer Service is used even though the IP Data Bearer Service is available.

The encapsulation of IP datagrams into SNDCP messages is defined in [6].

At the CAI, the logical message consists of the blocks shown in Figure 37. The Manufacturer's ID shall be set to a value of $00. The Data Header Offset field is set to a value of 2 per [6]. Since additional encryption is not performed by the CAI and DLI OTAR is only supported in the Trunked and Conventional FNE Data configurations, all CAI data is sent as unencrypted data using asymmetrical addressing. Confirmed and Unconfirmed Unencrypted Asymmetrically Addressed CAI data is defined in [4]. The SAP Identifier in the Data Packet Header Block is set to the Unencrypted User Data SAP per [6].

12.4 Maximum Message Length

KMM length shall be limited such that a CAI logical message resulting from a KMM has a maximum length of 512 octets. The maximum length of a KMM varies depending on the OTAR protocol. These limits are specified in Table 80. If an invalid message length is received (less than 7 octets or greater than Maximum KMM Data Length minus 3) the KMM shall be discarded.

Table 80 - Maximum Message Data Length

Protocol	SNDCP	IPv4	UDP	KMM Preamble Version 0	Max KMM Data Length	KMM Message Length (Data Length – 3)
OTAR DLD	-	-	-	-	512	509
OTAR DLI	2	24	8	13	465	462

13 KEY MANAGEMENT SECURITY REQUIREMENTS FOR BLOCK ENCRYPTION ALGORITHMS

This section describes the general security requirements for transmitting encrypted KMMs. They include the encryption, decryption, and integrity (MAC) of KMMs; as well as a mechanism used to protect against the replay of previously sent KMMs.

This document provides support for legacy algorithms (DES and Triple DES) as well as AES, but is not limited to them. OTAR supports block encryption algorithms with a block size that is a multiple of 2 octets. Standardized encryption algorithms are specified in [5]. Algorithms not listed in [5] indicate non-standardized or proprietary encryption algorithms.

Key Management Messages defined in this document shall be transmitted over the Common Air Interface using the data transport mechanisms as specified in Section 12.

The key used to outer layer encrypt the KMM shall be a TEK. The choice of the TEK should be left to the responsibility of the Administrator of the Key Management System. For encryption, any TEK and algorithm that the KMF and SU have in common may be selected in accordance with the following provisions.

KMMs shall be outer layer encrypted and MAC'd using the same algorithm type and key length. When inner layer encryption is used, it shall use the same algorithm type and key length as the outer layer encryption and MAC. When a key is transported in a KMM, the algorithm type and key length used to inner layer encrypt, outer layer encrypt and MAC the KMM shall be equal to or greater than the security strength of the transported key. In other words, a less secure algorithm SHALL NOT be used to inner layer encrypt, outer layer encrypt, or MAC a KMM whose payload includes a key (TEK or KEK) of a more secure algorithm.
KMM commands that do not transport a key shall be encrypted and authenticated using the strongest algorithm present in the KMF and SU, as determined by the initiator of the command. Provided the KMM (command or key transporting) is validated and authenticated according to the rules in this document, the radio SHOULD execute the command.

Table 81 provides a recommended KMM encryption hierarchy for transporting keys.

Table 81 - Recommended Security Hierarchy

Transported Key Type	Acceptable Inner Layer Algorithm	Acceptable Outer Layer Algorithm	Acceptable Message Authentication Algorithm
DES[9]	DES or AES-256	DES or AES-256	DES or AES-256
TDES	TDES or AES-256	TDES or AES-256	TDES or AES-256
AES-256	AES-256	AES-256	AES-256

[9] DES is only utilized for backward compatibility, see NOTE in section 1.1.

Whenever possible, the recipient of the encrypted and/or authenticated message responds with an encrypted and/or authenticated response message using the same key (or keys) and algorithm (referred to as “Respond in Kind”). In other words, if a validated KMM is received clear, the response may also be sent clear. If the command is encrypted and/or authenticated, the key (or keys) used to encrypt and/or authenticate the command shall be the same key (or keys) used to encrypt and/or authenticate the response. The Warm-Start Procedure is an exception to “Respond in Kind” (see Section 8.16).

All messages shall be encrypted with the following exceptions:

1. Capabilities-Command. It is recommended that the Capabilities-Command be sent encrypted; however, it may be sent clear at the manufacturer’s discretion.
2. Capabilities-Response. The Capabilities-Response shall be encrypted if the Capabilities-Command was received encrypted. If the Capabilities-Command is received unencrypted the response may be sent back either encrypted or clear.
3. Delete-Key-Response. The Delete-Key-Response shall be encrypted and/or authenticated when the key specified for deletion is NOT the key used to encrypt and/or authenticate the Delete-Key-Command message.
   If the key specified for deletion is the key used to encrypt the Delete-Key-Command message, then the Delete-Key-Response message may be encrypted and/or authenticated. In this case, the Delete-Key-Response may be sent clear and authenticated, or it may be sent encrypted and/or authenticated using the specified key(s) immediately prior to deletion of the key(s). For continued interoperability with existing equipment, both encrypted and unencrypted/authenticated responses shall be supported in this case.
4. Delete-Keyset-Response. The Delete-Keyset-Response shall be encrypted and/or authenticated when the keyset specified for deletion does NOT contain the key used to encrypt the Delete-Keyset-Command message.
   If the keyset specified for deletion contains the key(s) used to encrypt and/or authenticate the Delete-Keyset-Command message, then the Delete-Keyset-Response message may be encrypted and/or authenticated. In this case, the Delete-Keyset-Response may be sent clear and authenticated, or it may be sent encrypted and/or authenticated using the appropriate key(s) immediately prior to deletion of the specified keyset. For continued interoperability with existing equipment, both encrypted and unencrypted/authenticated responses shall be supported in this case.
5. Deregistration-Command. The Deregistration-Command may be sent encrypted or clear at the manufacturer’s discretion.
6. Deregistration-Response. The Deregistration-Response shall be encrypted if the Deregistration-Command was received encrypted. If the Deregistration-Command is received unencrypted, the response may be sent back either encrypted or clear.
7. Hello. The Hello command is typically transmitted clear, however the Hello command may also be sent encrypted in accordance with local security policies.

8. No Service. The No Service response shall be encrypted if the Hello or Registration-Command message was received encrypted. If the Hello or Registration-Command message is received unencrypted the No Service response may be sent back either encrypted or clear.
9. Registration-Command. The Registration-Command may be sent encrypted or clear at the manufacturer's discretion.
10. Registration-Response. The Registration-Response shall be encrypted if the Registration-Command was received encrypted. If the Registration-Command is received unencrypted, the response may be sent back either encrypted or clear.
11. Unable-to-Decrypt-Response. The Unable-to-Decrypt-Response may be sent encrypted or clear at the manufacturer's discretion.
12. Warm-Start-Command. The Warm-Start-Command by definition is sent clear, although the key variable contained in the command is encrypted (wrapped).
13. Zeroize-Response. The Zeroize-Response message may be sent encrypted, or it may be sent clear and authenticated. For continued interoperability with existing equipment, both encrypted and unencrypted/authenticated responses shall be supported. In either case, the Zeroize-Response message is constructed and if all TEKs and KEKs are deleted successfully, the Zeroize-Response is then sent.

A Negative Acknowledgment shall not be sent if it cannot be sent encrypted and authenticated. If a target receives an unencrypted NACK, the message shall be discarded without any action(s) taken.

13.1 **Encryption Modes**

Security of a KMM requires that an n-bit block encryption algorithm be used, where n is defined as the block size. It also requires that the block size be an integer multiple of 2 octets (n modulo 16 = 0). An n-bit algorithm shall consist of an n-bit input register, a cipher function that operates on the n-bit input using a k-bit key variable, denoted as K, to produce an n-bit output result and an n-bit output register. The key variable may be any size of k bits. The cipher function typically consists of permutations and non-linear substitutions done in multiple rounds controlled by the key variable.

The block encryption algorithm typically consists of an encryption and a decryption function that are inverses of each other. Encryption is the transformation of a usable message, called the plaintext, into an unreadable form, called the ciphertext. Decryption is the transformation that recovers the plaintext from the ciphertext.

For any given key, K, the underlying block encryption algorithm consists of two functions that are inverses of each other. The encryption function is denoted as $CIPH_K$ and the decryption function is denoted as $CIPH^{-1}_K$.

Key Management Messages use the following block cipher modes:

1. Output Feedback (OFB) for outer-layer encryption (see [2])
2. Electronic Code Book (ECB) for key wrap (see 13.1.1)

3. Cipher Block Chaining (CBC) for CBC-MAC and CMAC (see 13.1.2 and 13.1.3, respectively)

NOTE: CMAC is the recommended OTAR MAC.

The above block cipher modes of operation are described in the following sections.

The following parameters are defined for the encryption of the key frame and for the CBC-MAC method of authentication of a Key Management Message (KMM). For the parameters used in the CMAC method of authenticating a KMM, refer to section 13.5.2.2.2.

A key frame contains the key variable and pad bits as required. The KMM includes the message and any required pad octets.

K = the key used to encrypt (or decrypt) the plaintext (ciphertext), the Key Encryption Key
n = number of bits in the encryption algorithm block
k = number of bits in the key variable (including any parity bits)
x = number of n/2-bit blocks required to encrypt the key variable
= ceiling [2*k/n]
r = number of pad bits required to expand the length of the key variable to an integer multiple n/2-bit blocks
= (x * n/2) – k
t = number of bits in the MAC field
L = number of octets in the key or message block
= (x+1) * n/2
m# = specifies one of the octets in the encryption algorithm block
0 <= m# < M
M = the total number of octets in the encryption algorithm
= n/8

P_i = the i^{th} plaintext data block
C_i = the i^{th} ciphertext data block
A = the n/2-bit integrity check register
B = the output of the cipher or inverse cipher function
s = the number of steps in the key wrapping process

$MSB_j(W)$ = return the most significant j bits of the register W
$LSB_j(W)$ = return the least significant j bits of the register W
$R_1|R_2$ = concatenate R_1 and R_2

13.1.1 Electronic Codebook (ECB) Description

The Electronic Codebook (ECB) mode is defined as follows and is shown in Figure 40:

ECB Encryption: $C = CIPH_K (P)$

ECB Decryption: $P = CIPH^{-1}_K (C)$

In ECB encryption, the forward cipher function is applied directly, and independently, to each block of the plaintext. The resulting sequence of output blocks is the ciphertext. In ECB decryption, the inverse cipher function is applied directly, and independently, to each block of the ciphertext. The resulting sequence of output blocks is the plaintext.

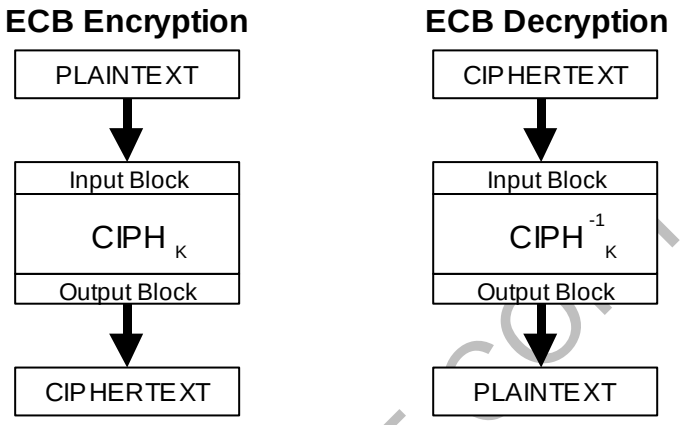


Figure 40 - The ECB Mode

In ECB encryption and ECB decryption, multiple forward cipher functions and inverse cipher functions can be computed in parallel. In the ECB mode, under a given key, a given plaintext input block always results in the same ciphertext output block.

13.1.2 The Cipher Block Chaining-Message Authentication Code Mode

When the initiating party creates a MAC using CBC-MAC, the “Version” field in the enhanced MAC frame is set to %00000 and the message is authenticated using the CBC-MAC mode.

The CBC-MAC mode for authentication of datagrams is shown in Figure 41.

CBC-MAC Generation:
$O_1 = CIPH_K(P_1)$;
$O_j = CIPH_k(P_j \wedge O_{j-1})$ for j = 2 .. n;
$MAC = MSB_t(O_n)$

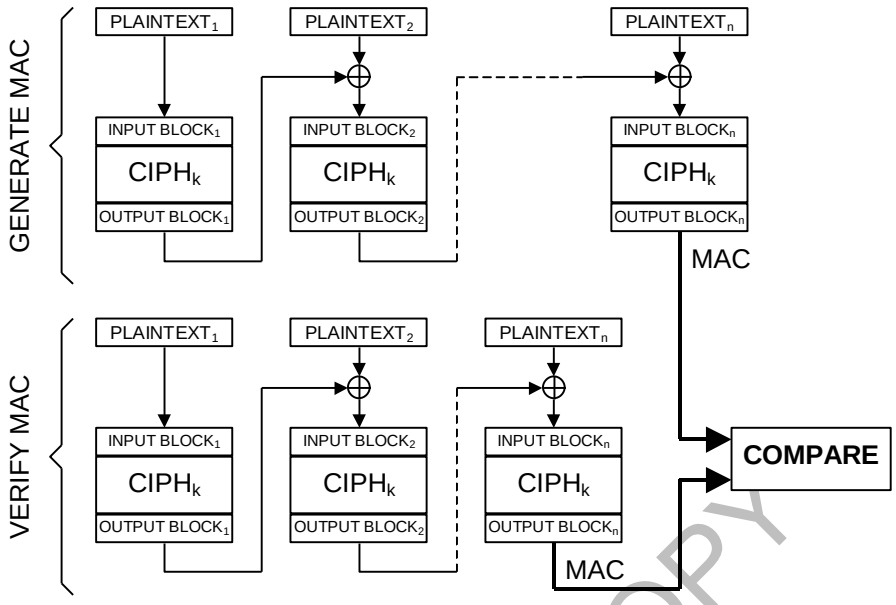


Figure 41 - The CBC-MAC Mode

In CBC-MAC generation, the first output block is generated by applying the forward cipher function to the first block of plaintext. The first output block is exclusive-ORed with the second plaintext data block to produce the second input block and the forward cipher function is applied to produce the second output block. This output block is exclusive-ORed with the next plaintext block to generate the next input block. All the successive plaintext blocks are "chained" in this way with the previous output block to produce the input blocks. The forward cipher function is applied to each input block to produce the output blocks. The MAC is taken from the final output block.

For CBC-MAC verification, the verifying party generates the CBC-MAC as described above, and then compares the result to the received MAC value. If the MAC values are the same, then the verification is successful, and the data is considered to be authentic. If the MAC values do not match, then the verification is unsuccessful.

The final output block, OUTPUT $BLOCK_n$, may be truncated, so that its length in bits, denoted as t, is less than or equal to n but greater than or equal to 32 bits. In this case the MAC is defined as $MAC = MSB_t(OUTPUT\ BLOCK_n)$.

13.1.3 The Cipher-based Message Authentication Code Mode

The Cipher-based Message Authentication Code (CMAC) as defined in [26], is a mode used for authentication of datagrams (KMMs).

Use of the CMAC algorithm versus the CBC-MAC algorithm is indicated by the "Version" parameter contained in the enhanced MAC frame (section 13.5.2.3).

When an initiating party authenticates a KMM using CMAC, the “Version” in the enhanced MAC frame is set to %00001 and the message is authenticated using CMAC as defined in section 13.5.2.3.2.

Upon receipt of an authenticated message, the receiving party checks the “Version” field of the enhanced MAC frame. If the value of the “Version” field is %00001, then the receiving party recreates the MAC over the received message using CMAC as defined in section 13.5.2.3.2. The resulting output is compared against the received MAC value. If the MAC values are the same, then the verification is successful and the data is considered to be authentic. If the MAC values do not match, then the verification is unsuccessful.

13.2 DES Key Encryption (Key Wrapping) Requirements

All DES[10] Type 3 keys contained in KMMs, whether KEKs or TEKs, are 64-bits in length i.e., parity corrected DES keys. All DES keys contained in KMMs that are inner layer encrypted using DES shall use the Electronic Codebook Mode (ECB) as defined in [11]. The key used for this encryption process shall be a KEK. KEKs shall only be used to encrypt other keys.

The process for encrypting a plain text key is illustrated in Figure 42. The parity corrected key is entered into the DES input Block and a single DES encryption is performed resulting in a 64-bit output block. As described in [11], bit B1 of the key (leftmost bit) is mapped into DES input block bit I1, bit B2 is mapped into bit I2 of the input block and so forth with key bit P8 (rightmost key bit) mapped into bit I64 of the input block.

[10] DES is only utilized for backward compatibility, see NOTE in section 1.1.

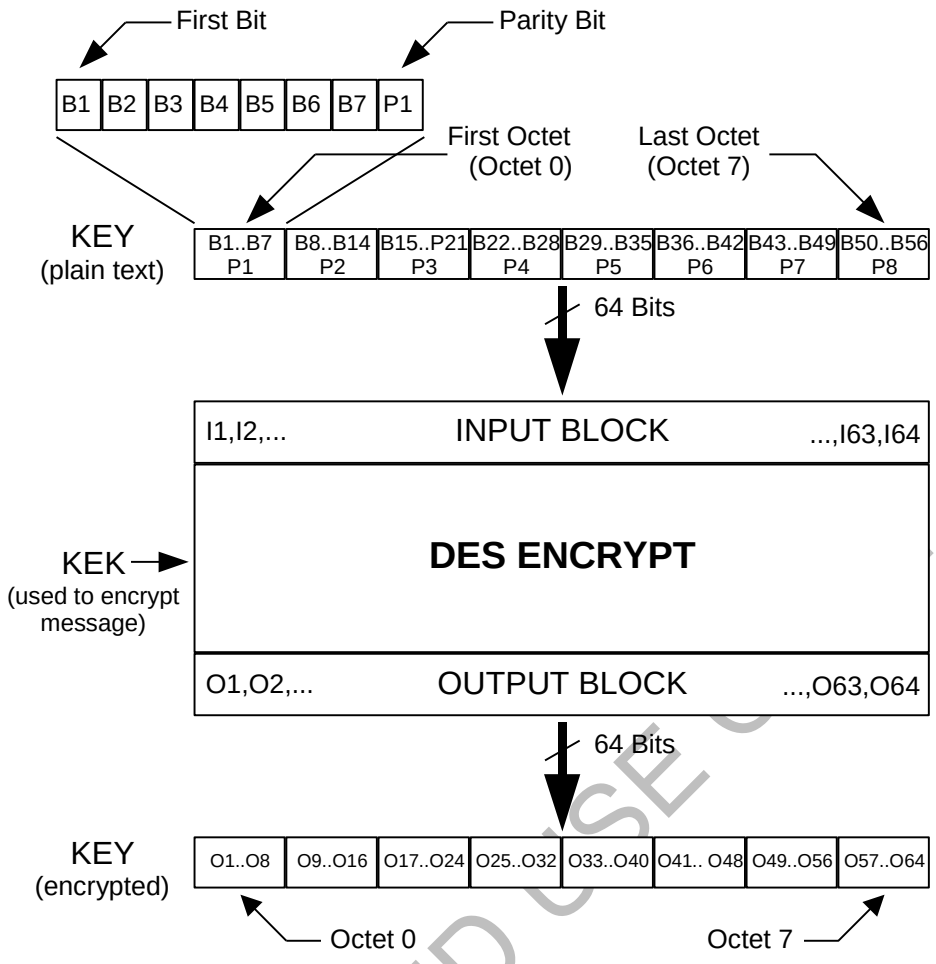


Figure 42 - Key Encryption Process

13.3 Enhanced Key Encryption (Key Wrapping) Requirements

All supported block encryption algorithms (except DES) encrypt keys using the KW-AE encrypt and KW-AD decrypt functions as specified in SP 800-38F [14]. Figure 43 provides a description of the key encryption process.

Specification SP 800-38F [14] defines a key frame as the combination of the key bits and pad bits, if required. The key field shall contain the key variable (including the parity or CRC bits as defined in the specification for the encryption algorithm). The cipher function called by the key wrapping algorithm shall execute the encryption algorithm in the ECB mode.

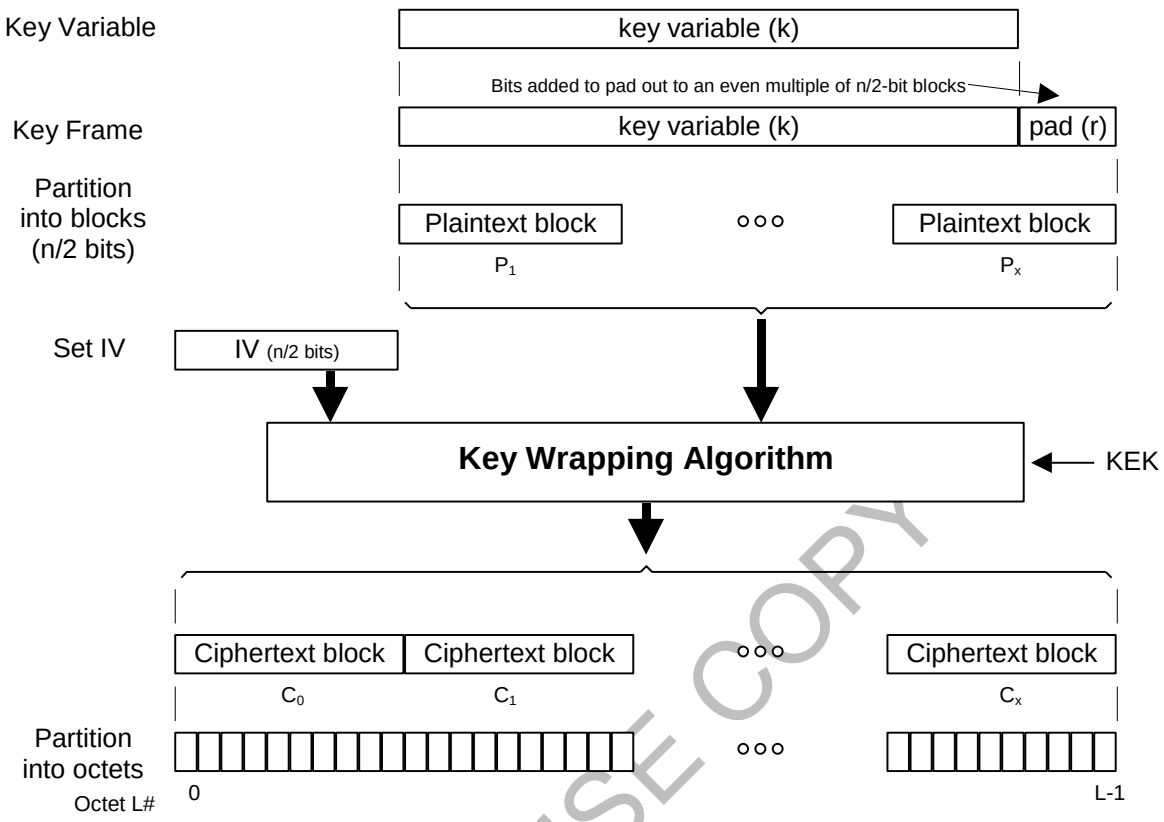


Figure 43 - Key Frame Encryption

Figure 43 shows the procedure to encrypt a key variable. This procedure is outlined below:

1. Calculate the number of n/2-bit blocks (x) required to encrypt the key variable (k). x = ceiling[2*k/n]
2. Calculate the number of pad bits (r) required to fill the encryption blocks and append the pad bits to the end of the key. The pad bits should be random. r = (x * n/2) – k
3. Break the key and the pad bits into x n/2-bit blocks (P_1, …, P_x). These blocks are the plaintext input into the Key Wrap Algorithm. P_1 is the left most key bits and P_x is the right most key bits (and pad bits if required).
4. Generate an IV of n/2 bits where the IV consists of a binary pattern of 10100110 repeated until all n/2 bits are defined.
5. Using the IV generated in step 4 and the plaintext array generated in step 3, call the key wrap algorithm to generate a x+1 array of n/2-bit ciphertext vectors.
6. Parse the ciphertext (C_0, C_1, …, C_x) into octets and place them in the key field of the KMM.

The above procedure is executed in reverse to recover the key. The key unwrap algorithm is used in place of the key wrap algorithm in step 5. The final output of the A register is compared to the expected initialization vector in step 4 and if the two registers match the key is accepted. Any pad bits inserted in step 2 shall be removed after the key is recovered.

Enhanced key encryption applies to TEKs and KEKs, and includes encryption of keys with algorithm types that are not the same as the inner layer algorithm (e.g. DES, TDES, or AES-128 if using AES-256). The plaintext key (regardless of algorithm type) is inserted as the Key Variable component shown in Figure 43 and padded out to the appropriate block size (n/2 bits).

Fields in the transporting KMM provide indication as to the type of key that has been encrypted. For example, if AES-256 were used to inner layer encrypt DES[11] or AES-128 keys, the Algorithm ID field (immediately following the Keyset ID field) in the Modify-Key Command would indicate an algorithm type of DES if the key(s) being transported were DES keys, and an algorithm type of AES-128 if the key(s) being transported were AES-128 keys. The Algorithm ID field (octet 1 of the Decryption Instruction Block) would indicate an inner layer algorithm type of AES-256.

Restrictions for inner layer encryption of keys with different algorithm types can be found in Table 81.

13.3.1 Key Wrap Algorithm

The inputs to the key wrapping process are the KEK and the plaintext to be wrapped. The plaintext consists of x n/2-bit blocks containing the key data being wrapped. The key wrapping process is described below.

Inputs: Plaintext, x n/2-bit values $\{P_1, \ldots, P_x\}$,
Key, K (the KEK)
Initialization Vector, IV
Outputs: Ciphertext, (x+1) n/2-bit values $\{C_0, C_1, \ldots, C_x\}$

1) Initialize variables
Set A = IV
For i = 1, ..., x
$R_i = P_i$
2) Calculate intermediate values
For j = 0, 1, … , 5
For i = 1, …, x
$B = ECB_K (A|R_i)$
$A = MSB_{n/2}(B) \text{ XOR } ((x*j) + i)$
$R_i = LSB_{n/2}(B)$
3) Output the results
Set $C_0 = A$
For i = 1, …, x
$C_i = R_i$

[11] DES is only utilized for backward compatibility, see NOTE in section 1.1.

13.3.2 Key Unwrap Algorithm

The inputs to the unwrap process are the KEK and (x+1) n/2-bit blocks of ciphertext consisting of previously wrapped key. It returns x blocks of plaintext consisting of the x n/2-bit blocks of the decrypted key data.

Inputs: Ciphertext (x+1) n/2-bit values $\{C_0, C_1, \ldots, C_x\}$,
Key, K (the KEK)
Outputs: Plaintext x n/2-bit values $\{P_1, \ldots, P_x\}$
Error status

1) Initialize variables
Set $A = C_0$
For $i = 1, \ldots, x$
$R_i = C_i$
2) Compute intermediate values
For $j = 5, \ldots, 0$
For $i = x, x-1, \ldots, 1$
$B = ECB^{-1}_K ((A \text{ XOR } ((x*j) + i)) \mid R_i)$
$A = MSB_{n/2}(B)$
$R_i = LSB_{n/2}(B)$
3) Output results
If A is equal to IV
Then
For $i = 1, \ldots, x$
$P_i = R_i$
Else
Return an error

13.4 Field Primitives

13.4.1 Key Field Primitive for DES[12] Key Encryption

The Primitive Field Definition for an encrypted DES Type 3 Key consists of 8 octets and is given in Table 82 in terms of the DES output block.

[12] DES is only utilized for backward compatibility, see NOTE in section 1.1.

Table 82 - Key Format

Octet		Output Bits O1-O8
	1	Output Bits O9-O16
	2	Output Bits O17-O24
	3	Output Bits O25-O32
	4	Output Bits O33-O40
	5	Output Bits O41-O48
	6	Output Bits O49-O56
	7	Output Bits O57-O64
		7 6 5 4 3 2 1 0

Output Bits O1-O8 - This field contains the first octet of the DES Output register as shown in Figure 42. The MSB (b_7) contains the 1st output bit (O1) and the LSB (b_0) contains the 8th output bit (O8).

Output Bits O9-O16 - This field contains the second octet of the DES Output register as shown in Figure 42. The MSB (b_7) contains the 9th output bit (O9) and the LSB (b_0) contains the 16th output bit (O16).

Output Bits O17-O24 - This field contains the third octet of the DES Output register as shown in Figure 42. The MSB (b_7) contains the 17th output bit (O17) and the LSB (b_0) contains the 24th output bit (O24).

Output Bits O25-O32 - This field contains the fourth octet of the DES Output register as shown in Figure 42. The MSB (b_7) contains the 25th output bit (O25) and the LSB (b_0) contains the 32nd output bit (O32).

Output Bits O33-O40 - This field contains the fifth octet of the DES Output register as shown in Figure 42. The MSB (b_7) contains the 33rd output bit (O33) and the LSB (b_0) contains the 40th output bit (O40).

Output Bits O41-O48 - This field contains the sixth octet of the DES Output register as shown in Figure 42. The MSB (b_7) contains the 41st output bit (O41) and the LSB (b_0) contains the 48th output bit (O48).

Output Bits O49-O56 - This field contains the seventh octet of the DES Output register as shown in Figure 42. The MSB (b_7) contains the 49th output bit (O49) and the LSB (b_0) contains the 56th output bit (O56).

Output Bits O57-O64 - This field contains the eighth (last) octet of the DES Output register as shown in Figure 42. The MSB (b_7) contains the 57th output bit (O57) and the LSB (b_0) contains the 64th output bit (O64).

13.4.2 Key Field Primitive for Enhanced Key Encryption

The primitive field definition for an encrypted Type 3 key consists of L octets. This is shown in Table 83 in terms of the cipher block (C_i) from the output of the key wrap algorithm and the octet number (m) in the cipher block. The first n/2 bits are the integrity check vector, the next k bits are the key bits and the last r bits are the pad bits, if used.

Table 83 - Key Field Format

Octet 0	Cipher Block 0 Octet M/2-1							
1	Cipher Block 0 Octet M/2-2							
m/2-2	Cipher Block 0 Octet 1							
m/2-1	Cipher Block 0 Octet 0							
m/2	Cipher Block 1 Octet M-1							
L-1	Cipher Block x Octet 0							
	7	6	5	4	3	2	1	0

Key Field Format

Cipher Block 0 Octet M/2-1 - This field contains the left most (most significant) 8 bits of the Ciphertext Block (C_0) output from the key wrap algorithm. The MSB (b_7) contains the left most ciphertext bit.

Cipher Block 0 Octet M/2-2 - This field contains the next 8 bits of the Ciphertext Block.

…

Cipher Block x Octet 0 - This field contains the last (least significant) 8 bits of the Ciphertext Block (C_x). The LSB (b_0) contains the right most ciphertext bit.

13.5 **Message Authentication**

With a few exceptions (see below), all KMMs require a Message Authentication Code (MAC). The key used to authenticate the KMM is identified in the MAC field of the message itself. If the message fails to authenticate, a Negative-Acknowledgement message may be sent in response.

All Key Management Messages shall contain a Message Authentication Code (MAC) block except in the following cases:

1. Capabilities-Command. It is recommended that the Capabilities-Command contain a MAC, however, it may be sent without a MAC at the manufacturer's discretion as long as the message does not include a Message Number.
2. Capabilities-Response. The Capabilities-Response shall contain a MAC if the Capabilities-Command that was received contained a MAC. If the Capabilities-Command is received without a MAC then the response may be sent back without a MAC.
3. Deregistration-Command. It is recommended that the Deregistration-Command contain a MAC, however, it may be sent without a MAC at the manufacturer's discretion as long as the message does not include a Message Number.
4. Deregistration-Response. The Deregistration-Response shall contain a MAC if the Deregistration-Command that was received contained a MAC. If the Deregistration-Command is received without a MAC then the response may be sent back without a MAC.
5. Hello. The Hello message is typically sent without a MAC.
6. No Service. The No Service message is typically sent without a MAC.

7. Registration-Command. It is recommended that the Registration-Command contain a MAC, however, it may be sent without a MAC at the manufacturer's discretion as long as the message does not include a Message Number.
8. Registration-Response. The Registration-Response shall contain a MAC if the Registration-Command that was received contained a MAC. If the Registration-Command is received without a MAC then the response may be sent back without a MAC.
9. Unable-to-Decrypt-Response. It is recommended that the Unable-to-Decrypt-Response contain a MAC, however, it may be sent without a MAC at the manufacturer's discretion as long as the message does not include a Message Number.

Any message, including the messages listed above, which contain a Message Number, shall contain a MAC. A message is discarded if it is received containing a Message Number and does not include a MAC. The KMF and SU shall only accept a message that has a valid MN and MAC.

A Negative Acknowledgment shall not be sent if it cannot be sent authenticated (and encrypted). If a target receives an unauthenticated NACK, it shall be discarded.

There are three modes that may be used for KMM authentication; DES message authentication using CBC-MAC, Enhanced Message Authentication using CBC-MAC, or Enhanced Message Authentication using CMAC.

NOTE: DES message authentication using CBC-MAC should not be used other than for backwards compatibility with older equipment that ONLY supports DES CBC-MAC.

13.5.1 DES[13] Message Authentication using CBC-MAC

Messages shall be authenticated using DES in either the 64-bit Cipher Block Chaining (CBC) mode of operation or the 64-bit Cipher Feedback (CFB) mode of operation as specified in [11]. Either mode of operation produces the same MAC, provided the algorithm is initialized as follows: 1) for CBC, the Initialization Vector (IV) is loaded with all zeros or 2) for CFB; the Initialization Vector is loaded with the first 64 bits of the data to be authenticated.

Figure 44 shows the authentication process for the CBC mode of operation. The DES input register is initialized with the first 64-bit block of data (D1). (For KMMs this first input block would consist of the Message ID, Message Length, Message Format, and Destination RSI fields, and the first octet of the Source RSI field with the Message ID left justified in the DES input register.) This input block is encrypted using the key chosen for authentication, which results in a 64-bit output block (O1). This cipher text output block is bit-wise exclusive-ored with the second 64-bits of data to be authenticated. The result of this operation is then loaded into the DES input register.

[13] DES is only utilized for backward compatibility, see NOTE in section 1.1.

This block is encrypted and the resultant output block is exclusive-ored with the third 64-bits of data to be authenticated.

If the message does not consist of an integral number of 64-bit data blocks, the final partial data block (D_n) shall be left justified and 0's appended to fill out a full data block. This padded data block is then exclusive-ored with the current output cipher text block (O_{n-1}). The resultant block is input into the DES and encrypted to produce a final output block. The leftmost 32-bits of this final block are defined as the MAC. (All cipher text generated by this process is discarded.) The message data to be authenticated starts with the Message ID field and continues through the first three octets of the MAC field i.e., the (authentication) Algorithm ID and the (authentication) Key ID. This identical process is used to compute a MAC on the received message and verify the received MAC at the message destination.

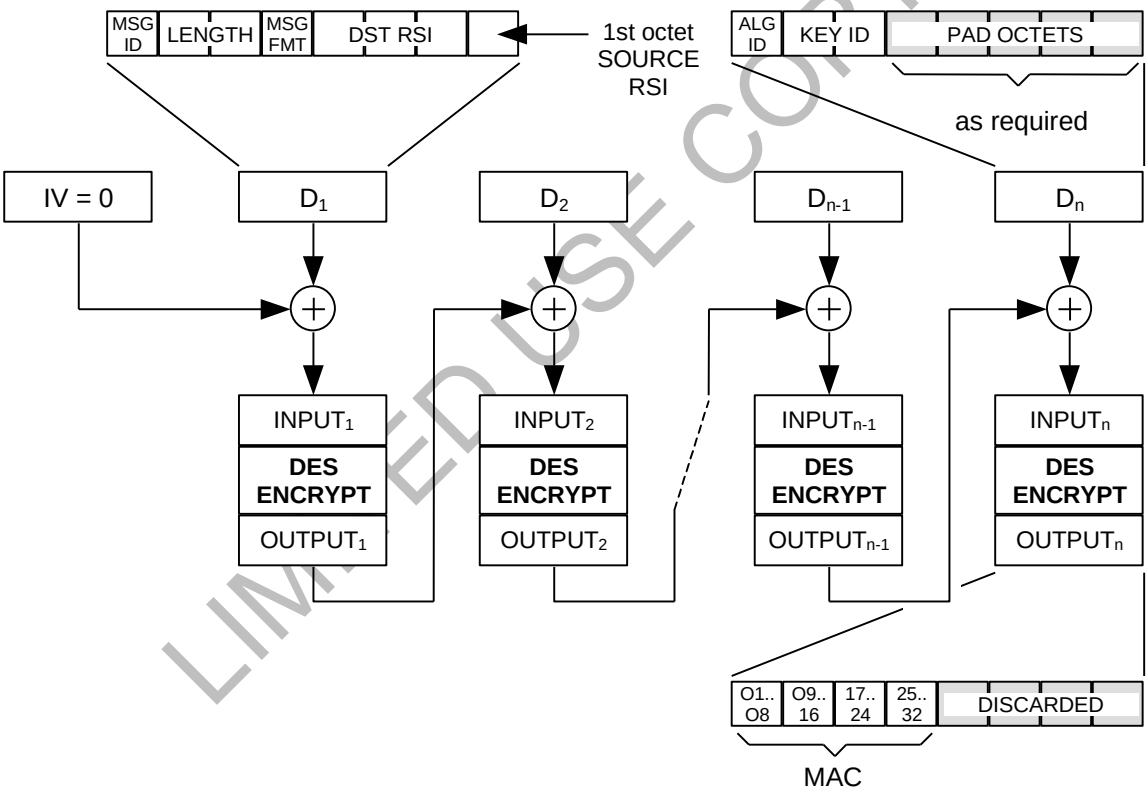


Figure 44 - The Authentication Algorithm

13.5.1.1 DES[14] Message Authentication Key Requirements

The key used to generate the MAC shall be any TEK held in common between the originator and destination(s). (Note: It is permissible that a TEK be reserved strictly

[14] DES is only utilized for backward compatibility, see NOTE in section 1.1.

for authentication purposes.) Warm Start messages are authenticated using the TEK contained in the message (after it is decrypted). The Unique Key ID field in the MAC block is undefined for Warm Start messages and Reverse Warm Start segments. The SU only accepts messages with a valid MAC and MN.

13.5.1.2 DES[15] Message Authentication Code (MAC) Block

This DES MAC block is used to identify the key used to authenticate the message as well as providing the MAC result itself. The message body format is shown in Table 84.

Table 84 - MAC Block Format

Octet		7	6	5	4	3	2	1	0
	0	Algorithm ID							
	1	Key ID							
	2								
	3	MAC							
	4								
	5								
	6								

Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select a key. The format for this field is defined in the Primitive Field Definition for Algorithm ID.

Key ID - The Key ID of the key used to generate the MAC. The format for this field is defined in the Primitive Field Definition for Key ID.

MAC - The Message Authentication Code for the message. The format for this field is defined in Section 13.5.1.3.

13.5.1.3 DES[15] Message Authentication Code (MAC) Primitive Field

The Message Authentication Code consists of 4 octets. The MAC Primitive Field Definition is shown in Table 85.

Table 85 - MAC Format

Octet		7	6	5	4	3	2	1	0
	0	MAC Output Bits O1-O8							
	1	MAC Output Bits O9-O16							
	2	MAC Output Bits O17-O24							
	3	MAC Output Bits O25-O32							

[15] DES is only utilized for backward compatibility, see NOTE in section 1.1.

MAC Output Bits O1-O8 - This field contains the first octet of the DES Output register as shown in Figure 44. The MSB (b_7) contains the 1st output bit (O1) and the LSB (b_0) contains the 8th output bit (O8).

MAC Output Bits O9-O16 - This field contains the second octet of the DES Output register as shown in Figure 44. The MSB (b_7) contains the 9th output bit (O9) and the LSB (b_0) contains the 16th output bit (O16).

MAC Output Bits O17-O24 - This field contains the third octet of the DES Output register as shown in Figure 44. The MSB (b_7) contains the 17th output bit (O17) and the LSB (b_0) contains the 24th output bit (O24).

MAC Output Bits O25-O32 - This field contains the fourth octet of the DES Output register as shown in Figure 44. The MSB (b_7) contains the 25th output bit (O25) and the LSB (b_0) contains the 32nd output bit (O32).

13.5.2 Enhanced Message Authentication

Enhanced message authentication is calculated using either the Cipher Block Chaining Message Authentication Code (CBC-MAC) method or the Cipher-based Message Authentication Code (CMAC) method.

Messages authenticated using the enhanced CBC-MAC mode of operation shall be implemented as defined in section 13.5.2.1.

Messages authenticated using the enhanced Cipher-based Message Authentication Code (CMAC) mode of operation shall be implemented as defined in section 13.5.2.2.

13.5.2.1 Enhanced Message Authentication using CBC-MAC

Messages authenticated using the enhanced Cipher Block Chaining Message Authentication Code (CBC-MAC) mode of operation shall be implemented as defined in this section. Figure 45 shows an example of the CBC-MAC process.

The message data to be authenticated includes all of the octets in the KMM starting with the Message ID field and continues through the MAC frame, however with the exception of the MAC field. The MAC field in the MAC frame is not included in the MAC calculation. The sum of the message data octets to be authenticated becomes the length of data variable used to derive the MAC.

When generating a CBC-MAC, the input block is initialized with the first n-bit block of data ($Plaintext_1$). For KMMs this first input block would start with the Message ID, Message Length, Message Format, etc., where the Message ID octet would be the left most octet in the encryption block. This input block is encrypted using the key chosen for authentication and results in an n-bit output block ($Output\ Block_1$). This ciphertext output block is bit-wise exclusive-ored with the second n-bits of plaintext data to be authenticated. The result of this operation is loaded into the input register ($Input\ Block_2$) of the encryption algorithm. This block is encrypted and the resultant output block is exclusive-ored with the third n-bits of plaintext data to be authenticated. This process continues until all of the octets in the KMM to be authenticated are processed.

If the message does not consist of an integral number of n-bit data blocks, the final partial data block ($Plaintext_n$) shall be left justified and 0's appended to fill out a full data block. This padded data block is then exclusive-ored with the current output ciphertext block ($Output\ Block_{n-1}$). The resultant block is input into the encryption algorithm and encrypted to produce a final output block. The leftmost t-bits of this final block are defined as the MAC. All ciphertext generated by this process is discarded. This identical process is used to compute a MAC on the received message. The calculated MAC is then compared to the received MAC and if they are identical the message has been positively authenticated.

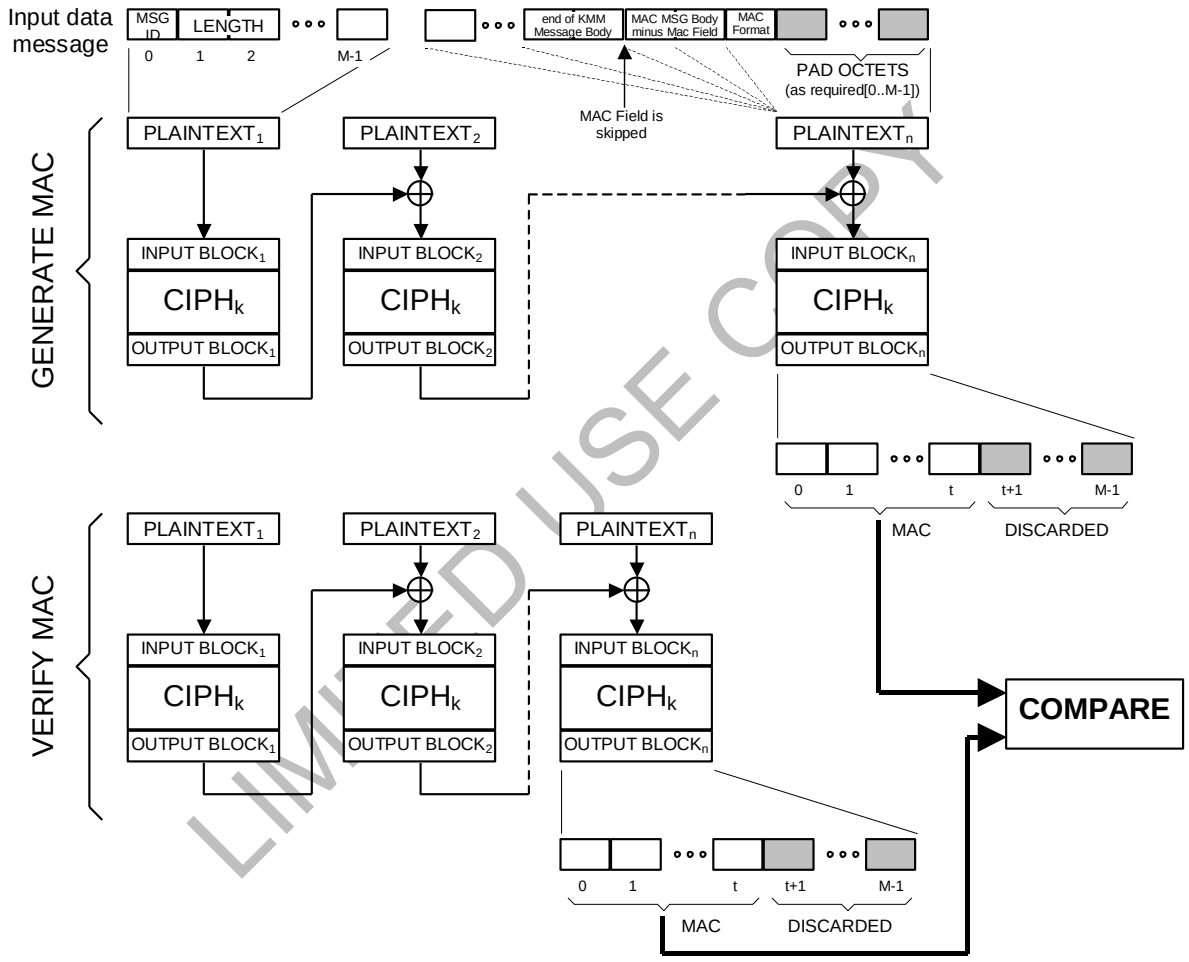


Figure 45 - Message Authentication

13.5.2.1.1 Enhanced Message Authentication Key Requirements for CBC-MAC

The key used to generate the CBC-MAC shall be either a TEK reserved exclusively for authentication purposes (ie. A MAC key) or derived from any TEK held in common between the originator and destination(s). Warm Start messages are authenticated

using a key derived from the TEK contained in the message (after it is decrypted). The Unique Key ID field in the MAC block is undefined for Warm Start messages and Reverse Warm Start segments. The SU only accepts messages that have a valid MAC and MN (exceptions to this rule are defined in Section 13.5).

13.5.2.1.2 Enhanced MAC Key Derivation for CBC-MAC

The key wrap algorithm described in the previous section shall be used to derive a MAC key from a TEK. The inputs to the key wrapping algorithm are the TEK and the length of the data being authenticated. The TEK shall be used for both the KEK and the plaintext inputs (P_1, …, P_x) to the algorithm. If the TEK is not an integer multiple of n/2 bits then the pad bits added to the key shall be set to 0. The length of the data in octets that is being authenticated (see section 13.5.2 for calculating this value) shall be right justified and used as the IV. The resulting ciphertext (C_1, …, C_x) shall be used as the MAC key. The C_0 output shall be discarded. If the key is shorter than the resulting ciphertext the left most bits shall be used starting with C_1.

When deriving a MAC key for Triple DES, the block size (n) is set at 64 bits (AES is 128 bits) and normal DES[16] key formatting applies to the resultant derived MAC key where each of the eight octets includes an odd parity bit.

13.5.2.2 Enhanced Message Authentication using CMAC

Messages authenticated using the enhanced CMAC mode of operation shall be implemented as defined in this section.

The MAC algorithm shall be AES256 CMAC as defined in reference SP800-38B [25]. The Key Derivation Function (KDF) shall be KDF in Counter mode as specified in section 13.5.2.2.2.

For a description of the CMAC algorithm process and an algorithm data block diagram, the reader is referred to SP800-38B [25].

13.5.2.2.1 Enhanced Message Authentication Key Requirements for CMAC

When using the CMAC method to calculate the KMM MAC, the MAC key shall be derived from any TEK held in common between the originator and the destination(s). The key derivation process for the CMAC method is defined in section 13.5.2.2.2.

Warm Start messages are authenticated using a key derived from the TEK contained in the message (after it is decrypted). The Unique Key ID field in the MAC block is undefined for Warm Start messages and Reverse Warm Start segments.

The SU only accepts messages that have a valid MAC and MN (exceptions to this rule are defined in Section 13.5).

13.5.2.2.2 Enhanced MAC Key Derivation for CMAC

The Key Derivation Function (KDF) shall be 'KDF in Counter mode' as defined in section 4.1 of SP800-108r1 [24].

When using 'KDF in Counter mode', the Pseudo Random Function (PRF) shown in step 4a of [24] section 4.1 shall be HMAC-SHA-256 as defined in rfc4634 [25]. The input parameters to the PRF are K_{IN}, a counter (i), Label, a separator, Context, and L, and are concatenated as described in [24] section 4.1. These parameters shall be defined as follows:

K_{IN} = the TEK (see section 13.5.2.2.1),

i = the counter value. *i* shall have a binary representation that is 1-byte, big-endian with a fixed binary value of %00000001.

Label = a fixed ASCII string "OTAR MAC" (8 bytes in length with no NULL terminator),

separator = a 1-byte fixed value of 0x00,

Context = the concatenation of several KMM header fields. There are two different concatenation approaches based on whether or not the MN bits in the Message Format field (Octet 3) of the header indicate the presence of a message number. If the MN bits in the Message Format field are "10" (indicating a message number is present), then the context shall be:

MessageID||MessageLength||MessageFormat||DestinationID||SourceID||MessageNumber

If the MN bits in the Message Format field are "00" (indicating a message number is not present), then the context shall be:

MessageID||MessageLength||MessageFormat||DestinationID||SourceID

L = the length of the output in bits. L shall be a 2-byte fixed value of 256 (0x01, 0x00).

13.5.2.3 Enhanced Message Authentication Code (MAC) Frame

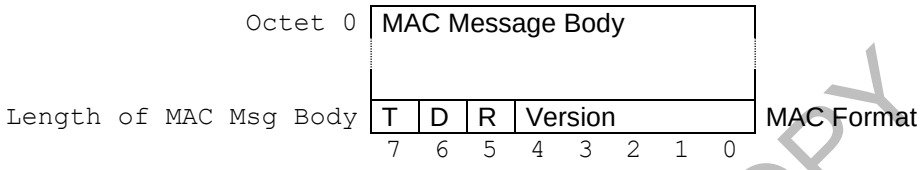
The Enhanced Message Authentication Code shall be used when the MAC bits in the Message Format field of the Message Header are binary %10 as defined in Section 10. The Enhanced MAC Frame is used to identify characteristics of the enhanced MAC, including whether the CBC-MAC or CMAC authentication method is being used.

The enhanced MAC frame shall consist of a specific block of octets (MAC Message Body) as defined by the Enhanced MAC version number. The version number is contained in the last octet (the MAC Format) along with a set of control bits (Table 86).

The version number is used to indicate either the CBC-MAC or CMAC mode of enhanced message authentication.

The MAC field shall always be the first field in the body of the MAC Message. The MAC Format octet is always the last octet in an authenticated message so the message may be authenticated before parsing the entire message. This was done since the MAC Message Body is a variable size that cannot be determined until the version number is known. The MAC Format octet may be found by knowing the length of the message from the KMM header.

Table 86 - MAC Message Structure



MAC Message Body – This field shall be uniquely defined by the encryption type and the version fields. The Encryption Type field and the Version field select the definition of the MAC Message Body.

MAC Format – This octet identifies the encryption type used to generate the MAC, if the MAC key should be derived from the key specified and a version number.

T (Encryption Type) – The encryption type bit is used to indicate if a Type 1 or Type 3 encryption algorithm was used to generate the MAC. The usage of this bit has been deprecated and shall be set to 0.

D (Derived Key) – The usage of this bit has been deprecated and shall be set to 1.

R (Reserved) – This bit is reserved for future use and shall be set to 0.

Version – These 5 bits define which version of the MAC message body is used.

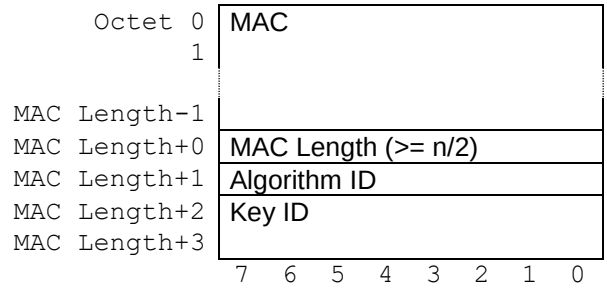
A value of 0 (%00000) indicates a CBC-MAC message body (section 13.5.2.3.1).

A value of 1 (%00001) indicates a CMAC message body (section 13.5.2.3.2).

13.5.2.3.1 MAC Message Body Definition Version 0 (%00000)

The MAC Message Body Definition for Version 0 shall be used when the MAC Format octet has the Encryption Type bit set to 0 and the version number is set to %00000. A Version 0 MAC Message Body is used for enhanced authentication when using CBC-MAC as the authentication method. The definition for the MAC message body is shown in Table 87.

Table 87 - MAC Message Body Format (Version 0)



MAC Message Body Format

MAC – The MAC field contains the output of the Message Authentication procedure as shown in Figure 45. The first octet in the MAC field shall contain the first octet of the MAC. The second octet in the MAC field shall contain the second octet of the MAC. This process continues for the number of octets as defined by the MAC Length field.

MAC Length – An 8-bit binary number used to indicate the number of octets in the MAC field. This field shall be set to 0x08 (indicating 8 octets) for enhanced AES256 CBC-MAC.

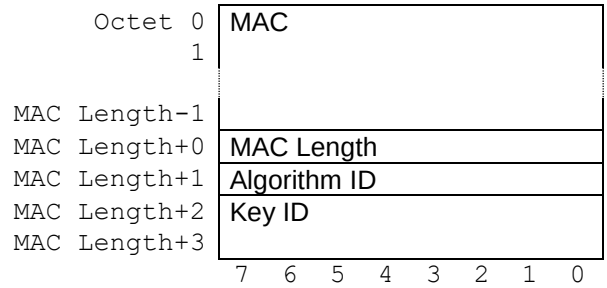
Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select the key used to authenticate the message. These fields are used to select the TEK used to derive the MAC key. The format for this field is defined in the Primitive Field Definition for Algorithm ID.

Key ID - The Key ID is used in conjunction with the Algorithm ID to uniquely select the key used to authenticate the message. These fields are used to select the TEK used to derive the MAC key. The format for this field is defined in the Primitive Field Definition for Key ID.

13.5.2.3.2 MAC Message Body Definition for Version 1 (%00001)

The MAC Message Body Definition for Version 1 shall be used when the MAC Format octet has the version number set to %00001. A Version 1 MAC Message Body is used for enhanced authentication when using CMAC as the authentication method. The definition for the MAC message body is shown in Table 88 - MAC Message Body Format (Version 1)Table 88.

Table 88 - MAC Message Body Format (Version 1)



MAC Message Body Format

MAC – The MAC field contains the output of the Message Authentication procedure as described in section 13.5.2.2. The first octet in the MAC field shall contain the first octet of the MAC. The second octet in the MAC field shall contain the second octet of the MAC. This process continues for the number of octets as defined by the MAC Length field.

MAC Length – An 8-bit binary number used to indicate the number of octets in the MAC field. This field shall be set to 0x08 (indicating 8 octets) for enhanced AES256 CMAC.

Algorithm ID - The Algorithm ID is used in conjunction with the Key ID to uniquely select the key used to authenticate the message. These fields are used to select the TEK used to derive the MAC key. The format for this field is defined in the Primitive Field Definition for Algorithm ID.

Key ID - The Key ID is used in conjunction with the Algorithm ID to uniquely select the key used to authenticate the message. These fields are used to select the TEK used to derive the MAC key. The format for this field is defined in the Primitive Field Definition for Key ID.

13.5.3 KMF MAC Validation

The KMF shall validate the MAC by calculating a MAC over the received KMM. A MAC is considered valid if the KMF calculated MAC and the KMM specified MAC is identical. If the calculated MAC does not match the received MAC, the KMF shall respond as follows:

- If the KMM was sent as a Response Kind 3 (Immediate), the KMF shall format a Negative-Acknowledgement and send it to the SU. The status field shall be set to Invalid MAC (0x04).
- If the KMM was sent as a Response Kind 2 (Delayed), no OTAR response is sent back to the SU.
- If the KMM was sent as a Response Kind 1 (None), no OTAR response is sent back to the SU.

If the key used to generate the MAC does not exist, the KMF shall respond as follows:

- If the KMM was sent as Response Kind 3 (Immediate), the KMF shall format a Negative-Acknowledgement and send it to the SU. The status field shall be set to Item (Key) Does Not Exist (0x02).

- If the KMM was sent as a Response Kind 2 (Delayed), no OTAR response is sent back to the SU.
- If the KMM was sent as a Response Kind 1 (None), no OTAR response is sent back to the SU.

13.6 **Message Number (MN)**

The Message Number (MN) in the KMM is a 16-bit sequence number used to prevent message playback. Most KMMs sent by the KMF or SU contain a message number.

MNs are sent both inbound and outbound (as viewed from the KMF perspective). Outbound message numbers are used for OTAR procedures originated at the KMF and sent to the SU. Inbound message numbers are used for OTAR procedures originated at the SU and sent to the KMF.

The KMF shall assign and keep track of an outbound (KMF to SU) MN for each individual RSI to which it sends a KMM. The KMF shall keep track of an inbound (SU to KMF) MN for each individual RSI for which it receives a KMM; i.e., independent inbound and outbound message numbers are associated with each individual RSI. The KMF also assigns and keeps track of an outbound MN for each group RSI and All Call RSI (0xFFFFFF) for which it sends a KMM.

The SU shall assign and keep track of an inbound (SU to KMF) MN for the KMF RSI to which it sends a KMM. The SU shall keep track of an outbound (KMF to SU) MN for its own individual RSI. It also keeps track of an outbound MN for each group RSI it contains, as well as an outbound MN for the All Call RSI (0xFFFFFF).

When the SU sends a response to a KMF command, the SU uses the MN received in the KMF command and inserts it into the header MN field of the response message.

13.6.1 Outbound Message Number Processing

Key Management Messages shall contain a 2-octet Message Number (MN) except in the following cases:

- Capabilities-Command. It is recommended that the Capabilities-Command contain a Message Number, however, it may be sent without a Message Number at the manufacturer's discretion.
- Capabilities-Response. The Capabilities-Response shall contain a Message Number if the Capabilities-Command was received with a Message Number. If the Capabilities-Command is received without a MN then the response may be sent back without a MN.
- Delayed Acknowledgement. The Delayed Acknowledgement response does not require a Message Number in the KMM header since the required Message Number information is contained in the body of the message.
- Hello. The Hello message is typically sent without a MN.

- No Service. The No Service message is typically sent without a MN.

Any message, including the messages listed above, which contain a MN, shall contain a MAC. A message is discarded if it is received containing a MN and does not include a MAC.

The KMF increments the destination RSI outbound MN for each new message that it creates. It is the responsibility of the receiver to validate the MN. The SU maintains an MN for its own individual RSI, and for each group RSI that it is a member of. The SU uses the MN in the command of the procedure as the MN for the response. The message number is a modulo 2^{16} value and rolls over to 0 after reaching 65,535.

13.6.2 Inbound Message Number Processing

Messages originated from an SU shall use an inbound message number as defined in this section.

KMMs shall adhere to the following rules for Inbound Message Numbers:

- Deregistration-Command. It is recommended that the Deregistration-Command contain a MN, however, it may be sent without a MN at the manufacturer's discretion.
- Deregistration-Response. The Deregistration-Response shall contain a MN if the Deregistration-Command that was received contains a MN. If the Deregistration-Command is received without an MN then the response may be sent back without an MN.
- Registration-Command. It is recommended that the Registration-Command contain a MN, however, it may be sent without a MN at the manufacturer's discretion.
- Registration-Response. The Registration-Response shall contain a MN if the Registration-Command that was received contains a MN. If the Registration-Command is received without an MN then the response may be sent back without an MN.
- Unable-to-Decrypt Response. It is recommended that the Unable-to-Decrypt Response contain a MN, however, it may be sent without a MN at the manufacturer's discretion. If the MN is included, the MN shall be the inbound message number.
- Rekey-Request (SU initiated Hello). It is recommended that the SU Initiated Hello message (Rekey-Request) contain a MN, however, it may be sent without a MN at the manufacturer's discretion.

Any message listed above which contains a MN shall contain a MAC. A message is discarded if it is received containing a MN and does not include a MAC. The KMF and SU only accept messages that have a valid MN and MAC.

The SU shall keep track of an inbound MN (SU to KMF) in non-volatile memory for the KMF RSI for which it sends a KMM. For any procedure initiated by the SU that requires an MN, the SU shall use and increment the MN it has stored for the indicated KMF RSI.

It shall be the responsibility of the KMF to validate the MN. This validation involves ensuring the received MN falls within the range of allowable values (see Section 13.6.4).

For each individual SU RSI that a KMF manages, the KMF shall maintain a record of the last valid received inbound MN. When sending a response to an SU command, the KMF uses the inbound MN received in the SU command and inserts it into the MN field of the response.

Whenever an SU initiated Response Kind 1 (None) message addressed to a KMF is retransmitted, it is retransmitted with a new MN.

13.6.3 Message Number Synchronization

The message number for Key Management applications is a modulo 2^{16} value and rolls over to 0 after reaching 65,535. The inbound MN associated with the KMF RSI within an SU is initialized to zero when the SU is first put into service. The inbound MN may also be changed via the Change RSI command, which also affects the outbound MN, setting both MNs to the same value.

If the KMF receives a message from an SU with an incorrect MN and chooses to correct the MN, the KMF SHALL either send an encrypted NACK with the invalid MN error code, or execute the Change-RSI procedure to adjust the MNs in the SU. If the KMF chooses not to correct the MN, then the KMF should discard the message. If the KMF sends a NACK, the KMF is indicating that the inbound MN is out of sync. If the SU receives a NACK with an invalid MN error indication, the SU is then permitted to synchronize its inbound MN with the MN in the NACK.

When the SU receives a message from a KMF with an incorrect MN, the SU SHALL respond as described in #6 of Section 7.4. If the SU sends a NACK with an invalid MN error indication, the SU is indicating that the outbound MN is out of sync. If the KMF receives a NACK with an invalid MN error indication, the KMF MAY execute the Change-RSI to reset the MNs at the SU, or the KMF MAY synchronize its outbound MN with the MN in the NACK.

If an SU reaches the point where MNs are out of sync and the SU has no TEKs in common with the KMF, then the “New Unit” Procedure should be executed to resynchronize the MNs.

13.6.4 Message Number Validation

The MN validation process is used to determine if the MN received is acceptable. If the received MN is within the valid message number range, the KMM is accepted; if the MN is not valid, the KMM is discarded.

For Response Kind 1 (None) messages and Response Kind 3 (Immediate) non-retry messages, the MN is validated using RULE 1.

RULE 1:

If $(MNL + MNP) \bmod 2^{16} < MNL$
then $MNR > MNL$ **or** $MNR < (MNL + MNP) \bmod 2^{16}$

If $(MNL + MNP) \bmod 2^{16} > MNL$
then $MNR > MNL$ **and** $MNR < (MNL + MNP) \bmod 2^{16}$

where: MNP is the Message Number Period
MNL is the Last valid Message Number
MNR is the Received Message Number

For Response Kind 2 (Delayed) messages and Response Kind 3 (Immediate) retry messages, the MN is validated using RULE 2.

RULE 2:

If $(MNL + MNP) \bmod 2^{16} < MNL$
then $MNR >= MNL$ **or** $MNR < (MNL + MNP) \bmod 2^{16}$

If $(MNL + MNP) \bmod 2^{16} > MNL$
then $MNR >= MNL$ **and** $MNR < (MNL + MNP) \bmod 2^{16}$

Where: MNP is the Message Number Period
MNL is the Last valid Message Number
MNR is the Received Message Number

RULE 2 for Response Kind 3 (Immediate) retry messages is only applicable for MNs in back-to-back identical Response Kind 3 (Immediate) messages. Any deviation from this requires incremental MNs, and therefore RULE 1 would apply.

Figure 46 illustrates the message number validation process. The circle represents the total MN space from 0 to 65,535 in a clockwise direction. Since MNs are used to guard against message playback (spoofing), a period of MNs is identified as MNP. The MNP is equal to:

MNP = (Rate * Period) * M

where: Rate is the expected OTAR messaging rate.
Period is the maximum time between receiving a message.
M is a factor to provide a margin of safety.

This period (MNP), as defined on the circle, represents the possible message numbers that would be valid if received. For example, if the Rate is 5 messages per hour, the period is 1 week and a safety factor of 2 is used, the MNP is set to 1680.

For SUs, MNP programming is described in [AACD]. For KMFs, the MNP programming is manufacturer-specific.

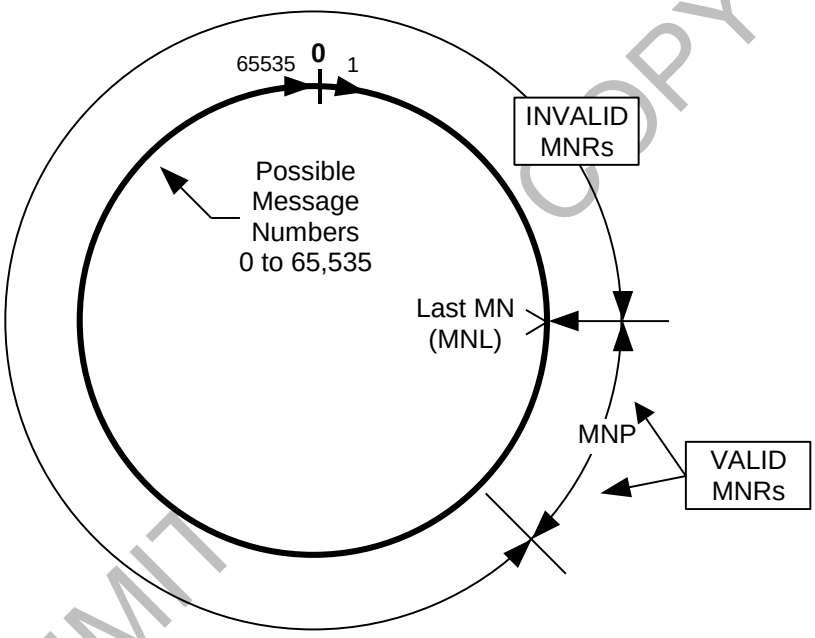


Figure 46 - Message Number Validation

13.6.5 Message Number Primitive Field

If a KMM includes a MN, the two octet Message Number field shall be used. The value is a binary number representing values from 0 to 65,535 decimal. The bits are

labeled from MN(15..0), where MN(15) is the MSB and MN(0) is the LSB. The Message Number primitive field definition is shown in Table 89.

Table 89 - Message Number Format

		7	6	5	4	3	2	1	0
Octet	0	Message Number MN(15..8)							
	1	Message Number MN(7..0)							

Message Number MN(15..8) - This field contains the MS octet of the Message Number. The MSB (b_7) contains the Message Number bit 15 and the LSB (b_0) contains Message Number bit 8.

Message Number MN(7..0) - This field contains the LS octet of the Message Number. The MSB (b_7) contains the Message Number bit 7 and the LSB (b_0) contains Message Number bit 0.

14 ALGORITHMS

14.1 Data Encryption Standard (DES[17]) also known as Data Encryption Algorithm (DEA)

14.1.1 Algorithm ID and Description

An Algorithm ID of $81 (see [5]) shall indicate an encrypted message using the Data Encryption Standard. The DES algorithm is a 64-bit block algorithm (n = 64) with a 56-bit key and 8 parity bits to make up a 64-bit key variable (k = 64). A complete description of this algorithm is given in [10].

Summary of parameters:

n = 64
k = 64
r = 0

14.1.2 MAC Requirements

The DES algorithm shall use the Message Authentication Code described in Section 13.5.1. The MAC key may be any DES traffic key held in common between the KMF and the SU and is specified by the Algorithm ID and Key ID fields in the MAC Message Body. The MAC shall be truncated to 32 bits (t=32).

14.1.3 Key Encryption Example

Given:

KEK = $1313 1313 1313 1313
UKEK = $1313 1313 1313 1313
CKEK = $1313 1313 1313 1313

and

TEK = $1313 1313 1313 1313

and

Key1 = $1313 1313 1313 1313
Key2 = $1313 1313 1313 1313
Key3 = $1313 1313 1313 1313

the encryption of Key1, Key2, and Key3 with the KEK produces the following:

$E(Key1)_{KEK}$ = $ 2911 CF5E 94D3 3FE1
$E(Key2)_{KEK}$ = $ 2911 CF5E 94D3 3FE1

[17] DES is only utilized for backward compatibility, see NOTE in section 1.1.

$E(Key3)_{KEK}$ = $ 2911 CF5E 94D3 3FE1

14.1.4 MAC Generation Example

The following Rekey-Command Message is used as an example of a message being authenticated with a MAC. The MAC key is the same as the traffic key used to encrypt the message. The key used to authenticate the message is therefore:

TEK = $1313 1313 1313 1313

The following is an example of a Rekey-Command message which contains 5 keys in 4 keysets:

Message ID:	$1E
Message Length	$0082
Message Format:	$AC
Destination ID:	$000001
Source ID:	$98967F
Message Number:	$0009
Decryption Instruction Format:	$00
Algorithm ID:	$81
Key ID:	$F5A0
Number of Keysets:	$04
Keyset Format:	$08 <Keyset 1>
Keyset ID:	$01
Algorithm ID:	$81
Keyset Name:	$5345542030303320
Key Length:	$08
Number of Keys:	$01
Key Format:	$00
Storage Location Number:	$0003
Key ID:	$0002
Key [$E(Key1)_{KEK}$]:	$2911 CF5E 94D3 3FE1 <Key1>
Keyset Format:	$08 <Keyset 2>
Keyset ID:	$02
Algorithm ID:	$81
Keyset Name:	$5345542030303320
Key Length:	$08
Number of Keys:	$01
Key Format:	$00
Storage Location Number:	$0003
Key ID:	$0002
Key [$E(Key2)_{KEK}$]:	$2911 CF5E 94D3 3FE1 <Key2>
Keyset Format:	$08 <Keyset 3>
Keyset ID:	$03
Algorithm ID:	$81
Keyset Name:	$5345542030303320
Key Length:	$08
Number of Keys:	$01
Key Format:	$00

Storage Location Number:	$0003
Key ID:	$0002
Key [E(Key3)$_{KEK}$]:	$2911 CF5E 94D3 3FE1 <Key3>
Keyset Format:	$08 <Keyset 4>
Keyset ID:	$FF
Algorithm ID:	$81
Key Length:	$08
Number of Keys:	$02
Key Format:	$80
Storage Location Number:	$F000
Key ID:	$F5A0
Key [E(UKEK)$_{KEK}$]:	$2911 CF5E 94D3 3FE1 <UKEK>
Key Format:	$80
Storage Location Number:	$F001
Key ID:	$F5A1
Key [E(CKEK)$_{KEK}$]:	$2911 CF5E 94D3 3FE1 <CKEK>
MAC Algorithm ID:	$81
MAC Key ID:	$0002

The resultant MAC is therefore:

MAC = $B744 24C7

The data block for the rekey message before being encrypted is shown below. The last 4 octets are the MAC.

Data Block:

1E00 82AC 0000 0198 967F 0009 0081 F5A0
0408 0181 5345 5420 3030 3320 0801 0000
0300 0229 11CF 5E94 D33F E108 0281 5345
5420 3030 3320 0801 0000 0300 0229 11CF
5E94 D33F E108 0381 5345 5420 3030 3320
0801 0000 0300 0229 11CF 5E94 D33F E108
FF81 0802 80F0 00F5 A029 11CF 5E94 D33F
E180 F001 F5A1 2911 CF5E 94D3 3FE1 8100
02B7 4424 C7

The MI is:

$35E6 4D66 1F56 353C

Using the above parameters, the encrypted block output is therefore;

3D32 2BF0 0877 B930 ED0E F3BC 0EDB 74CE
0FEC C595 D88D A55B 9CE0 20DE 7CAD 81D5
BC46 4034 0AAA FA24 8F2F 30D5 AFDC ACE5
4B04 E8B7 C996 0AC6 5487 33BE 2968 E26D
B937 139F 570D 4316 75D5 19C9 6169 58A4

```
D13E 9767 C145 DF11 68BB 2D91 636A 67B8
279E D484 7B92 ED09 0458 6813 3693 80EA
72D2 48BA 0B5C 1415 4E01 6EE9 D510 9F88
919D CD02 0A
```

14.2 Three Key Triple Data Encryption Algorithm (TDEA) also known as Triple Data Encryption Standard (Triple DES)

14.2.1 Algorithm ID and Description

TDEA uses 3 chained DES (see Section 14.1) encryptions/decryptions with a key bundle consisting of separate 64-bit DES key variables (K_1, K_2, and K_3) for each encryption/decryption. The basic encryption operation can be described as

$$\text{Output} = E_{K3}(D_{K2}(E_{K1}(\text{Input})))$$

where E stands for a DES encryption operation and D stands for a DES decryption operation. Since the DES operations are chained, the size of the input and output registers is identical to DES, n = 64. A complete description of this algorithm is given in [15].

An Algorithm ID of $83 (see [2]) indicates the three-key version of TDEA (keying option 1 in [11]). There are three independent 64-bit key variables, K_1, K_2, and K_3 (every 8th bit going from the left to the right is a parity bit), for a total k = 192).

Summary of parameters:

n = 64
k = 192
r = 0
L = 24
M = 8

14.2.2 Enhanced MAC Requirements

The Triple DES algorithm shall use the Enhanced Message Authentication Code defined in Section 13.5.2. The MAC key shall be derived from the traffic key specified by the Algorithm ID and Key ID fields in the MAC Message Body. The MAC shall be truncated to n/2 or 32 bits (t=32).

Summary of MAC parameters:

T = 0
D = 1
R = 0
Version = 0
MAC Length = 4

14.2.3 Key Encryption Example

Given:

TEK_1 = $2A19 38CD 0B6B 6BD0
TEK_2 = $A43D 5145 C4AB 3E1C
TEK_3 = $641A 8932 1A1C DA37

and

KEK_1 = $4940 02BF 1631 32A4
KEK_2 = $1FA2 8092 4C43 FBFD
KEK_3 = $86EA 8ACD 436B AE13

the following encrypted key frame is produced:

$E(TEK)_{KEK}$ =	$56E0 871C	(C_0)
	$6104 1C93 428E 962F	(C_1|C_2)
	$227D 96CA 9419 395A	(C_3|C_4)
	$61D3 BE3C 78FD D354	(C_5|C_6)

14.2.4 MAC Generation Example

The Rekey Command Message is used here as an example of a message being authenticated with a MAC. The encrypted key $E(TEK)_{KEK}$ calculated in the above example is used here in the authenticated rekey message. The key used to authenticate the message is:

TEK_1 = $1685 6245 3B3E 7F61
TEK_2 = $1934 6D80 37C2 2F9D
TEK_3 = $86EA 8ACD 436B AE13.

Given the length of the data is $003C (see Section 13.5.2 on how this is determined), the derived MAC key (after corrected for parity) is:

MAC Key_1 = $8AE0 2A54 32EA C4D9	(C_1|C_2)
MAC Key_2 = $851F 75D6 62AB 1AAE	(C_3|C_4)
MAC Key_3 = $0E61 F8CE 0746 5215	(C_5|C_6)

The following parameters shall be used for the Rekey Command message:

Message ID:	$1E
Message Length	$003D
Message Format:	$A8
Destination ID:	$643BA8
Source ID:	$712B1D
Message Number:	$1772
Decryption Instruction Format:	$00
KEK Algorithm ID:	$83
KEK Key ID:	$50BC
Number of Keysets:	$01
Keyset Format:	$00
Keyset ID:	$01
Algorithm ID:	$83
Key Field Length:	$1C
Number of Keys:	$01
Key Format:	$00
Storage Location Number:	$0000
Key ID:	$4983
Key [$E(TEK)_{KEK}$]:	$56E0 871C 6104 1C93 $428E 962F 227D 96CA $9419 395A 61D3 BE3C $78FD D354
MAC Length:	$04
MAC Algorithm ID:	$83
MAC Key ID:	$2F62
MAC Format:	$40

The resultant MAC shall be:

MAC = $2603 9DD5

The data block for the rekey message before being encrypted is shown below (MAC bytes are shown boxed).

Data Block:

1E00 3DA8 643B A871 2B1D 1772 0083 50BC
0100 0183 1C01 0000 0049 8356 E087 1C61
041C 9342 8E96 2F22 7D96 CA94 1939 5A61
D3BE 3C78 FDD3 54[26 039D D5]04 832F 6240
#

The MI is:

$7030 F1F7 6569 2667

Using the above parameters, the encrypted output block is therefore;

8F04 7D2C C2DA AA99 07EB 21F4 234F 19D0

```
E3F8 9D73 385A 4C72 0500 FD25 B69A 7DF7
921A 3B00 6125 02AC BCE7 F4D6 7B4F 361E
80EC 0630 D51A 7106 BE0E 3935 1606 164B
```

14.3 Advanced Encryption Standard (AES)

14.3.1 Algorithm ID and Description

An Algorithm ID of $84 shall indicate an encrypted message using the Advanced Encryption Standard. The AES algorithm is a 128-bit block algorithm (n = 128) with a 256-bit key variable (k = 256). A complete description of this algorithm is given in the document *Advanced Encryption Standard*, FIPS Publication 197, November 26, 2001 (reference [12]).

Summary of parameters:

n = 128
k = 256
r = 0
L = 40
M = 16

14.3.2 Enhanced MAC Requirements

When an enhanced MAC is used, the Version field of the MAC Format octet determines whether the enhanced MAC is CMAC or CBC-MAC (Table 86).

When the Version field indicates CMAC, section 13.5.2.3.2 shall be followed.

When the Version field indicates CBC-MAC, the following applies.

The AES algorithm shall use the Enhanced Message Authentication Code specified in Section 13.5.2. The MAC key shall be derived from the traffic key specified by the Algorithm ID and Key ID fields in the MAC Message Body. The MAC shall be truncated to n/2 or 64 bits (t=64). For CMAC, truncation is accomplished by setting the T_{len} parameter to 64-bits.

Summary of CBC-MAC parameters:

T = 0
D = 1
R = 0
Version = 0
MAC Length = 8

Summary of CMAC parameters:

T = 0
D = 1
R = 0
Version = 1
MAC Length = 8

14.3.3 Key Encryption Example

Given:

TEK = 2A19 38CD 0B6B 6BD0 B774 5692 FE19 14F0
3876 612F C29D 5777 89A6 2F65 FA05 EF83

and

KEK = 4940 02BF 1631 32A4 21FB EF11 7F98 5A0C
AADD C250 A4C2 1947 D593 E6C0 67DE 402C

The following encrypted key frame is produced:

$E(TEK)_{KEK}$ = 8028 9CF6 35FB 68D3 45D3 4F62 EF06 3BA4
E05C AE47 56E7 D304 46D1 F07C 6EB4 E9E0
8409 4537 2372 FB80

14.3.4 CBC-MAC Generation Example

The Rekey Command Message is used here as an example of a message being authenticated with a MAC. The encrypted key $E(TEK)_{KEK}$ calculated in the above example is used here in the authenticated rekey message. The key used to authenticate the message is:

TEK = 1685 6245 3B3E 7F61 8D68 B387 E0B9 97E1
FB0F 264F A83B 74E4 3B17 2917 BD39 339F

Given the message length is $0048 (see Section 13.5.2 on how this is determined), the derived MAC key is:

MAC Key = 09E7 117B 4E42 06DE D366 EA5D 6933 01CA
8321 BCC2 0FA5 05DF 1267 DC2A E458 A057

The following parameters shall be used for the Rekey Command message:

Message ID:	$1E
Message Length	$004D
Message Format:	$A8
Destination ID:	$643BA8
Source ID:	$712B1D
Message Number:	$1772
Decryption Instruction Format:	$00

KEK Algorithm ID:	$84
KEK Key ID:	$50BC
Number of Keysets:	$01
Keyset Format:	$00
Keyset ID:	$01
Algorithm ID:	$84
Key Field Length:	$28
Number of Keys:	$01
Key Format:	$00
Storage Location Number:	$0000
Key ID:	$4983
Key [E(TEK)$_{KEK}$]:	$8028 9CF6 35FB 68D3 $45D3 4F62 EF06 3BA4 $E05C AE47 56E7 D304 $46D1 F07C 6EB4 E9E0 $8409 4537 2372 FB80
MAC Length:	$08
MAC Algorithm ID:	$84
MAC Key ID:	$2F62
MAC Format:	$40

The resultant MAC shall be:

MAC = $42A0 9156 F0D4 721C

The data block for the rekey message before being encrypted is shown below (MAC bytes are shown boxed).

Data Block:

1E00 4DA8 643B A871 2B1D 1772 0084 50BC
0100 0184 2801 0000 0049 8380 289C F635
FB68 D345 D34F 62EF 063B A4E0 5CAE 4756
E7D3 0446 D1F0 7C6E B4E9 E084 0945 3723
72FB 80**42 A091 56F0 D472 1C**08 842F 6240
#

The MI is:

$7030 F1F7 6569 2667

Using the above parameters, the encrypted output block is therefore;

6775 B1D1 8ABD CF86 0854 DF09 8EA3 4129
132A 0E48 4CCC 5CAE 8008 0B19 F708 AE8F
B840 AA2E 3E5E CD03 7352 75FE E288 0E6D
DD00 C111 428F EE39 C62B F3C1 D2EE 3BEB
BB7C 44A5 E3C9 308C 5DE9 1784 7C17 AF23

14.3.5 CMAC Generation Examples

14.3.5.1 CMAC example with a message number

The Rekey Command Message is used here as an example of a message being authenticated with CMAC when a message number is present (i.e. the MN bits in the Message Format field are equal to '10'). The key used to authenticate the message is:

TEK = 1685 6245 3B3E 7F61 8D68 B387 E0B9 97E1
FB0F 264F A83B 74E4 3B17 2917 BD39 339F

Label = 4F54 4152 204D 4143

Context = 1E00 4DA8 643B A871 2B1D 1772

The derived CMAC key is:

CMAC Key = 5FB2 91D0 9EE3 991E 131A 04B0 E3A0 BF58
B4A1 CE46 1048 EB14 B497 AE95 22D0 0D31

The following parameters shall be used for the Rekey Command message:

Message ID:	$1E
Message Length	$004D
Message Format:	$A8
Destination ID:	$643BA8
Source ID:	$712B1D
Message Number:	$1772
Decryption Instruction Format:	$00
KEK Algorithm ID:	$84
KEK Key ID:	$50BC
Number of Keysets:	$01
Keyset Format:	$00
Keyset ID:	$01
Algorithm ID:	$84
Key Field Length:	$28
Number of Keys:	$01
Key Format:	$00
Storage Location Number:	$0000
Key ID:	$4983
Key [$E(TEK)_{KEK}$]:	$8028 9CF6 35FB 68D3 $45D3 4F62 EF06 3BA4 $E05C AE47 56E7 D304 $46D1 F07C 6EB4 E9E0 $8409 4537 2372 FB80
MAC Length:	$08
MAC Algorithm ID:	$84
MAC Key ID:	$2F62

MAC Format: $41

The resultant MAC shall be:

MAC = $2185 2233 41D9 8A97

The data block for the rekey message before being encrypted is shown below (MAC bytes are shown boxed).

Data Block:

1E00 4DA8 643B A871 2B1D 1772 0084 50BC
0100 0184 2801 0000 0049 8380 289C F635
FB68 D345 D34F 62EF 063B A4E0 5CAE 4756
E7D3 0446 D1F0 7C6E B4E9 E084 0945 3723
72FB 80 21 8522 3341 D98A 97 08 842F 6241

14.3.5.2 CMAC example without a message number

The Hello Command Message is used here as an example of a message being authenticated with CMAC when a Message Number is not present (i.e. the MN bits in the Message Format field are equal to '00'). The key used to authenticate the message is:

TEK = 1685 6245 3B3E 7F61 8D68 B387 E0B9 97E1
FB0F 264F A83B 74E4 3B17 2917 BD39 339F

Label = 4F54 4152 204D 4143

Context = 0C00 1508 643B A871 2B1D

The derived CMAC key is:

CMAC Key = 677B 22E2 004D 5A64 E816 8BF7 606F 7C14
2AC4 35DE 7767 E9BB EFE1 D5D6 FDE5 40B2

The following parameters shall be used for the Hello message:

Message ID:	$0C
Message Length	$0015
Message Format:	$08
Destination ID:	$643BA8
Source ID:	$712B1D
Message Number:	(field not present)
Hello Message Format:	$00
MAC Length:	$08
MAC Algorithm ID:	$84
MAC Key ID:	$2F62

MAC Format: $41

The resultant MAC shall be:

MAC = $B7C5 5411 935E 129A

The data block for the Hello message is shown below (MAC bytes are shown boxed).

Data Block:

0C00 1508 643B A871 2B1D 00B7 C554 1193
5E12 9A08 842F 6241

Annex A (Informative) Evolution of this Document

Over time, clarifications, enhancements, and corrections have occurred as part of the technical evolution of the OTAR messages and procedures. These message and procedure changes have resulted in equipment being fielded with varied implementations of OTAR. Interoperability between these varied implementations can be limited in some operational scenarios to the original OTAR standardized definitions, and therefore it's important to identify the changes between the different OTAR documents.

The following message and procedure changes have occurred between TIA-102.AACA-A and TIA-102.AACA-B.

- General editorial corrections and clarifications.
- For legacy purposes, the sections describing the DES algorithm have been footnote annotated and a descriptive note has been added to the introduction, per FPIC memo "Federal Partnership for Interoperable Communications (FPIC) Recommendation for Data Encryption Standard (DES) Reference in Project 25 (P25) Standards".
- Clarifications added to KMM protection rules (inner layer protection, outer layer protection and MAC) based on transported key, and encryption algorithm and key length.
- Incorporation of addendum AACA-A-1, which clarifies single key and multi-key subscriber interoperability.

The following message and procedure changes have occurred between TIA-102.AACA-B and TIA-102.AACA-C.

- Addition of the CMAC method for OTAR KMM authentication.
- Deprecation of the *D (Derived Key*) bit in the Enhanced Mac Frame.

Compliance to the OTAR features is determined via published test specifications specifically written in support of each published OTAR document. Equipment compliance is expressed in terms of equipment version and published test document version, and so, by association, published OTAR document version. Compliance with the TIA-102.AACA-B does not impugn the compliance of prior implementations with prior versions of the standard.

Annex B (Normative) Historical Key ID

B.1 Overview

This Annex provides a historical definition of the Key ID parameter and applies only to single-key and multi-key equipment developed prior to the publication of ANSI/TIA-102.AACA-A-1. When using mixed single-key and multi-key equipment developed prior to the publication of ANSI/TIA-102.AACA-A-1, understanding the impact of the Key ID parameter for each equipment type is needed to ensure successful interoperability.

B.2 Key ID (Historical)

The Key ID is a 2 octet field which contains a value in the range between 1 and 65535 and is used to specify either a KEK or a TEK.

The Key ID has a reserved value of $0000 which is used as a default Key ID by equipment which does not operate in a multi-key system. A value of $0000 means that a default key variable is to be used on the channel. Normally, all of the equipment on such a channel stores a single key variable and uses the default Key ID. The single key equipment then ignores any messages which originate from multi-key equipment using non-default key variables. The default Key ID value is also used for unencrypted traffic, which is signified by the reserved ALGID value.

KEY IDs for KEKs in support of multiple algorithms require special consideration. KEKs are stored in Crypto Group 15 and Keyset IDs 241-255. The KEY ID values $F5A0 and $F5A1 are reserved and may be required by legacy equipment for the UKEK Key ID and CKEK Key ID respectively. Any valid Key ID may be used for a KEK.

THE TELECOMMUNICATIONS INDUSTRY ASSOCIATION

TIA represents the global information and communications technology (ICT) industry through standards development, advocacy, trade shows, business opportunities, market intelligence and world-wide environmental regulatory analysis. Since 1924, TIA has been enhancing the business environment for broadband, wireless, information technology, cable, satellite, and unified communications.

TIA members' products and services empower communications in every industry and market, including healthcare, education, security, public safety, transportation, government, the utilities. TIA is accredited by the American National Standards Institute (ANSI).

TELECOMMUNICATIONS INDUSTRY ASSOCIATION **1310 N. COURTHOUSE ROAD, SUITE 890 ARLINGTON VA, 22201; USA** **+1.703.907.7700 MAIN +1.703.907.7727 FAX**